修士 (工学) 学位論文

Master's Thesis of Engineering

脳波を用いたデバイスコントロールに関する研究

Brain Wave Processing Modular System

2017 年 3 月
March 2017

センショーサビノ　バツェラット　ブルーノ
SENZIO-SAVINO BARZELLATO BRUNO

琉球大学大学院 理工学研究科
情報工学専攻

Information Engineering Course
Graduate School of Engineering and Science
University of the Ryukyus

指導教員：M.R.　アシャリフ
Supervisor : Mohammad Reza Alsharif

THESIS REVIEW & EVALUATION COMMITEE MEMBERS

（Chairman） M.R. Alsharif

（Committee） S. Tamaki

（Committee） M. Nakamura

要旨

商業と研究の目的のためにBCIを使用することは、人間の増強やIoTのプラットフォームもしくは感情コンピューティングのような新しい技術を組み合わせたものを安く作成するために注目を浴びている。
商業用BCIデバイスは脳からの信号を記録することができ、それを利用者に価値のあるデータとして提供する。
商業BCIデバイスの使用を広げるには脳波の知識やデバイスの使用方法を学ぶ必要がある。
また商業BCIデバイスのコストを下げるための技術の問題がある。目的は高いデバイスを安くすることだが、現実に行うにはハードウェアとソフトウェアの技術的問題に対して研究をする必要がある。
この研究の目的は安い商業BCIデバイスを用い、ソフトウェアで動くハードウェアモジュールの作成である。
モジュールは、有用な分類および信号処理アルゴリズムを使用して、高速処理の上で現在の機械学習アルゴリズムを実行できる必要がある。
更にモジュールで信号処理、高解像度グラフィックの出力、現実での操作、ホームオートメーションを同時に実行できることも必要である。
実地された実験の一つに、異なる信号の特徴抽出法だけでなく、作成したオーディオとビデオフィードバックを伴った仮想トレーニング環境から得られる信号も用い、高い認識率を得るために一つの電極のついた商業用BCIデバイスツールを使用した。
データは、特定の経路を介して仮想環境にいるキャラクターを誘導することによって得られる注目および瞑想信号からなる30個のサンプルデータから構成される。
30個のサンプルデータのうち25個が訓練データで5個がテストデータである。
教師付きの学習法では80%の認識率を示した。もう一方の教師なしの学習法では並列処理を使わない高速マルチコアプロセッサラップトップユニット上での平均処理時間は約132秒であり、60%の認識率を示した。
加えて、このデータは同様にモジュールハードウェアの実装段階での目的である組込みシステム上でリアルタイムでの分類の実装をするためにOpenMPとCUDAの方法を用いたマルチスレッド環境でも使用して、b − SOM ／ RBF − SVMの特徴分析方法で認定率は70%、7.73秒 (OpenMP) と 8.78秒 (CUDA) の実装時間でありました。
以下の論文では、被験者のめの環境開発、特徴抽出法および実装だけでなく並列処理の実装方法も掲示し、そして結果と考察について示す

Abstract

The use of Brain Computer Interfaces (BCI) for commercial and research purposes is gaining hype and impulse due to its competitive advantage and suitability for the combination of innovative technologies such as, Human Augmentation, IoT Platforms or Affective Computing. Commercial BCI devices measure signals from the brain and turn them into outputs that provide value to the end user. The success of widespread dissemination of commercial BCI devices depends on reducing the barriers to acquiring and using these systems. These requirements entail several challenges that relate mainly to cost and ease of use. Reducing cost of a traditional BCI system (usually at least 5000 USD) is mainly a technical problem that can be solved, but does require appropiate resources.

The purpose of this research was to elaborate a series of hardware and software tools for low cost, commercial BCI device operation enhancement and integration for an integrated hardware module implementation. The module must be capable of performing current machine learning algorithms on a fast speed processing basis with the use of useful classification and signal processing algorithms. Moreover, the module must be capable of implementing multiple and simultaneous user operation while displaying high resolution graphics and output operation for home automation or other real world applications.

One of the performed experiments utilized a commercial single electrode commercial BCI tool for obtaining a good recognition rate by means of different signal feature extraction methodologies as well as basic supervised and unsupervised learning approaches from the signals obtained by using a custom made virtual training environment with audio and video feedback. The data was composed of 30 trials consisting of attention and meditation signals obtained from guiding a virtual character through a specific path. Both methods used a 85-15 basis. The supervised method provided an 80% recognition rate while the unsupervised one provided a 60% recognition rate with an average processing time of around 132 seconds on a high-speed multicore processor laptop unit without parallelization.

The parallel batch-SOM/RBF-SVM method provided the best clustering/recognition performance, with a 70% recognition rate and around 7.73 seconds performance on embedded hardware with OpenMP implementation and 8.78 seconds with CUDA support. The following thesis presents the methodologies utilized for the user environment development, feature extraction methodology and implementation as well as parallel processing implementation approaches, results and discussion.

List of Publications

- C1 **Bruno Senzio-Savino** and Koji Yamada (2014) Test and development of a mind wave signal pattern password application. Proceedings of the 15th IEEE/SICE International Symposium on System Integration (SICE 2014), 1182-1184.
- C2 **Bruno Senzio-Savino**, M.R. Alsharif, C.E. Gutierrez and K. Yamashita (2015) Synchronous Emotion Pattern Recognition with a Virtual Training Environment. Proceedings of the International Conference on Artificial Intelligence (ICAI 2015), 650-654.
- C3 **Bruno Senzio-Savino**, Mohammad Reza Alsharif, Carlos E. Gutierrez, Katsumi Yamashita and Jason Noble (2015) Density Based Support Vector Machine Classification for a Synchronous EEG Path Tracing Virtual Environment. Proceedings of the 1st International Conference on Intelligent Informatics and BioMedical Sciences (ICIIBMS 2015), 223-226.
- C4 **Bruno Senzio-Savino**, Mohammad Reza Alsharif, Carlos E. Gutierrez, Christian Penaloza and Katsumi Yamashita (2016) Brain Wave Pattern Classification from Virtual Training Environment by Self-Organizing Maps. Proceedings of the 31st International Technical Conference on Circuits/Systems, Computers and Communications (ITC-CSCC 2016), 779-781.
- J1 **Bruno Senzio-Savino**, Mohammad Reza Alsharif, Carlos E. Gutierrez, Katsumi Yamashita and Jason Noble (2015) Path Detection in Virtual Environment for Synchronous EEG by Density Based Support Vector Machine. Journal of Information and Communications Engineering (JICE), 36-40.
- J2 **Bruno Senzio-Savino**, Mohammad Reza Alsharif, Carlos E. Gutierrez, Katsumi Yamashita, Kamal Setarehdan, Mahdi Khosravy and Faramarz Alsharif (2016) Brain Wave Pattern Classification: Towards the Design of an Effective Online Classifier. European Journal of Information Science and Technology (EJIST),18-29.
- J3 **Bruno Senzio-Savino**, Mohammad Reza Alsharif, Carlos E. Gutierrez and Kamal Setarehdan (2017) An Online Brain Wave Signal Pattern Classifier by Using Supervised and Unsupervised Learning with Parallel Processing Optimization. International Journal of Advanced Computer Science and Applications(IJACSA), 412-419.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Research background

In this chapter, a brief introduction of the fundamental principles of the main layout of this research is described by undergoing the main aspects of artistic brain computer interfaces and the use of brain wave biometrics as a way for authentication. Also, the research objectives of this work are presented and a brief summary of the whole content of this thesis is shown.

1.1.1 Artistic Brain-Computer Intefaces (BCI)

Artistic BCI is a form of affective BCI that focuses on the understanding of the user's affective state through artistic forms.

In 2013, Gurkok and Nijholt introduced and formalized the idea of artistic BCIs and its three categories[1]: audification/visualization, musification/animation, and instrument control. They provided examples of previous work based on Electroencefalogram (EEG) devices that can relate to artistic BCI and their components. It can be said that artistic BCI consists of three major components:

- a visual or audial representation of the user's affective and cognitive state;
- conversion of the brain signals to control commands for virtual and real artistic instruments
- the use of the brain rhythms to compose sound and draw visualizations; this does not serve as the representation, but as the control.

Artistic BCIs are also a way to create art for physical and virtual environments through cognitive control using a BCI. Artistic BCI may consist of four major fields:

- Art: Aesthetics, Illustration, Animation, Fine Arts, Design and Music
- HCI (Human-Computer Interface): User Experience, Multimedia Interaction, Multimodal Interaction, Sonification/Visualization, Wearable Computing and Audification
- Neurophysiology: Affective Brain-Computer Interface, Cognitive Psychology, Brain-Computer Music Interface, Physiological Computing, Symbiotic Interaction and Signal Processing

- Computing: Computer Graphics, Machine Learning, Mobile Computing, Web Technologies, Create/Compose and Spatial Audio and Control

These fields contain knowledge and tools, which can help the field of artistic BCI progress and become more established.

1.1.2 Brain Wave Authentication Background

With the embedding of EEG (electro-encephalography) sensors in wireless headsets and other consumer electronics, authenticating users based on their brainwave signals has become a realistic possibility.

Brain wave based authentication is another addition to the wide range of authentication systems, but with a brand new concept. The electrical activity in a human brain is used to confirm the identity. Instead of physically writing a password, one can think simply think about it. The password or "pass-thought" can be anything that a human mind may think about, like a color, a feeling, an image, text or something else.

The benefits over other systems are many. With a standard password someone can watch or "shoulder-surf" what others type, but no one can watch thoughts. Cards and keys can be lost, but the brain is always present. Handicaps can exclude people from systems like fingerprint- or retina scanners, but the brain still works[2]. There is a limited amount of related work in this area. However, there have been some attempt, for instance, a successful implementation over cell phones has been presented before (Figure 1.1)[3]), the term Brain-Mobile Phone Interface is also recent in this especial field of BCI[3].

Figure 1.1: The Neurophone: A Brain Wave Authentication Approach

As of the time of this graduation thesis being writen, both industry and academia are analyzing the great benefits and possibility of implementing this technology with comercial devices.

As an example, Japanese companies like Mistubishi Electric Research Laboratories[4] seem to take a trend similar as of this research, suggesting an interesting and intriguing path for this problem to be solved in further years. Terms like BrainID are also being formed and being investigated in different Japanese universities as well[5].

1.2 Objectives

The focus of this research is to obtain and establish the fundamentals towards the design and development of an embedded system module with high processing capabilities for high graphic resolution and signal classification with the use of low and high end commercial grade BCI devices. The use of trend or state of art tools is important as the use of latest technologies can help in obtaining better performance and a longer component lifetime in the commercial production line. In order to accomplish the main purpose, the following steps were set and accomplished:

- Understand the development of BCI technology before and now
- Understand the technology in terms of software and hardware
- Research about the uses an possibilities of using the technology in combination with other technological trends
- Research and understand the development of tools that can help utilize the technology in a simple but useful way
- Recreate a sample environment and sample processing for having a better understanding of the proper hardware and software tools to be used

Once that the objectives were set in a clear way, the following was performed throughout this research:

- Create a way to understand signal pattern generation with a simple low cost BCI tool
- Generate an environment that can allow the generation of the desired patterns
- Research and use some current classification and pattern recognition techniques as well as feature extraction methods
- Implement classification tasks with the aid of a high speed desktop or laptop computer
- Research and understand the use of parallel processing algorithms and techniques in the current state of arts

- Select a hardware that can provide good processing and graphic display capabilities in order to implement some of the learned techniques
- Process and reduce time of signal feature extraction and classification in order to generate an almost real time signal processing and classification tool

1.3 Research outiline

This thesis is divided into three main parts that will describe the theory and implementation of BCI with the use of low cost comercial grade tools and the use and application of different Machine Learning algorithms in order to analyze their capabilities and implementation on real high processing embedded hardware for this technology.

In Chapter 2, a brief history and background of Brain-Computer Intefaces is presented as well as its basic mathematical principles; also some of the future vision for this technology is shown.

For Chapter 3, a detailed description of the theory background of the implemented algorithms is described and also a clear description of the parallel programming approaches that were taken into account for the utilized tools in the hardware implementation stage.

Finally, in Chapter 4, the implementation of the overall research is demonstrated and an approach of a hardware approach is shown. The way that different tools were used is also described and examples of different graphic environments is presented as well.

Additionally, the results that were obtained are discussed and compared in order to present a resolution of this research and its hardware implementation. Also, the code implementation is presented on the Appendix section of this document.

2 Brain-Computer Interface Background

In the last fifteen years, a recognizable surge in the field of Brain Computer Interface (BCI) research and development has emerged. This emergence has sprung from a variety of factors. For one, inexpensive computer hardware and software is now available and can support the complex high-speed analyses of brain activity that is essential in BCI [6]. In this chapter a brief overview of the technology is given as well and an explanation of BCI current state of art in combination with other technologies is provided.

2.1 Overview of BCI

The Brain-Compute Interface (also known as Direct Neural Interface, DNI, or Brain-Machine Interface BMI) is a technology from over a century ago. This technology combines many different fields, including computer science, electrical engineering, neurosurgery (invasive BCI) and biomedical engineering. Its use has many applications from detection of health disorders such as cerebral palsy, strokes (epilepsy)to robot manipulation.

2.1.1 Brief History of EEG and BCI

In the 1970's, research on BCIs started at the University of California, which led to the emergence of the expression braincomputer interface. The focus of BCI research and development continues to be primarily on neuroprosthetics applications that can help restore damaged sight, hearing, and movement. The mid-1990's marked the appearance of the first neuroprosthetic devices for humans. BCI doesn't read the mind accurately, but detects the smallest of changes in the energy radiated by the brain when you think in a certain way. A BCI recognizes specific energy/ frequency patterns in the brain[7]. During the last fifteen years, the first BCI games that were demonstrated to the public (Braingate), and the use of voiceless phone calls has been demonstrated (Audeo) [9].

BCI is based on EEG, which is referred as the measurement of electrical activity in the living brain. The first electrical neural activities of the human brain were registered by Hans Berger (1924) using a simple galvanometer. On the human scalp was placed only one electrode, and one wave was identified. It was alpha wave (also called Berger's wave)[8]. Berger then measured

the brain waves of people with epilepsy, dementia and other disorders, as well as healthy subjects. Interest in EEGs flourished only after the noted British physiologist Lord Edgar Adrian replicated Berger ' s work in 1934. Nowadays in clinical use of EEG, 21 electrodes are used to identified 5 fundamental waves.

2.1.2 Types of BCI

There are three types of BCI approaches, which are listed below:

- Invasive BCI: Electrodes or measuring instruments are implanted directly into the grey matter of the brain during neurosurgery. Example of this technologies include Spikes (Intracortical neuron recording)
- Non Invasive BCI: This method do not involve neurosurgery. They are just like wearable virtual reality devices. Examples include EEG, functional Magnetic Resonance (fMRI) and functional Near Infrarred Spectroscopy (fNIRS).
- Partially Invasive BCI: Consist of devices that are implanted inside he skull but rest outside the brain rather than within the grey matter. Electrocorticograms (ECoGs) are usually presented as an example of this procedure.

The quality of the signal increases as the signal is detected from the source of origin. Despite of invasive BCI, non-invasive technologies involve a lot of noise not only from the local area but also movement and external factors produce a very noisy output, suggesting the use of signal processing techniques in order to retrieve an approximation of the desired output. Table 2.5 illustrates the characteristics of some of this brain wave acquisition methods [10]

Table 2.1: Spatiotemporal quality of common Brain Wave acquisition methodologies

Imaging method	Activity measured	Temporal resolution	Spatial resolution	Risk	Portability
EEG	Electrical	0.05s	~10 mm	Non-invasive	Portable
MEG	Magnetic	0.05s	~5 mm	Non-invasive	Non-ortable
ECoG	Electrical	0.003s	~1 mm	Partial	Portable
Intracortical neuron recordig	Electrical	0.003s	~0.5mm(LFP) ~1 mm(MUA) ~0.05mm(SUA)	Invasive	Portable
fMRI	Metabolic	1s	~1 mm	Non-invasive	Non-portable
fNIRS	Metabolic	1s	~5 mm	Non-invasive	Portable

2.2 The general BCI System Configuration

2.2.1 EEG Signal Types

As stated before, there are five major brain waves forms that can be distinguished by their frequency ranges. These frequency bands from low to high frequencies are called alpha, theta, beta, delta and gamma. Table 2.2 presents their typical voltage values respect to a common reference. There are also some other waves that are called disturbances or artifacts and are produced mainly due to ocular or external muscular activities of the user or electromagnetic interference (EMI).

Table 2.2: The summary of the basic human brain waves

Wave name	Typical Amplitude [μV]
Delta [δ]	100-200
Theta [θ]	Higher than 30
Alpha [α]	30-50 or higher
Beta [β]	2-20 or higher
Gamma [γ]	3-5 or higher
Mu/SMR Rythm/Artifact noise [μ]	Depends on noise

Our brain waves change according to what we are doing and feeling. When slower brain waves are dominant we can feel tired, slow, sluggish, or dreamy. The higher frequencies are dominant when we feel wired, or hyper-alert.

The descriptions that follow are only broadly descriptions (in practice things are far more complex, and brain waves reflect different aspects when they occur in different locations in the brain)[11]:

- Alpha Waves: These waves occur during sensorial rest (e.g. when eyes are closed), intellectual relaxation, deep relaxation, meditation or quieting of the mind. Alpha waves are the desired results of meditators. Alpha brain waves produce peaceful feelings. warm hands and feet, a sense of well being, improved sleep, improved academics, increased productivity, reduced anxiety and improved immune function.

- Beta Waves: These patterns are generated naturally when in an awake, alert state of consciousness. This type of pattern correlates with active problem solving and normal thinking activity. It requires more beta activity when you are learning a task than when it is mastered

- Delta Waves: This rhythm is observed in our sleep state. As we fall asleep, the dominant natural brain wave pattern presented is Delta.

- Theta Waves: This is commonly referred to as the dream or "twilight" state. Associated with hypnogogic states, REM sleep and dreaming. Memory development is enhanced in this state, particularly long-term memory, access to unconscious material, sudden insights and creative ideas.
- Gamma Waves: This are the highest frequency brain wave type. A variety of studies have associated gamma with the formation of ideas, linguistic processing and various types of learning.
- Mu Rhythms: Also called sensorimotor rhythms, these are synchronized patterns of electrical activity involving large numbers of neurons, probably of the pyramidal type, in the part of the brain that controls voluntary movement.
- SMR Rhythms: It is a brain wave rhythm for the sensorimotor cortex.
- Artifact noise: This is low frequency noise generated by electronic devices that emit electric noise. Most of the devices work at 50-60Hz from the supply line.

2.2.2 EEG Signal Acquisition

The principle of EEG measurement is to calculate the potential difference of two electrodes. The reason for this potential difference is due to interchange of ions across the cell membrane [12]. Equation 2.2 and Figure 2.1[12] refer about the way the potential is acquired and signal generation.

$$E = \frac{RT}{F} ln\left(\frac{P_k[K]_o + P_Na[Na]_o + P_Cl[Cl]_i}{P_k[K]_i + P_Na[Na]_i + P_Cl[Cl]_o}\right) \tag{2.1}$$

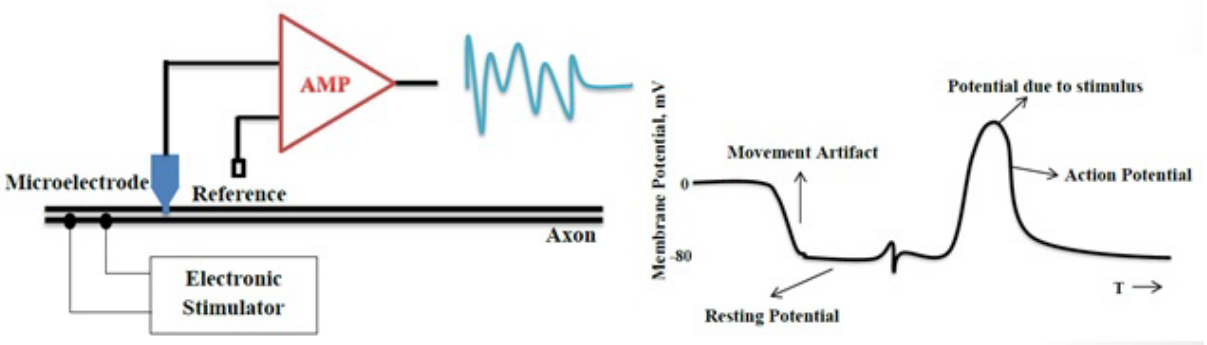

Figure 2.1: EEG Signal Generation and Characteristics

Due to ionic transfers during the neuronal activity in brain cells, an electric potential is developed in the neurons. This combined voltage potential from multiple neurons can be measured from different locations of the scalp.

2.2.3 EEG Electrode Positioning

The placement of the reference electrodes is important because if they are too close to the brain they are affected from the brain activity and also if it is on another part of the body, it is likely to be affected from muscle's electrical activity, especially from the heart. The 10/20 system or International 10/20 system is an internationally recognized method to describe the location of scalp electrodes [13].

The system is based on the relationship between the location of an electrode and the underlying area of cerebral cortex. The numbers '10' and '20' refer to the fact that the distances between adjacent electrodes are either 10% or 20% of the total front-back or right-left distance of the skull. Each site has a letter to identify the lobe and a number to identify the hemisphere location. Table 2.3[13] and Figure 2.2[14] illustrate different positioning and nomenclature for this system:

Table 2.3: 10/20 system nomenclature

Wave name	Lobe
F	Frontal
T	Temporal
C	Central
P	Parietal
O	Occipital

No central lobe exists, the 'C' letter is used for identification purposes only. The 'z' (zero) refers to an electrode placed on the mid line. Even numbers (2,4,6,8) refer to electrode positions on the right hemisphere. Odd numbers (1,3,5,7) refer to electrode positions on the left hemisphere.

Four anatomical landmarks are used for the essential positioning of the electrodes: first, the nasion which is the point between the forehead and the nose; second, the inion which is the lowest point of the skull from the back of the head and is normally indicated by a prominent bump; the pre auricular points anterior to the ear[13].

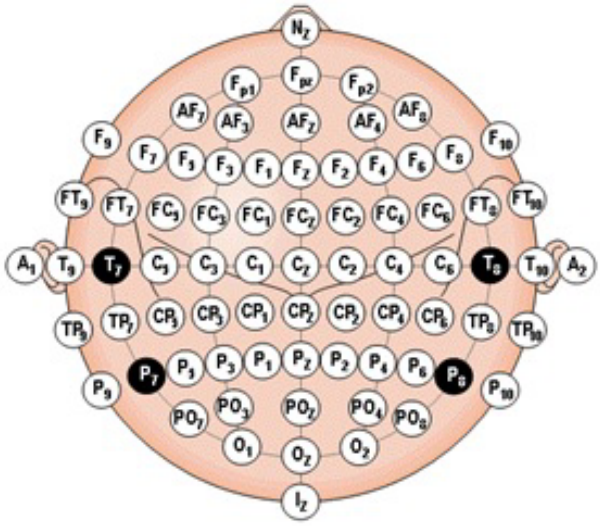

Figure 2.2: The 10/20 Positioning System

2.2.4 EEG Signal Analysis and the Fast Fourier Transform (FFT)

The raw EEG signal from an electrode can be represented as a periodic signal *x(t)* in terms of an infinite sum of sines and cosines. Mathematical formula of a Fourier series is presented in Equation 2.2

$$x(t) = \frac{a_0}{2} + \sum_{k=1}^{\inf}(a_n cos(\omega kt) + b_n sin(\omega kt)) \tag{2.2}$$

A generalization of Fourier series, for infinite domains, is Continuous Fourier Transform (CFT). If the signal is periodic, band-limited and sampled at Nyquist frequency of higher, the CFT is represented exactly by Discrete Fourier Transform (DFT) (Equation 2.3).

$$X_k = \sum_{n=0}^{N-1} x_n e^{\frac{-j2\pi kn}{N}} \tag{2.3}$$

Inverse conversion to the DFT is the Inverse DFT (IDFT) (Equation 2.4. It transforms back the data from the frequency domain to the time domain; by this method it is possible to decompose the Raw EEG signal into the different brain waves (Figure 2.3).

$$x_n = \frac{1}{N} \sum_{n=0}^{N-1} X_k e^{\frac{j2\pi kn}{N}} \tag{2.4}$$

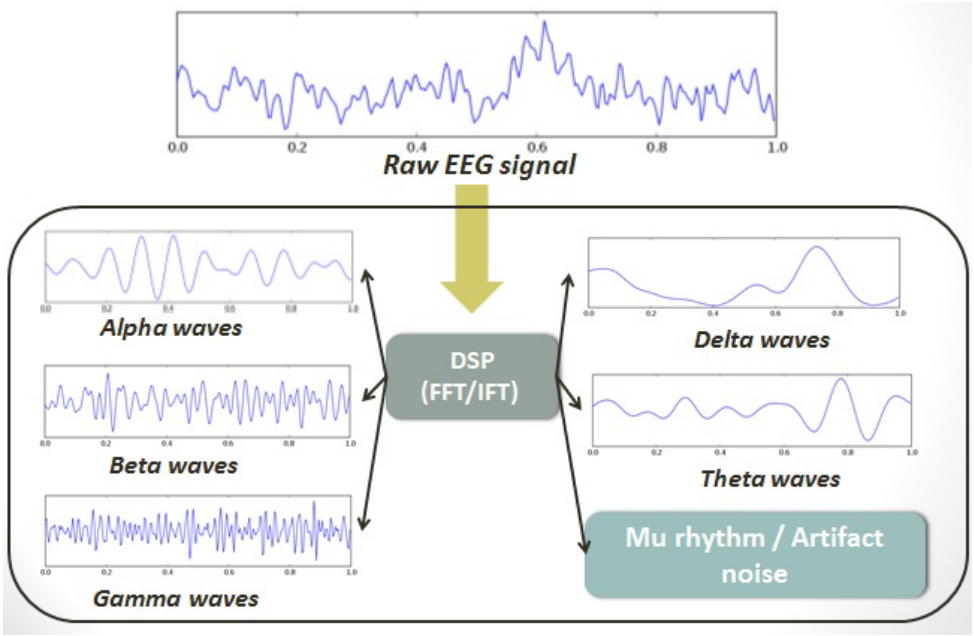

Figure 2.3: EEG Signal Analysis by FFT/IFFT

The Nyquist sampling theorem[15] states that *The sampling frequency should be at least twice the highest frequency contained in the signal*, in mathematical terms (Equation 2.5):

$$f_s \geq 2f_c \tag{2.5}$$

The Fast Fourier Transform (FFT) is a collection of algorithms for fast computation of the DFT. Since its introduction by Cooley-Tukey almost a half century ago has been playing historically sustained significant role in the development of Digital Signal Processing (DSP) since it is the most widely used "fast algorithm" in solving many engineering challenges, designing filters, performing spectral analysis, estimation, noise cancellation and benchmark testing devices and systems, etc. If we look at Equation 2.3, we can note the following in terms of computational complexity (1 operation = 1 *OP*):

1 *OP* = 1 *Complex Multiply* + 1 *Complex Add*
1 *OP* = 1 *Cos Multiply* + 1 *Sin Multiply* + 1 *Complex Add* + 1 *Complex Accumulate*

- To compute N-Point DFT we need N^2 *OPS*
- Cooley-Tukey basic FFT algorithm requires $Nlog_2N$ (Table 2.4)

Table 2.4: Cooley-Tukey (C-T) FFT OP Saving Comparison Chart

m	$N = 2^m$	DFT OP	C-T FFT OPs	FFT Savings
1	2	4	2	50.0%
2	4	16	8	50.0%
3	8	64	24	62.5%
4	16	256	64	75.0%
5	32	1024	160	84.4%
6	64	4096	384	90.6%
7	128	16384	896	94.5%
8	256	65536	2048	96.9%
9	512	262144	4608	98.2%
10	1024	1048576	10240	99.0%

- 99% savings in computational complexity is unheard of in any other fast algorithm in science. Even for reasonable and frequently observed FTT sizes of 128 or 256-points FFT we are in the 90% savings range[16].

2.2.5 BCI Signal Processing Configuration Diagram

Multi-stage procedure for on-line BCI: Preprocessing, feature extractions and intelligente classification play a key role in real-time high performance BCI systems (Figure 2.4[17]) .

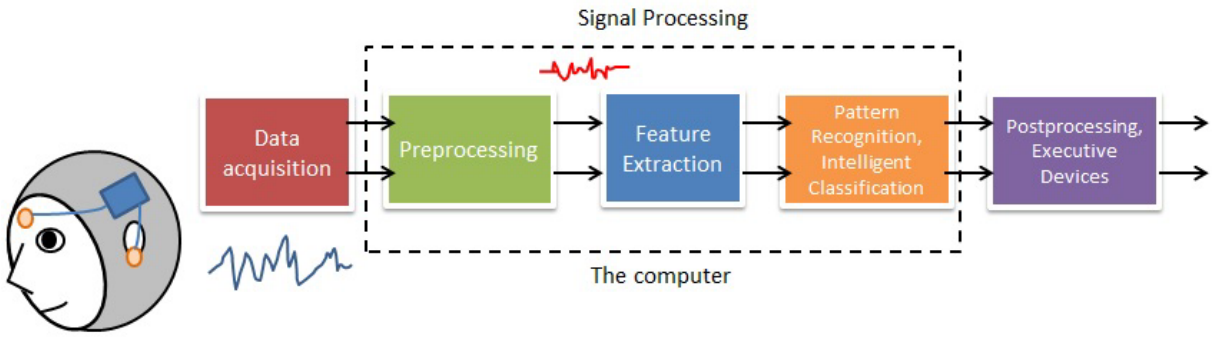

Figure 2.4: Common BCI Configuration Diagram

Since the EEG signals are rather weak and are substantially disturbed by on-going activity, artifacts and noise, need to be extracted and discriminated for useful brain patterns; powerful

signal processing techniques and machine learning algorithms are required in order to provide an interpretation of the acquired information.

Another promising extension of BCI is to incorporate various neurofeedbacks to train subjects to modulate EEG brain patterns and parameters. to meet a specific criterion or to learn self-regulations skills. The subject then changes their EEG patterns in response to some feedback. Such integration of neurofeedback in BCI (Figure 2.5) is an emerging technology for rehabilitation, and may also be a new paradigm in neuroscience that might reveal previously unknown brain activities associated with behavior or self-regulated mental states[17].

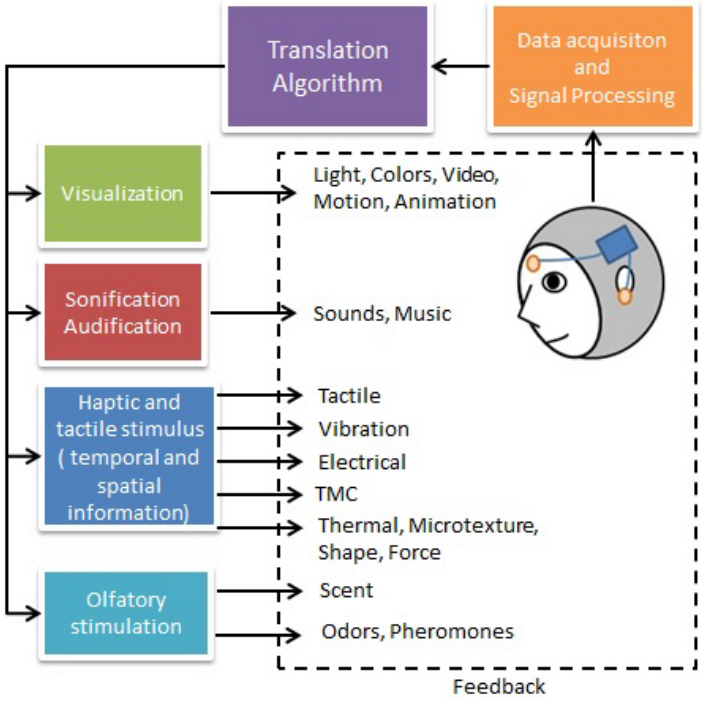

Figure 2.5: BCI with Neurofeedback

2.2.6 Power Spectral Density of EEG Signals

The Power spectral density function (PSD) shows the strength of the variations(energy) as a function of frequency. In other words, it shows at which frequencies variations are strong and at which frequencies variations are weak. It represents the power distribution of EEG series in the frequency domain is used to evaluate the predominance of a specific brain wave signal during a certain ammount of time. It is possible to visualize this information on a simple plot (e.g. Figure 2.6[18]). By determining the power value for a specific band it is possible to differentiate particular signal characteristics at a particular instant.

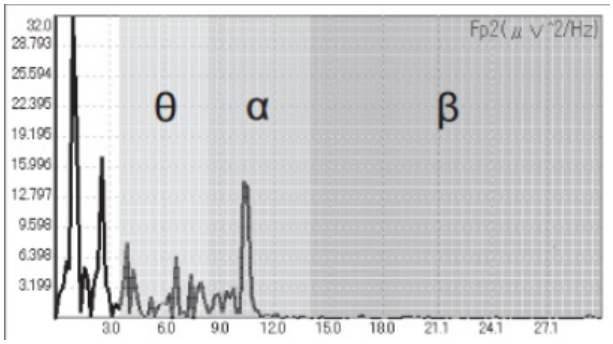

Figure 2.6: Spectrum analysis of EEG source

2.3 Commercial BCI: A Current Technology Trend

The use of Brain Computer Interfaces for commercial and research purposes is gaining hype and impulse due to its competitive advantage and suitability for the combination of innovative technologies such as, Human Augmentation, IoT Platforms or Affective Computing [19].

From Figure 2.7[19] it is inferred that BCI among other technologies are still on the verge of innovation with at least a decade to become part of the mainstream. Commercial BCI devices measure signals from the brain and turn them into outputs that provide value to the customer.

Figure 2.7: 2016's Hype Cycle for Emerging Technologies

2.4 Commercial FFT Based BCIs

Basic BCI research and development is based predominantly on recording and analysis of electrophysiological brain signals. The number of channels that are recorded usually varies from 8-64 for EEG. The brain signals recorded from these modalities vary substantially in their amplitudes and frequencies; these waves, preferred range of processing are specified in Table 2.1 [20]

Table 2.5: Basic human brain waves and ranges for FFT based devices

Name	Frequency range [Hz]	Preferred range [Hz]
Delta [δ]	0.5-2.75	0.5-3.0
Theta [θ]	3.5-6.75	2.0-7.0 or 4.0-8.0
Low Alpha [l-α]	7.5-9.25	8.0-10.0
High Alpha [h-α]	10.0-11.75	10.0-12.0
Low Beta [l-β]	13-16.75	SMR 12.0-15.0 Low Beta 15.0-18.0
Low Gamma [l-γ]	31-39.75	Mid Gamma
Mid Gamma [m-γ]	41-49.75	High Gamma

The use of Fast Fourier Transform BCI (FFT-BCI) based 3D virtual environments has been implemented effectively in different application fields; it is rather a new technological tool, which may be exploited to enhance motor retraining [21]. A simple development with software like Unity3D (https://unity3d.com) can be combined with other assistance technologies [22] or interfaces in order to improve a specific system's performance with an end user's feedback.

Recently, the use of this combination is representing an innovative treatment for conditions like ADHD [24]. Usually, FFT-BCI based devices obtain the power specific frequency bands that come from the raw EEG signal sample over a sampling period of time from the active electrodes and perfom a ratio calculation that then becomes a user output.

For this research purpose, a TGAM1 ASIC device, like the Neurosky Mindwave [25] with a single electrode tool was used for data collection. The output signals from this device include attention and meditation levels (from 0-100%) based on a power band calculation algorithm.

There are algorithms that focus on detecting a specific emotion from the end user according to their mind-wave patterns. Some of them can map and help to differentiate emotions like attention/inattention and meditation/uneasiness based on a simple beta/theta wave ratio as considered by Equation 2.6 [26]

$$Emotion_{ratio} = \frac{SMR + mBeta}{Theta} \tag{2.6}$$

2.4.1 BCI Commercial Devices

As with many other novel technologies, it is currently unclear in what situations BCI devices can provide maximum value for the largest number of users. Several manufacturers are currently exploring these questions by offering commercial BCI-like devices. These companies include Open BCI[27], Emotiv [28] and Neurosky[29] (Figure 2.8[28][30][31]) among others [32].

Figure 2.8: Some current comercial BCI devices (Left: OpenBCI, Center: Emotiv Epoc, Right: Neurosky Mindwave)

The success of widespread dissemination of commercial BCI devices depends on reducing the barriers to acquiring and using these systems. This requirements entails several challenges that relate mainly to cost and ease of use. Reducing cost is the of a traditional BCI system (usually at least 5000 USD) is mainly a technical problem that can be solved, but does require appropiate resources. One of the approaches included in this research is by means of recognition algorithms of output signals obtained by one of these devices.

2.4.2 The OpenBCI device

OpenBCI is an open source brain-computer interface platform. OpenBCI was created by Joel Murphy and Conor Russomanno, after a successful Kickstarter campaign in late 2013.

OpenBCI boards can be used to measure and record electrical activity produced by the brain (EEG), muscles (EMG), and heart (EKG), and is compatible with standard EEG electrodes. The OpenBCI boards can be used with the open source OpenBCI GUI, or they can be integrated with other open-source EEG signal processing tools. For our research, we used the tool for a preliminary tool testing and signal extraction[27] (Figure 2.9)

Figure 2.9: OpenBCI Tool Calibration and Training session

2.4.3 The Emotiv-Epoc

Emotiv Systems is an Australian electronics innovation company developing technologies to evolve human computer interaction incorporating non-conscious cues into the human-computer dialog to emulate human to human interaction. Developing braincomputer interfaces based on electroencephalography (EEG) technology, Emotiv Systems produced the EPOC near headset, a peripheral targeting the gaming market for Windows, OS X and Linux platforms. The EPOC has 16 electrodes and was originally designed to work as a brain-computer interface (BCI) input device[28]. In this reasearch, motor training test environment was tested with the use of an Emotiv headset (Figure 2.10).

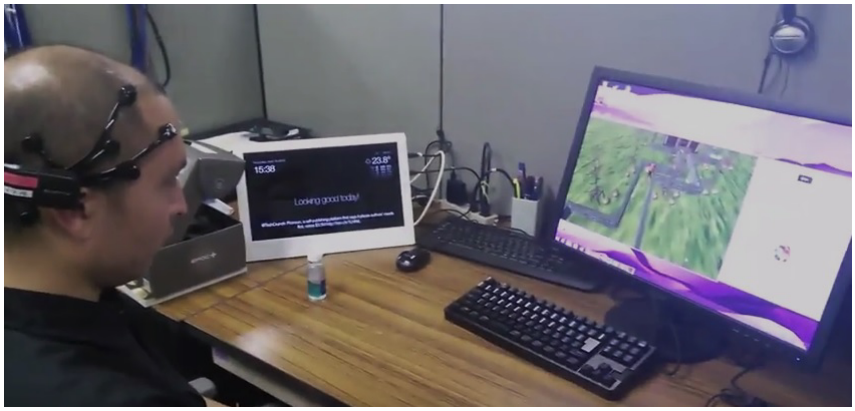

Figure 2.10: Training session with Emotiv Epoc headset

2.4.4 The TGAM-1 and the Neurosky Thinkgear

NeuroSky's ThinkGear TGAM-1 ASIC module is a chip that can integrate into any BCI form factor. The TGAM-1, which includes the TGAT ASIC, is 2.79cm x 1.52cm x 0.25cm, weighs 130mg and communicates its output through a UART interface at configurable baudrates of 1200, 9600 or 57600. Products such as Mattel's Mindflex and Star Wars Force Trainer use the TGAM-1[33]. The chip, as used from a Mindflex headset for this research data collection purposes (Figure 2.11) utilizes a Mindflex headset (compatible with the Neurosky ThinkGear functions).

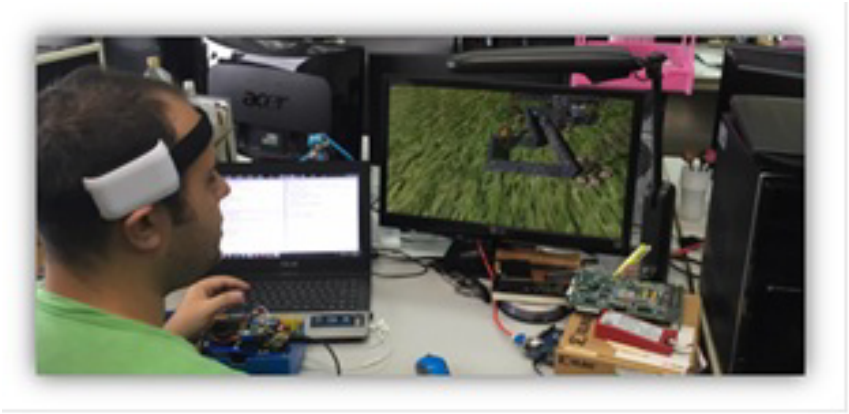

Figure 2.11: A TGAM-1 based headset used for data collection

ThinkGear[34] is the technology inside every NeuroSky product or partner product that enables the device to interface with the wearers ' brain aves. It includes:

- the sensor that touches the forehead,
- the contact and reference points located on the ear pad, and
- the onboard chip that processes all of the data.

Both the raw brain waves and the eSense Meters (Attention and Meditation) are calculated on the ThinkGear chip. The calculated values are output by the ThinkGear chip, through the headset, to a computer.

Types of data output from ThinkGear chips:

- Raw sampled wave values (128Hz or 512Hz, depending on hardware)
- Signal poor quality metrics
- eSense Attention and Meditation meter values

2.5 BCI: From the State of Art to Vision

By 2025, a wide array of applications will use brain signals as an important source of information. People will be supported in choosing the best time for making difficult and important decisions. People working in safety-relevant fields will be capable of anticipating fatigue, and authorities may find good (evidence-based) reasons to incorporate such applications in regulations.

Game, health, education, and lifestyle companies will link brain and other biosignals with useful applications for a broad community[35] . In Figure 2.12[35] it is possible to get an image of the future of this technology.

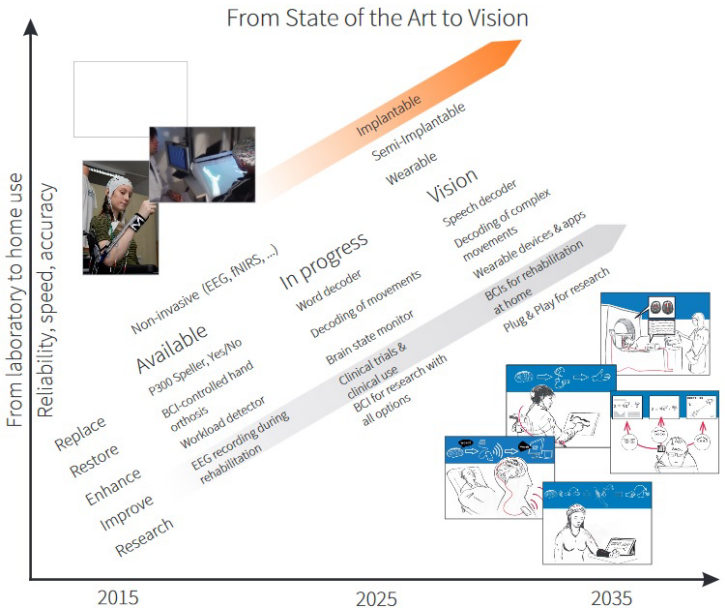

Figure 2.12: BCI: From State of Art to Vision

2.6 Common EEG and BCI Signal Processing Software Tools

The use of common software tools enhances and accelerates the signal processing tasks by using preprogrammed functions targeted for BCI research. Below there are some tools that become a current trend due to its ease of integration with commercial devices and system compatibility. However, most of them currently lack ARM architecture integration for small embedded device usage. Therefore, their use is currently limited to Intel or AMD (x86 or x64) architecture devices.

2.6.1 EEGLAB

EEGLAB is an interactive MATLAB toolbox for processing continuous and event-related EEG, MEG and other electrophysiological data incorporating independent component analysis (ICA), time/frequency analysis, artifact rejection, event-related statistics, and several useful modes of visualization of the averaged and single-trial data. EEGLAB runs under Linux, Unix, Windows, and Mac OS X[36].

2.6.2 OpenViBE

Intially, OpenViBE stood for "Open Virtual Brain Environment", as it was intended to provide BCI-based control of Virtual Reality (VR) platforms. OpenViBE is a software platform dedicated to designing, testing and using brain-computer interfaces. OpenViBE is a software for real-time neurosciences (that is, for real-time processing of brain signals). It can be used to acquire, filter, process, classify and visualize brain signals in real time. The software is free and open source. It works on Windows and Linux operating systems

The main OpenViBE application fields are medical (assistance to disabled people, real-time biofeedback, neurofeedback, real-time diagnosis), multimedia (virtual reality, video games), robotics and all other fields related to brain computer interfaces and real-time neurosciences[37].

2.6.3 neuromore Studio

neuromore Studio is a tool that allows different current comercial BCI/biosensing tool integration and apply a set of DSP features and Virtual Reality environments to provide online feedback and portability to different hardware platforms[38]. Despite the previous platforms described, this one is, to our knowledge, the nearest to BCI software development and comercialization. Figure 2.13 provides a glance at the Graphical User Interface (GUI) interface.

Figure 2.13: neuromore Studio GUI

3 Signal Feature Extraction and Classification Overview

In this chapter, a description of all the methodologies, hardware and software tools involved with the development of this research are presented.

First, feature extraction techniques and dimension reduction methods by digital signal processing are introduced. Then, Machine Learning methods used for the system implementation are defined. Also, a brief introduction on Parallel System programming is provided in order to understand the basic process utilized in the next chapter for online classification implementation. Finally. a detailed overview of the hardware tools and software suited for developing the neurofeedback techniques are presented.

3.1 Feature Extraction Methodologies

3.1.1 Values Above Zero (VAZ)

The number of values above zero[2] (Equation 3.1):

$$VAZ = \sum_{n=1}^{N-1} 1, if(x_n > 0), else0 \tag{3.1}$$

It becomes useful as a formal definition when setting up a value threshold for a specific signal (which can be different from zero.

3.1.2 Zero Crossing Rate (ZCR)

The rate at which values cross zero. If the product of two adjacent values is negative, they have opposite signs and a zero crossing has occurred. The rate is found by dividing the number of zero crossings by the number of samples (Equation 3.2[39]. It becomes useful when it is required to understand how many times a signal has been over a specific threshold.

$$ZCR = \frac{1}{N} \sum_{n=1}^{N-1} 1, if(x_n * x_{n+1} < 0), else0 \tag{3.2}$$

3.1.3 The Simple Moving Average Model (SMA)

By defining a window size *n*, an unweighted mean is obtained for a vector *x(n)* containing a series of data(Equation 3.3). the mean is normally taken from an equal number of data on either side of a central value [40].

$$SMA = \frac{1}{n}\sum_{i=0}^{n-1} x_{n-i}, \tag{3.3}$$

SMA has important applications in Economy. However, in this research, the smoothing of the collected sample series allows alternative options that were studied with Supervised Learning approaches.

3.1.4 Linear Prediction and Error Minimization

Given a set of original discrete values $(y_n)_{n\in[\![0,M]\!]}$ which is extended to $(y_n)_{n\in Z}$ with an infinite number of zeroes, it is required to find the best k coefficients $(a_n)_{n\in[\![1,k]\!]}$ that will approximate y_n by $-\sum_{i=1}^{k} a_i y_{n-i}$. A common way to define best is to use the least-square sense, which means finding $(a_n)_{n\in[\![1,k]\!]}$ so that to minimize the sum of the squares of the error between the original and approximated values[41].

$E = \sum_{n=-\infty}^{\infty}(y_n - (-\sum_{i=1}^{k} a_i y_{n-i}))^2 = \sum_{n=-\infty}^{\infty}(y_n + \sum_{i=1}^{k} a_i y_{n-i})^2$

Defining $a_0 = 1$ gives the simpler $\mathrm{E} = \sum_{\mathrm{n}=-\infty}^{\infty}(\sum_{\mathrm{i}=0}^{\mathrm{k}} \mathrm{a_i y_{n-i}})^2$ which is the value to be minimized. At E's minimum for $j \in [\![1,k]\!]$, $\frac{\partial E}{\partial a_j} = 0$. Calculating the partial derivatives of E gives $\sum_{n=\infty}^{\infty} 2y_{n-j}(\sum_{i=0}^{k} a_i y_{n-i}) = 0$. Although the sum is written as infinite, it is finite since all terms vanish to zero at some point, therefore the two sum signs can be swapped and get $2\sum_{i=0}^{k} a_i \sum_{n=\infty}^{\infty} y_{n-i}y_{n-i}) = 0$, which can be rewritten $\sum_{i=0}^{k} a_i \sum_{n=-\infty}^{\infty} y_n y_{n+j-i} = 0$.

Defining

$$R_l = \sum_{n=-\infty}^{\infty} y_n y_{n+l} \tag{3.4}$$

It takes the final following form $\forall j \in [\![1,k]\!], \sum_{i=0}^{k} a_i R_{|j-i|} = 0$. This can be represented in matrix form $MA_k = 0$ as:

$$M = \begin{bmatrix} R_1 & R_0 & R_1 & \cdots & R_{k-1} \\ R_2 & R_1 & R_0 & \cdots & R_{k-2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ R_{k-1} & R_{k-2} & \cdots & R_2 & R_1 \\ R_k & R_{k-1} & \cdots & R_1 & R_0 \end{bmatrix} and\, A_k = \begin{bmatrix} 1 \\ a_1 \\ a_2 \\ \vdots \\ a_k \end{bmatrix}$$

The matrix M has k-1 columns and k rows. The system is not under determined, however, it is more convenient to make the system under a square matrix form. $MA_k = 0$ could be put as a square system as below, but there is an easier but less direct way to solve this system[41].

$$\begin{bmatrix} R_0 & R_1 & \cdots & R_{k-1} \\ R_1 & R_0 & \cdots & R_{k-2} \\ \vdots & \vdots & \ddots & \vdots \\ R_{k-1} & R_{k-2} & \cdots & R_0 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_k \end{bmatrix} = - \begin{bmatrix} R_0 \\ R_1 \\ \vdots \\ R_{k-1} \end{bmatrix}$$

M looks similar to a symmetric Matrix, with the top row missing. If the top row is expanded, a square matrix and system can be composed.

$$N_k A_k \begin{bmatrix} E_k \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \; with\, N_k = \begin{bmatrix} R_0 & R_1 & \cdots & R_k \\ R_1 & R_0 & \cdots & R_{k-1} \\ \vdots & \vdots & \ddots & \vdots \\ R_k & R_{k-1} & \cdots & R_0 \end{bmatrix} \; and\, A_k = \begin{bmatrix} 1 \\ a_1 \\ a_2 \\ \vdots \\ a_k \end{bmatrix}$$

E_k is unknown at that point since it is a functon A_k and the coefficients $(R_j)_{j\in[\![0;k]\!]}$. The system can actually be solved by a very simple and recursive method called the Levinson-Durbin recursion[41].

3.1.5 The Levinson-Durbin Recursion Algorithm

One application of the Levinson-Durbin formulation is in the Yule-Walker Autoregresive (AR) problem, which concerns modeling an unknown system as an autorregressive process[42].

The basic simple ideas behind the recursion are first that it is easy to solve the system for k =1, and second that it is also very simple to solve for a k +1 coefficients sized problem when we have solved a for a k coefficients sized problem.

In general, none of the coefficients of the different sized problem match, so it is not a way to calculate a_{k+1} but a way to calculate the whole vector A_{k+1} as a function of N_{k+1},E_k and A_k[41]. A solution that satisfies E_1.

Size one problem

$$A_1 = \begin{bmatrix} 1 \\ a_1 \end{bmatrix}, N_1 A_1 = \begin{bmatrix} E_1 \\ 0 \end{bmatrix}, \; and\, N_1 = \begin{bmatrix} R_0 & R_1 \\ R_1 & R_0 \end{bmatrix}$$

is not necessary at this step. The dot product of the second line of N_1 and A_1 gives $R_1 + R_0 a_1 = 0$, with $R_0 = \sum_{-\infty}^{\infty} y_n^2 > 0$[41]. Therefore

$$a_1 = -\frac{R_1}{R_0} \tag{3.5}$$

$$A_1 = \begin{bmatrix} 1 \\ a_1 \end{bmatrix}$$

and also

$$E_1 = R_0 + R_1 a_1 \tag{3.6}$$

Size k+1 problem

Supposing that the one size problem is solved, it is assumed that

$$\begin{bmatrix} R_0 & R_1 & \cdots & R_k \\ R_1 & R_0 & \cdots & R_{k-1} \\ \vdots & \vdots & \ddots & \vdots \\ R_k & R_{k-1} & \cdots & R_0 \end{bmatrix} \begin{bmatrix} 1 \\ a_1 \\ a_2 \\ \vdots \\ a_k \end{bmatrix} = \begin{bmatrix} E_k \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

N_{k+1} has one more row and column than N_k so we cannot apply it directly to A_k, however if A_k is expanded with a zero and the vector is called U_{k+1}, N_{k+1} can be applied to obtain

$$\begin{bmatrix} R_0 & R_1 & \cdots & R_{k+1} \\ R_1 & R_0 & \cdots & R_k \\ \vdots & \vdots & \ddots & \vdots \\ R_{k+1} & R_k & \cdots & R_0 \end{bmatrix} \begin{bmatrix} 1 \\ a_1 \\ a_2 \\ \vdots \\ a_k \\ 0 \end{bmatrix} = \begin{bmatrix} E_k \\ 0 \\ 0 \\ \vdots \\ 0 \\ \sum_{j=0}^{k} a_j R_{k+1-j} \end{bmatrix}$$

Since the matrix is symmetric, by reversing U_{k+1} (V_{k+1})

$$\begin{bmatrix} R_0 & R_1 & \cdots & R_{k+1} \\ R_1 & R_0 & \cdots & R_k \\ \vdots & \vdots & \ddots & \vdots \\ R_{k+1} & R_k & \cdots & R_0 \end{bmatrix} \begin{bmatrix} 0 \\ a_k \\ \vdots \\ a_2 \\ a_1 \\ 1 \end{bmatrix} = \begin{bmatrix} \sum_{j=0}^{k} a_j R_{k+1-j} \\ 0 \\ 0 \\ \vdots \\ 0 \\ E_k \end{bmatrix}$$

A linear combination $U_{k+1} + \lambda V_{k+1}$ is for the form wanted for A_{k+1}. If there was a value of λ for which $N_{k+1}(U_{k+1} + \lambda)$ would look like $[E_{k+1}\, 0\, 0\, \cdots\, 0]$', E_{k+1} not being known at this stage, it means that A_{k+1} has been found[41].

Calculating $N_{k+1}(U_{k+1} + \lambda)$ gives

$$\begin{bmatrix} R_0 & R_1 & \cdots & R_{k+1} \\ R_1 & R_0 & \cdots & R_k \\ \vdots & \vdots & \ddots & \vdots \\ R_{k+1} & R_k & \cdots & R_0 \end{bmatrix} \begin{bmatrix} 1 \\ a_1 + \lambda a_k \\ a_2 + \lambda a_{k-1} \\ \vdots \\ a_k + \lambda a_1 \\ \lambda \end{bmatrix} = \begin{bmatrix} E_k + \lambda \sum_{j=0}^{k} a_j R_{k+1-j} \\ 0 \\ 0 \\ \vdots \\ 0 \\ \sum_{j=0}^{k} a_j R_{k+1-j} + \lambda E_k \end{bmatrix}$$

The value λ that satisfies $\sum_{j=0}^{k} a_j R_{k+1-j} + \lambda E_k = 0$ can be found easily. Therefore:

$$\lambda = \frac{-\sum_{j=0}^{k} a_j R_{k+1-j}}{E_k} \tag{3.7}$$

$$A_{k+1} = U_{k+1} + \lambda V_{k+1} \tag{3.8}$$

$$E_{k+1} = E_k + \lambda \sum_{j=0}^{k} a_j R_{k+1-j} = (1 - \lambda^2) E_k \tag{3.9}$$

3.1.6 Autoregresive Modeling (AR) by Yule-Walker method

Although the Yule-Walker (YW) method is not very suggested for error prediction by some sources, in this research the intention is to obtain a set of coefficients that serve the purpose of feature extraction for classification[43]. Consider a linear process of successive samples $y_i$that depends on their predecessors:

$$y_t + a_1 y_{t-1} + a_2 y_{t-2} + \cdots + a_k y_{t-k} = \eta_t \tag{3.10}$$

in which a_i are the autoregressive parameters and the innovations η_t are a stationary purely random process with zero mean. It can be shown that the autocovariance function R_r for delays 0 to k is related to the autoregressive parameters a_i, through te Yule-Walker equation for the autoregressive process[44]

$$\begin{bmatrix} R_0 & R_1 & \cdots & R_{k-1} \\ R_1 & R_0 & \cdots & R_{k-2} \\ \vdots & \vdots & \ddots & \vdots \\ R_{k-1} & R_{k-2} & \cdots & R_0 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_k \end{bmatrix} = - \begin{bmatrix} R_1 \\ R_2 \\ \vdots \\ R_k \end{bmatrix}$$

An estimation of the model can be rewritten as:

$$y_t + \hat{a}_1 y_{t-1} + \hat{a}_2 y_{t-2} + \cdots + \hat{a}_k y_{t-k} = \hat{\eta}_t \tag{3.11}$$

in which $\hat{a}_i$ are the autoregressive-parameter estimates and $\hat{\eta}_\mathrm{t}$ are the estimated innovations. This model is different from the real estimation, so a residue must be taken into account[43]. For this research purposes the estimated parameters are enough for a simple classification task. Each data sample can be predicted from its predecessors:

$$\hat{y_t} = -\sum_{i=1}^{k} \hat{a}_t y_{t-i} \tag{3.12}$$

Depending in the autoregressive order model the number of computations can be increased. For this cae, the model order k is known (in practice maximum likelihood function and the Akaike's criterion can be applied to set an order). There are various methods for AR estimation, however, in this research the Yule-Walker approach is utilized.

For Yule-Walker, from the first to last k data points in Equation3.1.6 we have

$$\begin{bmatrix} \hat{R_0} & \hat{R_1} & \cdots & \hat{R_{k-1}} \\ \hat{R_1} & \hat{R_0} & \cdots & \hat{R_{k-2}} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{R_{k-1}} & \hat{R_{k-2}} & \cdots & \hat{R_0} \end{bmatrix} \begin{bmatrix} \hat{a_1} \\ \hat{a_2} \\ \vdots \\ \hat{a_k} \end{bmatrix} = - \begin{bmatrix} \hat{R_1} \\ \hat{R_2} \\ \vdots \\ \hat{R_k} \end{bmatrix}$$

in which the matrix elements $\hat{R_\tau}$ contribute a biased estimate of the autocovariance function (Equation 3.13).

$$\hat{R_\tau} = \frac{1}{N} \sum_{t=\tau+1}^{N} y_t y_{t-\tau} \tag{3.13}$$

The Levinson-Durbin algorithm provides a fast solution of Toeplitz-style model as the previous one.

3.2 Signal Analysis and Common Processing Techniques

3.2.1 Singular Value Decomposition (SVD)

SVD is based on a theorem from linear algebra which says that a rectangular matrix A can be broken down into the product of three matrices - an orthogonal matrix U diagonal matrix S, and the transpose of an orthogonal matrix$V$. The theorem is usually presented as[45]:

$$A_{mm} = U_{mm} S_{mn} V_{nn}^{T} \tag{3.14}$$

where $U^T U = I, V^T V = I$; the columns of U are orthonormal eigenvectors of AA_T, the columns of V are orthonormal eigenvectors of $A^T A$ and S is a diagonal matrix containing the square roots of eigenvalues from U or V in descending order.

A full SVD over a non rectangular matrix can provide a good approximation its eigenvalues, especially if they are used for a calibration model.

3.2.2 Scree Plot for a Calibration Model

A scree plot displays the eigenvalues associated with a component or factor in descending order versus the number of the component or factor. The scree plots can be used in principal components analysis and factor analysis to visually assess which components or factors explain most of the variability in the data [46].

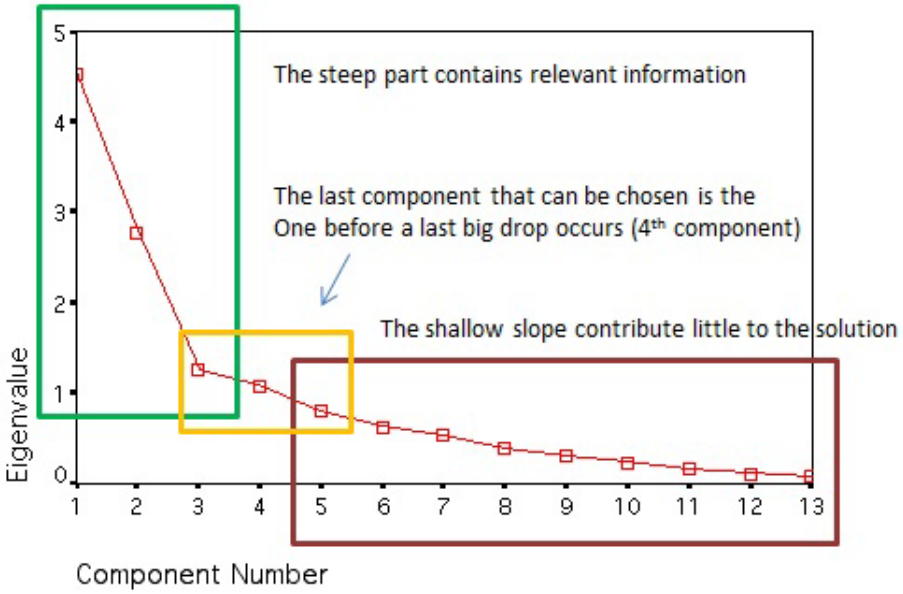

Figure 3.1: Scree plot and factor analysis

The ideal pattern in a scree plot (Figure 3.1[47]) is a steep curve, followed by a bend and then a flat or horizontal line. In practice, retaining those components or factors in the steep curve before the first point that starts the flat line trend will provide the usual number of dimensions to retain for the following step.

A problem of the scree test relates to finding a sharp break.It suffers from subjectivity and ambiguity, especially where there are either no clear breaks or two or more apparent breaks. Definite breaks are less likely with smaller sample sizes and when the ratio of variables to factors is low[48]. However, for this research the Scree plot provides a good method for finding an optimal number of components for dimension reduction purposes.

3.2.3 Principal Component Analysis (PCA)

Principal Component Analysis is a way of identifying patterns in data, and expressing the data in such a way as to highlight their similarities and differences. Since patterns in data can be hard to find in data of high dimension, where the luxury of graphical representation is not available, PCA is a powerful tool for analysing data [49]. PCA is a very useful tool for data compression without loss of much information.

With the assumption that PCA is now limited to reexpressing the data as a linear combination of its basis vectors. Let X and Y be *m* x *n* matrices related by a linear transformation P. $X$is the original recorded data set and Y is a re-representation of that data set (Equation 3.15)[50].

$$PX = Y \tag{3.15}$$

Where

- p_i are the rows of P
- x_i are the columns of P
- y_i are the columns of P

Equation 3.15 represents a change of basis and thus can have many representations.

- P is a matrix that transforms X into Y
- Geometrically, P is a rotation and a stretch which again transforms X into Y
- The rows of P, $(p_1, \cdots, p_m)$ are a set of new basis vectors for the columns of X.

Each coefficient of y_i is a dot product of x_i with the corresponding row in P. In other words, the j^{th} coefficient of y_i is a projection on to the j^{th} row of P. The rows of P are indeed a new set of basis vectors for representing of columns of X[50].

3.3 Supervised Learning Approaches on BCI

Linear classifers are discriminant algorithms that use linear functions to distinguish classes. They are probably the most popular algorithms for BCI applications. Two main kinds of linear classifers have been used for BCI design, namely, Linear Discriminant Analysis (LDA) and Support Vector Machine (SVM) [51].

3.3.1 Linear Discriminant Analysis (LDA)

The aim of LDA (also known as Fisher's LDA) is to use hyplerplanes to separate the data representing the different classes [52]. LDA assumes normal distribution of the data, with equal covariance matrix for both classes. This technique has very low computational requirements which makes it suitable for online BCI system. The main drawback of LDA is its linearity that can provide poor results on complex nonlinear EEG data[51].

3.3.2 Support Vector Machine (SVM)

An SVM also uses a discriminant hyplerplane to identify classes [51]. However, concerning SVM, the selected hyperplane is the one that maximizes the margins (e.g. the distance from the nearest training points (Figure 3.2[51]). Maximizing the margins is known to increase the generalization capabilities. An SVM uses a regularization parameter C that enables accomodation to outliers and allows errors on the training set.

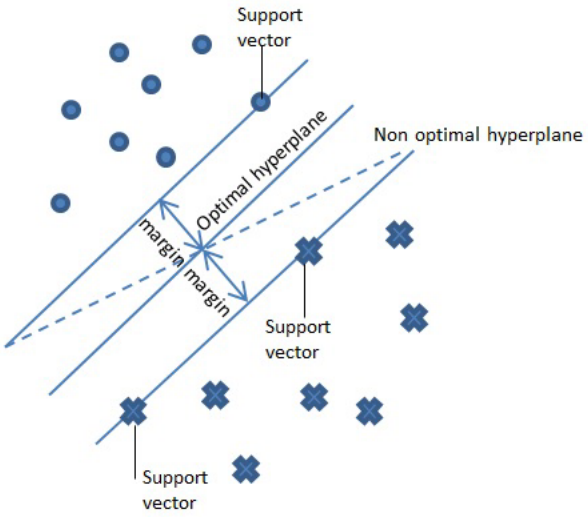

Figure 3.2: SVM for optimal hyperplane generalization

This classifier has been applied, always with success, to a relatively large number of synchronous BCI problems [53][54]. It is also possible to use the "kernel trick" for nonlinear decision boundaries. It consists of mapping the data into another space (generally of much higher dimensionality) by using a kernel funcion $K(x, y)$. In BCI research, usually the Gaussian or Radial Basis Function (RBF) kernel are used (Equation 3.16).

$$K(x, y) = e^{(\frac{-||x-y||^2}{2\sigma})^2} \tag{3.16}$$

The corresponding SVM is known as Gaussian SVM and RBF SVM. SVM has several advantages. Thanks to the margin maximization and regularization term, SVMs are known to have good generalization properties[51]. SVM have some parameters to be adjusted by hand, such as C and the RBF with σ if using kernel from Equation 3.16. These advantages are gained at the expense of a **low speed of execution**[51].

3.3.3 Density Based Support Vector Machines (DBSVM)

The Support Vector Machine (SVM) model proposed by Dr. Vapnik has been used as a form of supervised learning models in detecting certain features in the captured EEG data and in many other systems [58].

Unfortunately, the sensitivity of SVM to outliers is a weak point for this algorithm. One of the approaches for reducing this sensitivity is the Density Based Support Vector Machine (DBSVM) method, which is based in the simple expression of population density according to Equation 3.17 [59]:

$$Population\,density = \frac{number\,of\,population}{area} \tag{3.17}$$

Although the concept of population density is used to develop the DBSVM model, the formula is actually based on two separate algorithms to remove the outliers based on a vector's total Euclidean or Mahalanobis distance between its averages [60].

DBSVM with Euclidean Distance

For instance, let's suppose a set $(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)$ of n numbers of two-dimensional data is given . The Euclidean distance between two points is given by Equation 3.18:

$$D(1, 2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \tag{3.18}$$

The next step is to sum up all the distances for each point where n is the number of data points in the data set (i.e. $d_1 = [D(1, 2) + D(1, 3) + ... + D(1, n)]$. The total distance for all data points of one data set is needed to calculate the average distance that will be used to

determine data points inside and outside of densely populated area. Equation 3.19 shows the expression to calculate this:

$$Average_d = \frac{\sum_{j=1}^{N}\sum_{i=1}^{N}\sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}}{n}; \; if \; d_i > Average_d \rightarrow x_i = outlier \tag{3.19}$$

After calculating $Average_d$ by formula, those points that are not considered as outliers will become the new training data set in which the points will be considered important data points.

DBSVM with Mahalanobis Distance

The Mahalanobis distance is the distance between x from quantity μ. This distance is based on the correlation between variables or the variance-covariance matrix S which is defined by:

$$S_{xy} = cov(x, y) = E[(x - \mu_x)(y - \mu_y)]; \mu_x = E[x] \tag{3.20}$$

To determine the distance from two data points, the formula from Equation 3.21 is used:

$$D_m(x) = \sqrt{(x - y)^T S_{xy}^{-1}(x - y)} \tag{3.21}$$

Therefore, as used for the Euclidean Distance average calculation, the average Mahalanobis distance for excluding outliers is as described in Equation 3.22:

$$Average_d = \frac{\sum_{i=1}^{N}\sqrt{(x_i - y)^T S_{xy}^{-1}(x_i - y)}}{n} \tag{3.22}$$

3.4 Unsupervised Learning Approaches

3.4.1 Self-Organizing Maps (SOM)

The Kohonen Layer Self-Organizing Maps provide a way of representing multidimensional data in much lower dimensional spaces, usually one or two dimensions (vector quantization). In its simplest form it produces a similarity graph of input data[61].

It converts nonlinear statistical relationships between high-dimensional data into simple geometric relationships of their image points on a low-dimensional display. Usually, a regular two-dimensional grid of nodes is used. Assume vector $\mathbf{x} = \{\xi_1, \xi_2, ..., \xi_n\}^T \in \Re^n$. With each element in the SOM array, it is possible to associate a parametric real vector $\mathbf{m}_i = \{\mu_1, \mu_2, ..., \mu_n\}^T \in \Re^n$ that is called the model.

Assuming a general distance measure between $\mathbf{x}$ and $\mathbf{m}_i$ denoted $d(\mathbf{x}, \mathbf{m}_i)$, the image of an input vector $\mathbf{x}$ on the SOM array is defined as the array element m_c that matches best with $\mathbf{x}$. In

this research, the most common approach for determining the winner c, described by Equation 3.23 is used.

$$c = argmin_i\{d(\mathbf{x}, \mathbf{m}_i)\} \tag{3.23}$$

Differing from the traditional vector quantization, $\mathbf{m}_i$ is to be defined in such a way that the mapping is ordered and descriptive of the distribution of $\mathbf{x}$. For effective mapping, it will suffice that the distance measure is defined over all occurring $\mathbf{x}$ items and a sufficiently large set of models $\mathbf{m}_i$ [61]. An interesting feature of SOM is that the neuron grid m can be organized into different structures in an n dimensional space. Figure 3.3 illustrates the SOM structure and best matching unit selection [62].

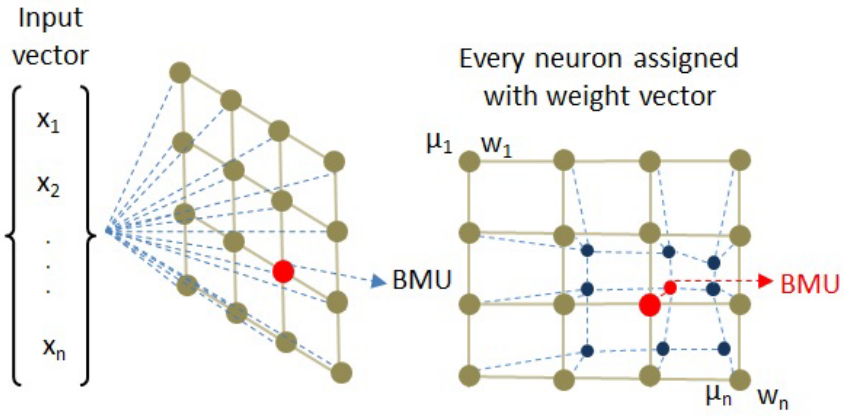

Figure 3.3: SOM Structure and Update of Best Matching Units

3.4.2 Batch Self-Organizing Maps (BSOM)

The SOM training algorithm constructs a nonlinear topology preserving mapping of the input data set $X = (x(t)|t \in (t_0, \cdots, t_f)$ where t_0 and t_f are the beginning and the end of the current training session, onto a set of neurons $M = \mu_1, \cdots, \mu_n$ of a neural network. The neurons of the network are arranged in a grid, with associated vectors[63]:

$$W = w_1(t), \cdots, w_n(t) \tag{3.24}$$

at a given time step t. W is known as the code book. Each data point $x(t)$ is mapped to its best matching unit

$$bm(x(t)) = n_b \in M \tag{3.25}$$

From the previous description, d is the distance function on the data set in the feature space. This can be redefined as Equation 3.26

$$d(x(t), w_b(t)) \leq d(x(t), w_j(t)) \, \forall w_j(t) \in W \tag{3.26}$$

The neurons re arranged on a two dimensional map (Figure 3.3): each neuron i has a two coordinates in a grid. Next, the weight vector of the best match neuron (BMN) and its neighbors are adjusted toward the input pattern using Equation 3.27:

$$w_j(t+1) = w_j(t) + \alpha h_{bj}(t)(x(t) - w_j(t)) \tag{3.27}$$

where $0 < \alpha < 1$ is the learning factor, and $h_{bj}(t)$ is the neighborhood function that decreases for neurons further away from the best match neuron in grid coordinates. A frequently used neighborhood function is the Gaussian (Equation 3.28):

$$h_{bj} = e^{(\frac{-||r_b - rj||}{\delta(t)})} \tag{3.28}$$

where r_b and r_j stand for the coordinates of the respective nodes. The width $\delta(t)$ decreases from iteration to narrow the area of influence. Eventually, the neighborhood function decreases to an extent that training might stop.

The time needed to train a SOM grows linearly with the dataset size, and it grows linearly with the number of neurons in the SOM. SOM has a batch formulation of updating the weights, which is widely used in parallel implementations(Equation 3.29)[63]:

$$w_j(t_f) = \frac{\sum_{t'=t_0}^{t_f} h_{bj(t')} x(t')}{\sum_{t'=t_0}^{t:f} h_{bj}(t')} \tag{3.29}$$

While not directly related to the training, it is worth mentioning the U-matrix associated with a SOM, which depicts the average Euclidean distance between the code book vectors of neighboring neurons. Let $N(j)$ denote the immediate neighbors of a node j. Then the height value of the U-matrix for a node j is calculated as

$$U(j) = \frac{1}{|N(j)|} \sum_{i \in N(j)} d(w_i, w_j) \tag{3.30}$$

The purpose of the U-matrix is to provide a visual representation of the topology of the network[63]. The use of the batch implementation can allow the reduction of the execution time for this research purpose. Its implementation will be described in the next chapter.

3.5 Parallel Processing

The use of parallel operations within a single processor or multicore units is a major source of speedup, especially when complex taks might demand high amounts of time. Also, in many

applications, an equally important consideration is that memory capacity is required. Distributed processing also plays an important role when a network is required to be used more efficiently[55]. However, in this research the focus will be on the local system implementations that can allow onlince BCI classification implementations. This section describes some of the software and hardware resources that can allow speed/memory optimization for signal processing and classification purposes.

3.5.1 Parallel Processing Platforms and Libraries

In this research, one of the classification processing libraries implements the use of different multiprocessing platforms that will be analyzed in the next chapter; the tools utilized and described are introduced below.

OpenMP: The *de facto* standard for shared-memory programming

OpenMP has become the environment of choice of many shared-memory parallel programming practitioners. It consists of a set of directives which are added to self C/C++/FORTRAN code that manipulate threads without personal thread manipulation[55]. Figure 3.4 illustrates an example of a code flow with an OpenMP program[56].

Message Passing Interface (MPI)

MPI is merely a set of Application Programmer Interfaces (APIs), called from user programs written in C, C++ and other languages. It has many implementations, with some being open source and generic, while others are proprietary and fine-tuned for specifc commercial hardware[55].

GPUs and the CUDA architecture

In comparison to the central processor's traditional data processing pipeline, performing general-purpose computations on a graphics processing unit (GPU) is relatively a new concept[64]. A big breakthrough in GPU programming is through the CUDA programming environment. It was first driven by videogame developers [55] but currently used by trending markets. The use of multicore processors with CUDA architecture is becoming popular in the recent years. The CUDA environment uses C plus some C++ features.

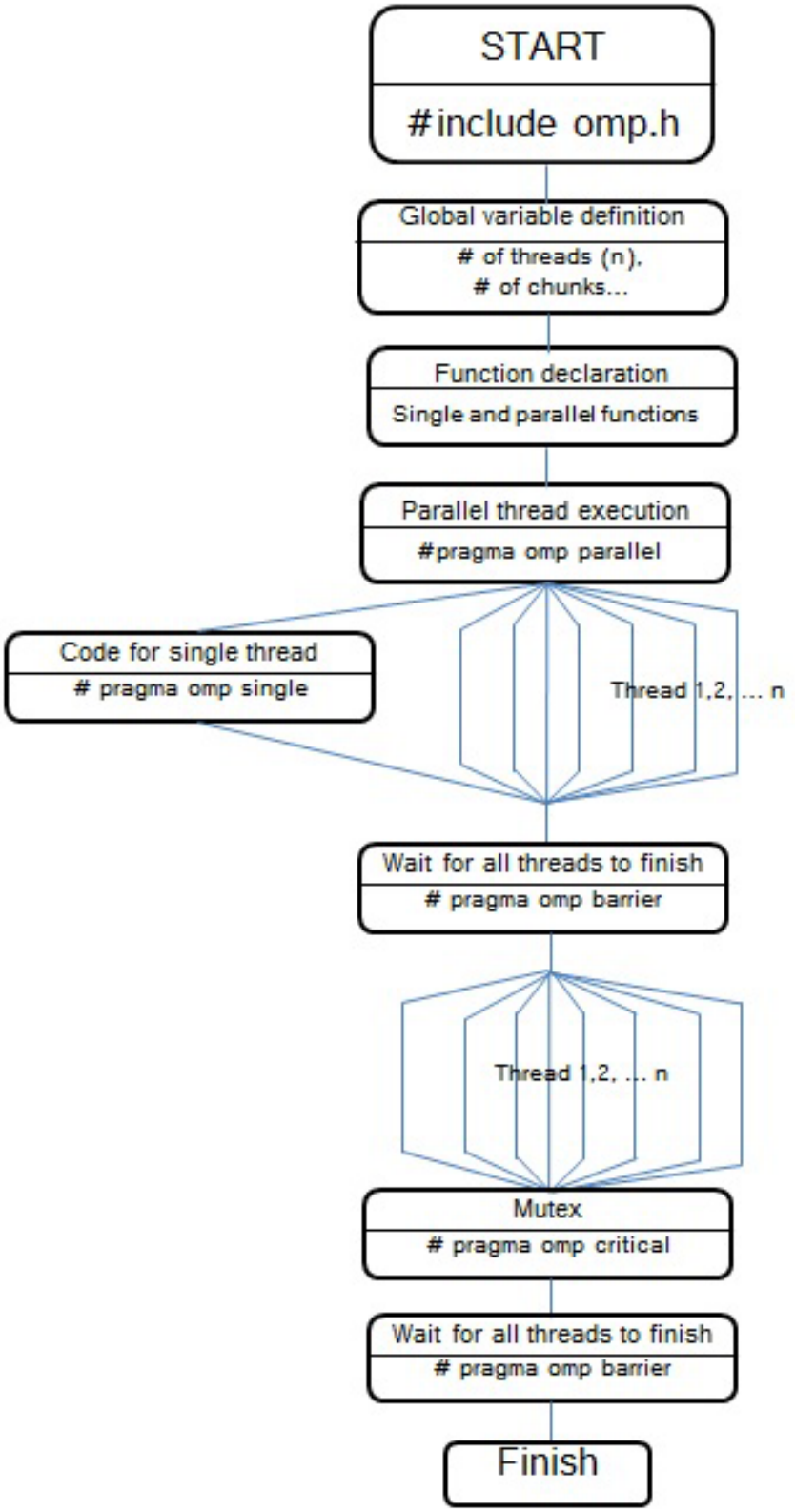

Figure 3.4: Workflow Example of an OpenMP Application

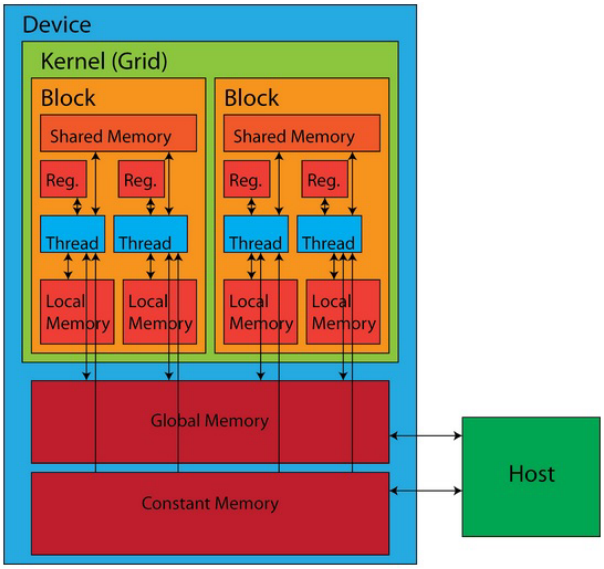

Figure 3.5: The CUDA Architecture

By observing the CUDA architecture from Figure 3.5[65]. As a brief introduction, the CUDA runtime executes multiple copies of a program in parallel on what is called *blocks*. A *grid* is a collection of blocks in wich the GPU is launched. A grid can be either a one or two dimensional collection of blocks. Each copy of the kernel can determine which block is executing with a built-in variable called *blockIdx*. Likewise, it can determine the size of the grid by using the built-in variable *gridDim*.

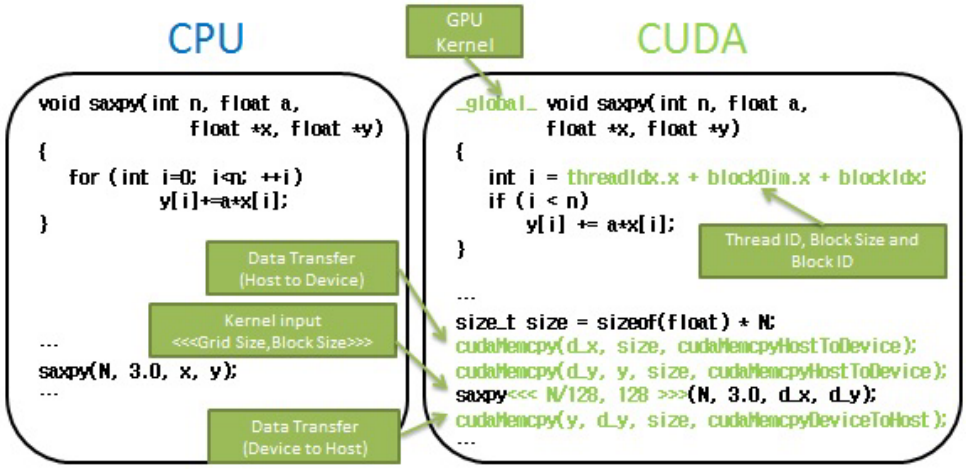

Figure 3.6: CUDA Implementation of function SAXPY(Y=A*X+Y)

Both of these built-in variables proved useful within the kernel to calculate the data index for which each block is responsible. A short segment of sample code might allow to understand the way the memory hierarchy is respected in order to assign tasks to the GPU through simple CUDA calls (Figure 3.6[66]).

3.5.2 SOM Parallelism: Somoclu

Currently available, there are software tools like SOM on Cluster (Somoclu)[63] that can help to provide a parallel implementation of the SOM algorithm by batch processing[63], allowing speed performance and reduction of execution time.Moreover, the use of this code library implements the main CPU speed perfomance by utilizing OpenMP-based parallelization (instead of MPI calls) or even utilize GPU resources on capable devices by the use of a sparse kernel CUDA operation with Thrust high primitives, (Figure 3.7[63]).

Another important advantage of this tool is that it allows easy integration with other interfaces such as Python, R or even Matlab. This options presents a suitable form of online unsupervised classification on GPU capable devices, especially on novel embedded systems with such characteristics.

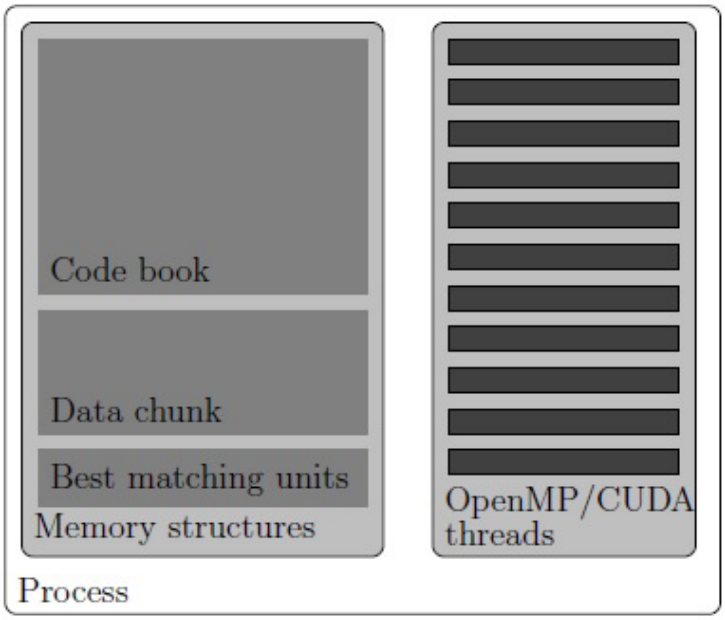

Figure 3.7: Parallel organization for SOM batch training

The use of high memory and high processing speed hardware may allow the implementation of extremely high dimensional data without the use of reduction in order to map pure raw data, as shown in the next chapter.

3.5.3 High Speed Parallel Processing Embedded System Tools

In this research, some trendy embedded hardware devices were tested with the purpose of achieving the objectives stated on Chapter 1[67][68][69]. However, many of them did not provide an adequate development environment in terms of high definition graphic display and parallelism at the same time for online classification purposes as well as cost effectiveness.

At the time of hardware testing, many newer devices were being coming up on the sales line, however, for this research, the most suitable device at the time was selected.

The NVIDIA Jetson TK1

The use of a GPU-based device suggests powerful threading capabilities to perform various processing tasks while executing extensive calculations in the background.

The selection of a powerful GPU based System on Chip (SoC) led to the use of a development board called NVIDIA Jetson TK1[70] which is featuring a NVIDIA "4-Plus-1" 2.32GHz ARM quad-core Cortex-A15 CPU with Cortex-A15 battery-saving shadow-core CPU and an NVIDIA Kepler "GK20a" GPU with 192 SM3.2 CUDA cores (up to 326 GFLOPS) [70]. The board and Kepler Architecture diagram is presented in Figure 3.8.

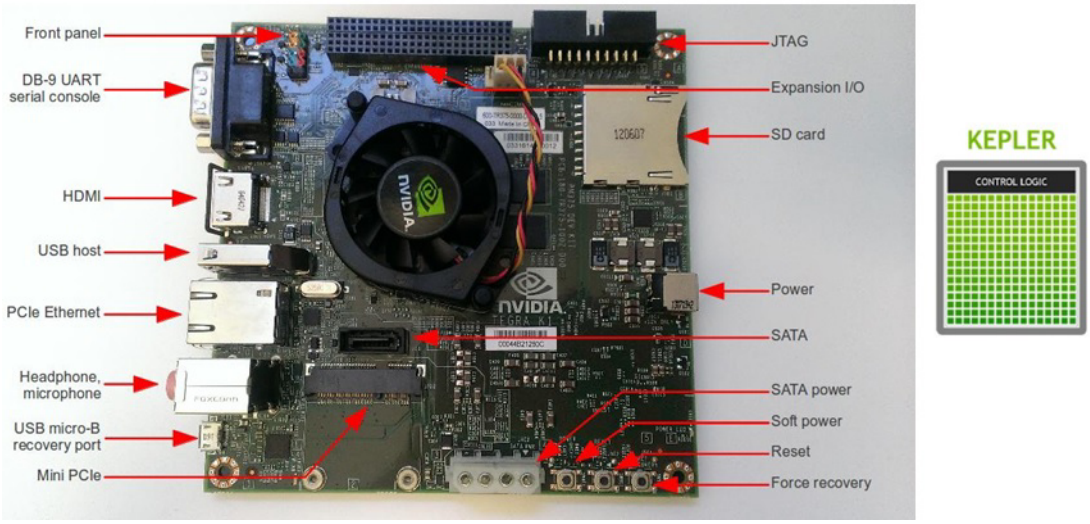

Figure 3.8: The NVIDIA Jetson TK1 and the Kepler Architecture

For this case, the architecture represents an almost (2014) state-of-the-art CPU-GPU combination (as of 2014) with computing capability of 3.2 [71]. In order to work out with the development board, some features were activated such as OS installation, Kernel compilation for WiFi and native CUDA compiling as well as the installation of the proper environment for CUDA development (Appendix A).

3.6 Virtual Environment Development Software

In order to obtain a series of training sets based on a conducted activity, for this research it was necessary to develop a training environment that could be transfered from a multicore CPU to a small embedded system computer while preserving its high resolution graphic properties. The tools that were used to develop the environment are described briefly below.

3.6.1 Blender

Blender is a professional free and open-source 3D computer graphics software product used for creating animated films, visual effects, art, 3D printed models, interactive 3D applications and video games. Blender's features include 3D modeling, UltraViolet unwrapping, texturing, raster graphics editing, rigging and skinning, fluid and smoke simulation, particle simulation, soft body simulation, sculpting, animating, match moving, camera tracking, rendering, video editing and compositing. It further features an integrated game engine[72]. For this research, the desing of different models for the environment were created through this environment. The main character can be seen in high resolution in Figure 3.9.

3.6.2 Unity3D

Unity is a cross-platform game engine developed by Unity Technologies and used to develop video games for PC, consoles, mobile devices and websites. First announced only for OS X, at Apple's Worldwide Developers Conference in 2005, it has since been extended to target 21 platforms[73]. Figure 3.9 shows an integration between Blender and Unity.

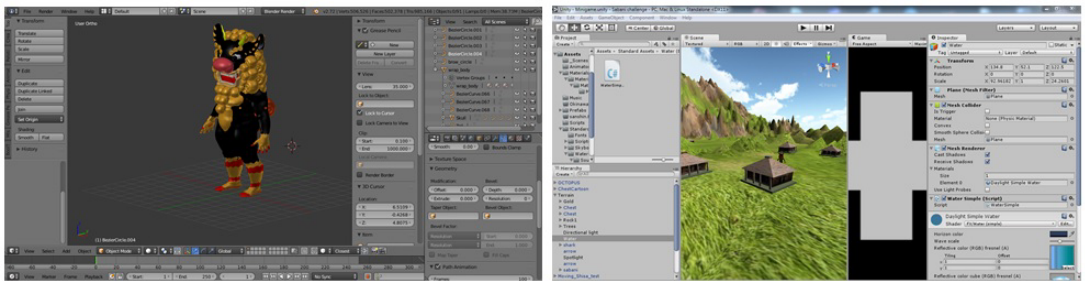

Figure 3.9: Virtual environment development with Blender and Unity

4 System Integration

In this chapter, the implementation of the different algorithms and software is explained. Furthermore, the integration of the software results is used to implement a proper prototype-like solution to an embedded hardware system. First, the signal creation and acquisition methods are explained. Next, their processing and classifications are detailed. Finally, the hardware integration and system implementation is described.

4.1 Software Implementation

4.1.1 Initial Brain Wave Pattern Proposal and Tests

There are different ways to measure brain wave signals from EEG that include both asynchronous and synchronous discriminative methods in order to compare and fit a signal output to an expected result. For the Neurosky Mindwave case, the use of the eSense Attention and Mediatation features will be used for data acquisition purposes.

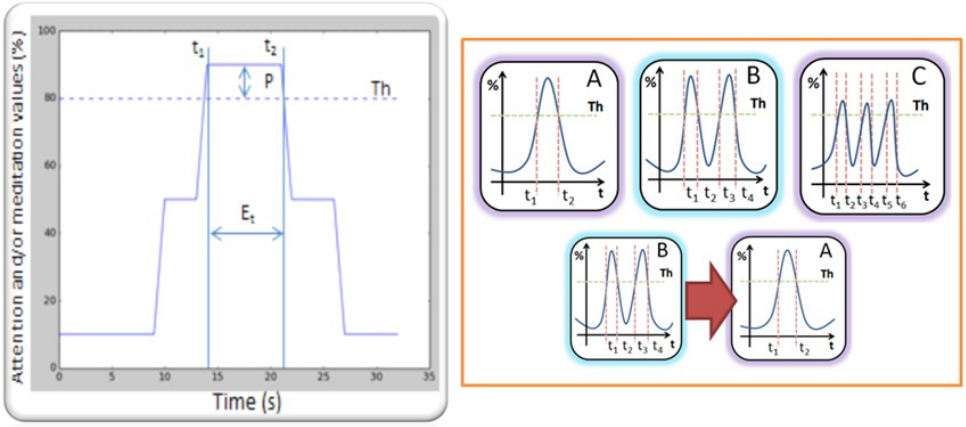

Figure 4.1: Mind pattern curve parameters and some examples

Due to the fact of the slow signal frequency output, which is one output set of attention and meditation values per second, the reduced number of EEG signal inputs from the EEG device and the ease to obtain more detailed values such as attention and meditation behaviors, the use

of a synchronous signal processing method was used (Figure 4.1[74]). This signal forms were defined as mind patterns.

The mind pattern generation method used consists of setting a specific time frame set in which a threshold value is configured depending on the attention and/or meditation percentage output from the EEG device, where 0% represents no relative signal and 100% is related to a full attention or meditation behavior.

As detailed in Figure 4.1, the mind pattern curve is created by selecting a time frame in which the expected signal will be compared, a specific set of time in which either an attention or meditation peak value is expected such as the expected time frame E_t between t_1 and t_2 in which peak P is referred to a value that exceeds a configured threshold value Th.

Also, there can be different sorts of mind patterns for a time frame that can use either attention or meditation values. Combined patterns can also be created to measure two different parameters on a time frame.

In order to process the signal information, a filtering algorithm is used to tell whether the signal is exceeding the threshold value or not and then compared to an expected pattern Value Over Zero check in order to verify that the value peaks are generated on the expected time (Appendix C).

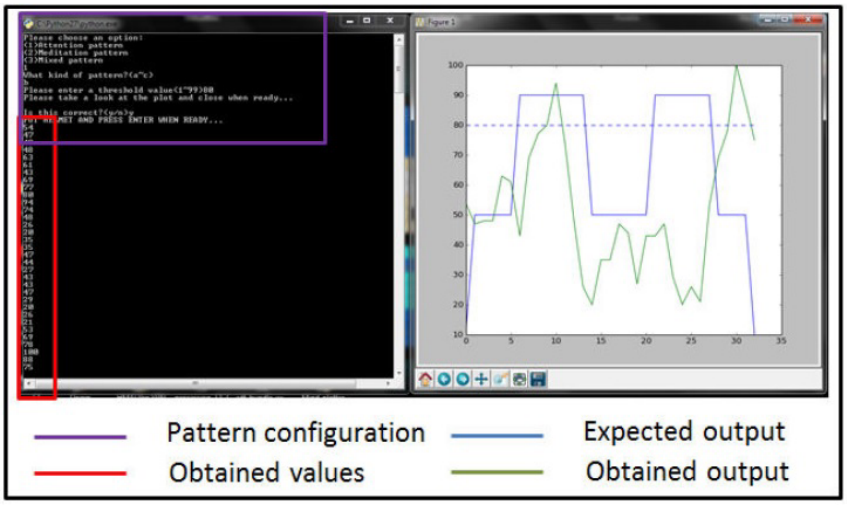

Figure 4.2: Mind pattern configuration and signal comparison

A GUI was developed to show how a mind pattern can be selected, configured and then compared to see if the obtained output from the EEG device matches the corresponding selection[74](Figure 4.2).

To test the brain wave matching capabilities of an end user both the attention and meditation value generation adaptability was tested in order to verify which one was simpler to recreate. For this case, meditation values resulted to a better choice.

From the mind wave patterns presented on Figure 4.1,patterns A, B and C were selected to be tested with five end users; three of them performed adaptability training tests before obtaining the corresponding data for the end results. Each pattern was to be recreated five times, each one with a corresponding threshold of 80% of meditation value on peaks.

After performing A, B and C pattern tests, an end test consisted of recreating pattern B followed by pattern A in order to see if the end user was able to recreate successfully one after the other to see how easy it could be for an end user to recreate a mind password set without failing with the present EEG device.

While Table 4.1 shows the specific results per user (users 1, 3 and 5 performed headset adaptability test previous to the experiment), Table 4.18 shows the average obtained success rate for the experiment.

Table 4.1: Specific User Test Results

Test	Pattern	Success Rate
1	A	80%
	B	60%
	C	40%
	B,A	NO
2	A	40%
	B	20%
	C	0%
	B,A	NO
3	A	80%
	B	20%
	C	20%
	B,A	YES
4	A	40%
	B	20%
	C	0%
	B,A	NO
5	A	80%
	B	80%
	C	0%
	B,A	YES

Table 4.2: Overall Average Success Rate

Pattern	Average Success Rate
A	64%
B	65%
C	15%
B,A	40%

While the overall average success rate for patterns Aand B was above 60%, pattern C was extremely difficult to recreate according to the end users, due mainly to the fact of having a small recovery time between peaks. It is important to point out the fact that the overall results include both trained and untrained end users while the specific user test results show evidence of better performance with trained users. Still, the generation of a pattern sequence such as B and A had a success rate below 50% for an overall result but a success of 66% when taking only trained users as the parameter[74].

The use of these results suggest that a trained user can perform a specific set of patterns but improves at a certain number of repetitions. Also, it is possible to recreate the signals on a bigger scale and enhance the classification of certain patterns from approximate signals that might not have been produced due to user or device errors, but an intention of generating that especific signal may be grasped. In order to attend this objective, which is also stated in Chapter 1, a bigger environment was created and more complex signals were processed and classified.

4.1.2 Brain Wave Signal Classification System

In order to acquire and classify more complex signals, and based on a certain degree of success from the initial tests, the system from Figure 4.3 was implemented.

An Arduino Uno board was used as an RF/RS232 data brigde between the headset and the computer script and an Arduino Micro was used for keyboard simulation purposes (Appendix D). The aim of this research is to elaborate a method for recognizing brain wave patterns for authentication or command application development[76].

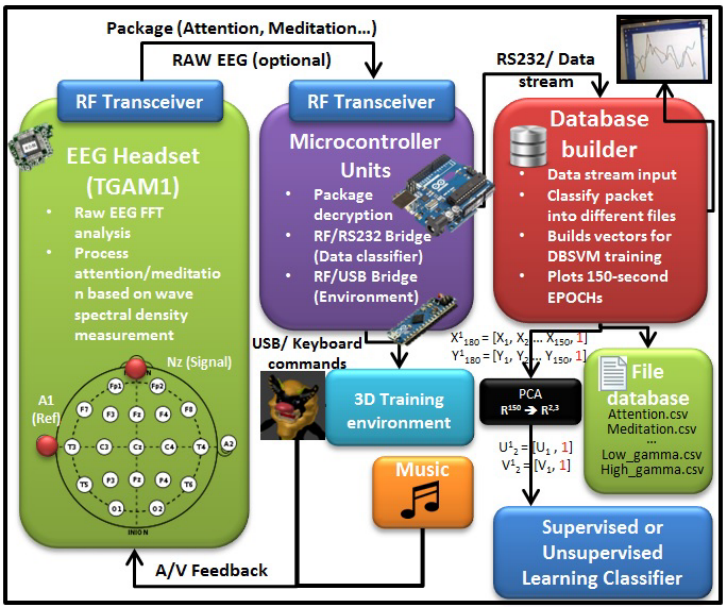

Figure 4.3: System Description

4.1.3 The 3D Virtual Training Environments

The use of simulators for training with EEG Biofeedback is a common application and has been proven effectiveness in solving problems such as attention enhancement as described in Chapter 2.

The proposed training environment was designed with the use of Blender for the animated models and structures and Unity3D for its ease of use and deployment. The environment has a series of features such as music and interaction with the experiment variables that are described as follows.

Ganbare Shisa Path Tracing Environment

The environment setup was created for the test subjects to familiarize with the objects and understand the way of interacting with them; an additional relaxation music track can be used for aiming in obtaining higher meditation behaviors.

The purpose of the training task is to guide the player (training subject) throughout a predefined path, as presented on Figure 4.4.

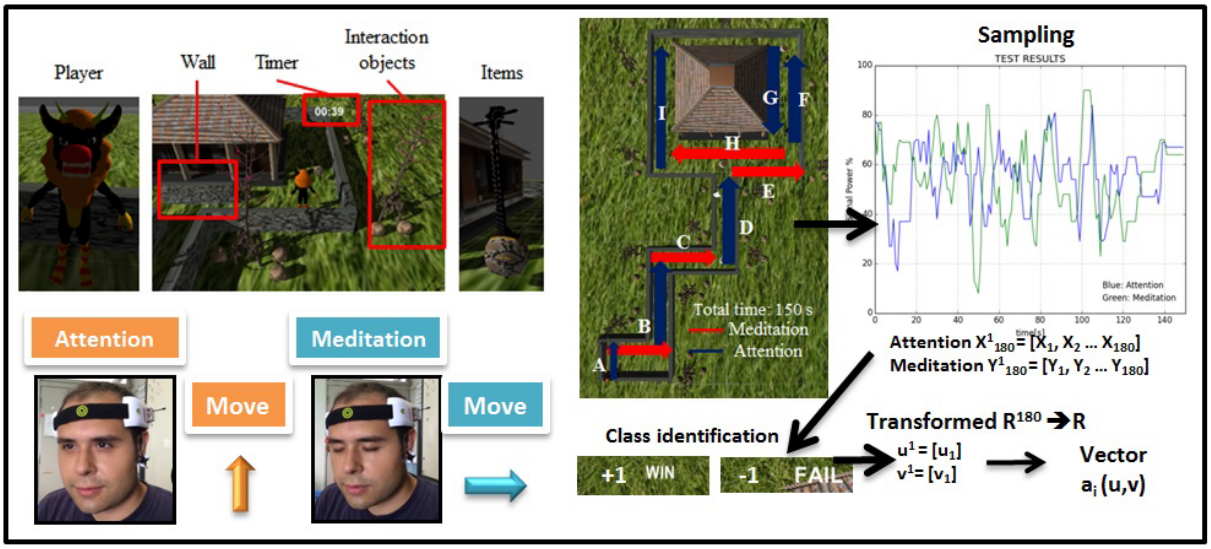

Figure 4.4: System Description

When concentrated, the user can move the player forward and then move the player to the right if the user begins to enter meditation. In either case, the user can stop the player by blinking the eyes constantly until the player stops due to the artifact noise produced across the path, there are four main items that the subject requires to obtain.

If the series of mind-waves came into the expected pattern in the required time, the attempt will be classified as successful and the class *+1* will be labeled for the output EPOCH. However, if the test subject is not able to perform the entire track along the timeframe, an unsuccessful attempt is recorded with the *-1* label.

Brain waves are dependent on each test subject, and so the performed wave patterns and headset calibration. Due to this factor, the training requires to be performed individually. Fortunately, the device has a fast self-calibration routine and a trained subject can easily adapt in controlling the headset in minutes.

In order to have a good classifier, 30 samples where obtained from the test subject. In the case it is complicated to generate the attention or meditation patterns, the use of the additional music tracks can stimulate the brain for generating the necessary beta or alpha waves for the device.

The testing area was a quiet room with the test subject sitting in front of a screen displaying the 3D environment and the data processing script running in the background for obtaining the corresponding data sets. Whenever the test subject was ready to perform a test a red button was pressed to restart the environment and script. Also, other virtual environments were produced in a sense of having an alternative for other brain motion signal acquisition or helping the users to adequate to the proper use of a specifig device (trainer application).

Sabani Challenge Motor Imagery Training Environment

The purpose of this setting is a path in which the user can decide to move either left or right, depending motor imagery signals. At the start of each sesion, an octopus and a treasure chest are presented in random positions (e.g. octopus on right side and treasure chest on the left). During online training, the signals can be recorded. Figure 4.5 shows the environment in which this application was tested.

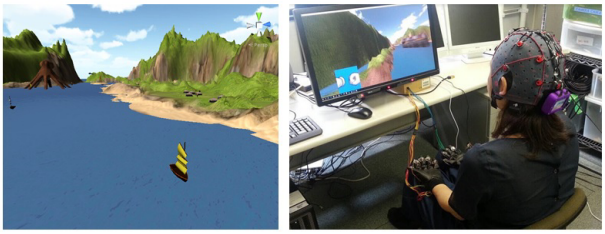

Figure 4.5: The Sabani Challenge Brain Motor Training Environment

4.1.4 SVM Implementations

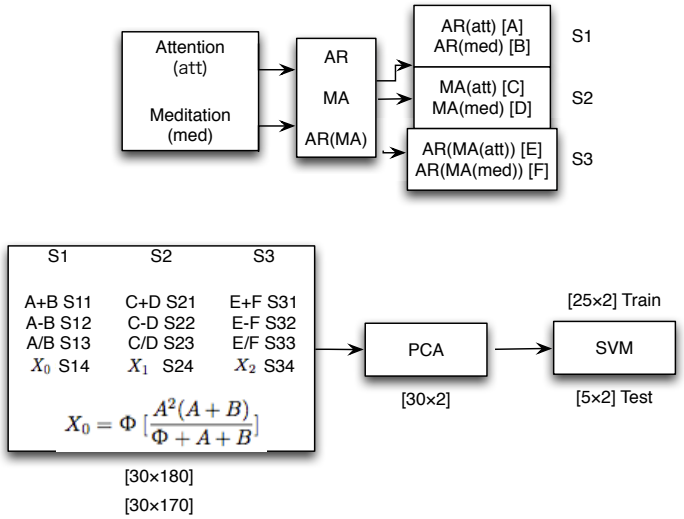

Figure 4.6: SVM Feature Extraction and Training

From the system description(Figure 4.6), the system provides a set of attention and meditation sets. Each sample has a length of 180 samples and consists of both an Attention **A** and Meditation **B** vectors. In order to obtain a feature vector that could provide a good real time classifier by using SVM, different test vectors were utilized. However, the array that could provide the best classification results was formed by using the empirical Equation 4.1 [75] for feature vector **X**:

$$X = \frac{\phi A^2(A-B)}{A+B+\phi}; \phi = 0.001 \tag{4.1}$$

Once the vectors were defined, other signal processing techniques such as autoregressive modeling and moving average of the features were utilized to train the SVM classifier [77] as shown in Figure 4.6.

The first dataset contains the autoregressive coefficients (AR) of the initial attention and meditation values, namely Attention and Meditation respectively. These resulting vectors are then trimed from their first component (since the first component is always one) and then labeled A for AR(Attention) and B for AR(Meditation). In such a similar way there is labeling for Moving Average (MA) of Attention (C) and Meditation (D), as well as for the Autoregressive coefficients of the Moving Average of Attention (E) and Meditation (F) vectors.

There was a grouping of three major subsets, S1, S2 and S3. The first subset contains a combination of the data that was labeled previously. For instance, for the first subset, S1, there is a first vector S11 consisting of A+B, a second component S12 conformed by A-B, a third one S13 generated by A/B (if B is zero at some point, that specific component also becomes zero). Figure 4.7 presents some examples.

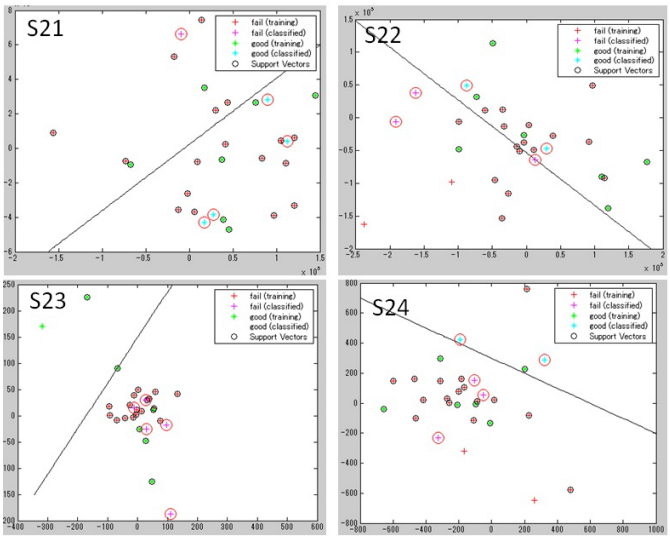

Figure 4.7: Subset S2 examples

After getting all the components, a dimension reduction is done by means of Singular Value Decomposition and Principal Component Analysis in order to be able to visualize the data to be trained an tested. Finally, on a 85-15 basis form the 30 initial samples (25 training samples and 5 test samples) the corresponding Support Vector Machine Classifiers were trained and tested.

The results from this classification procedure are presented in Table 4.3

Table 4.3: 5-fold SVM Classification Results

Subset	Avg. Rate	Subset	Avg. Rate	Subset	Avg. Rate
S11	80%	S21	60%	S31	80%
S12	80%	S22	20%	S32	60%
S13	80%	S23	32%	S33	80%
S14	80%	S24	60%	S34	60%
S1	80%	S2	43%	S3	70%

4.1.5 DBSVM Implementations

For this implementation, just as with the SVMs, after performing PCA2 to the feature vector **X**, outlier removal and training was performed for the obtained samples on an 85-15 basis (85% training, 15% test) according to the outlier removal procedure described in Chapter 3. The kernels that were used for the SVM implementation were the linear, RBF and polynomial degree 3. The LinearSVC kernel uses a different code library that allows a more flexible modeling and better choice of penalties and loss function scaling[80]. The classification success rate for the test data set could be augmented if a soft margin is implemented. The resulting classifiers are presented on Figure4.8 and the resulting classification rates are shown on Table 4.4:

Table 4.4: DBSVM Classification Results for X

Kernel	Normal	Euclidean	Mahalanobis
Linear	20%	60%	60%
RBF	40%	40%	40%
LinearSVC	80%	80%	60%
Polynomial	60%	20%	20%

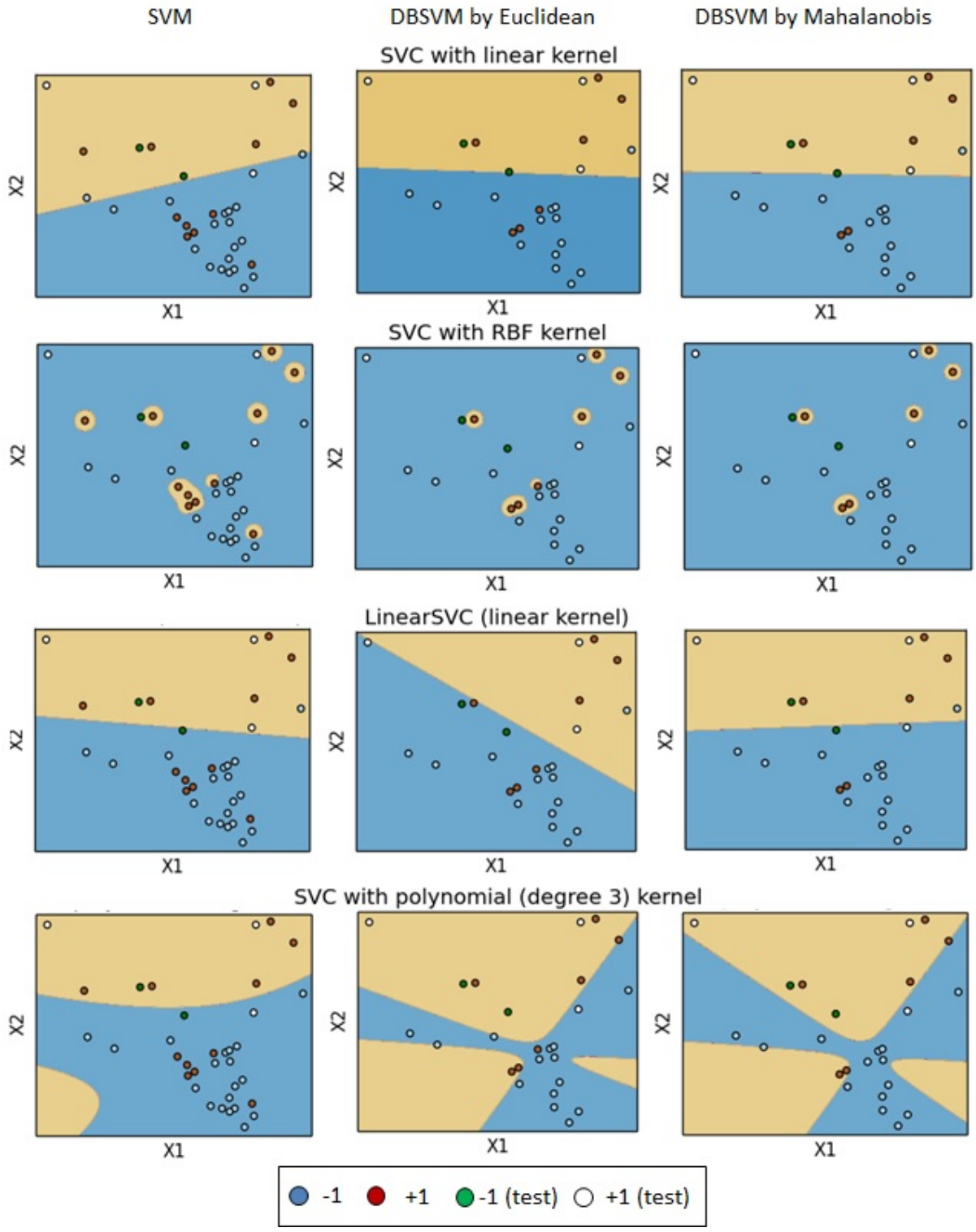

Figure 4.8: DBSVM classification results

4.1.6 SOM Implementations

For this part of the software implementation, whitening was applied to the feature vectors for different cases, three (PCA3), five (PCA5), seven(PCA7)and ten (PCA10) dimensions(Appendix E). The Scree plots after applying Singular Value Decomposition[79] for the Attention **A**, Meditation **B** and Feature vector **X** suggest that at least ten dimensions can be considered as the most important number of components to perform training for the Self-Organizing Map implementation[81](Figure 4.9).

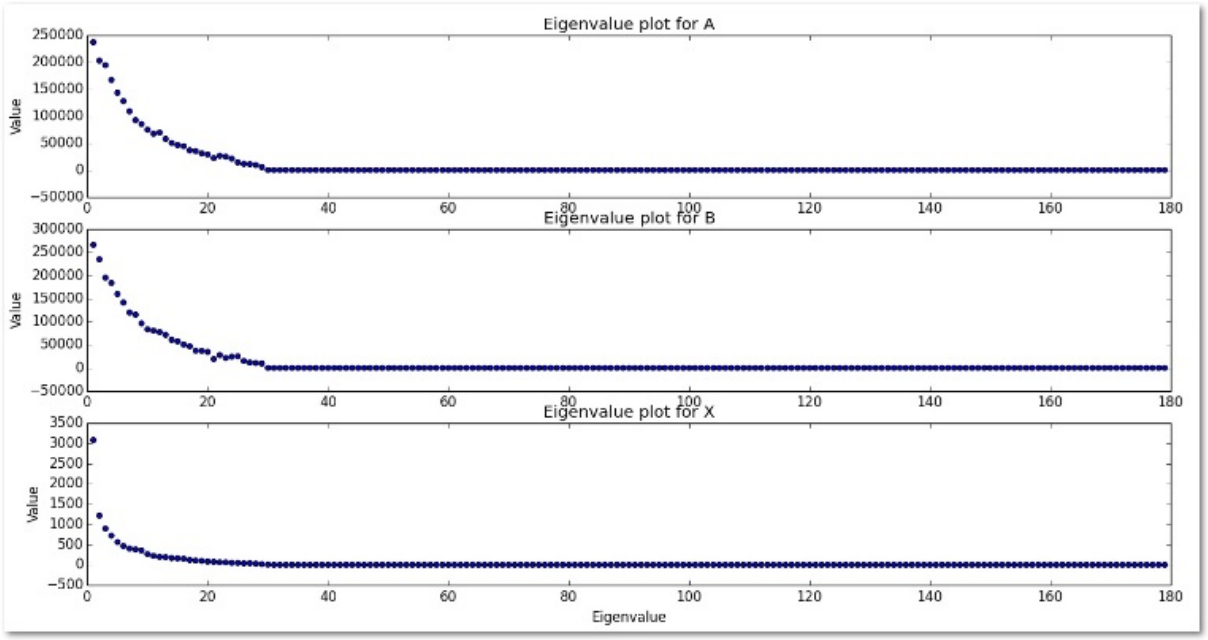

Figure 4.9: Scree plots of features A,B and X

In order to test the capabilities of Unsupervised Learning Method for classification, a small classification test was performed on the feature vector with a dimension reduction of three by PCA[78], because it allows distinguishing the colors of each of the regions.

A 30x30 grid Kohonen layer map with Gaussian kernel and exponential learning rate was implemented for the distance function described previously. At a glance, it looks that the map is not performing well when the number of iteration is low ($n < 10000$) [82]. However, once the number of iteration has been increased ($n > 20000$), the areas are getting defined and the map seems to be formed clearly, as presented in Figure 4.10 (the training is independent for each picture).

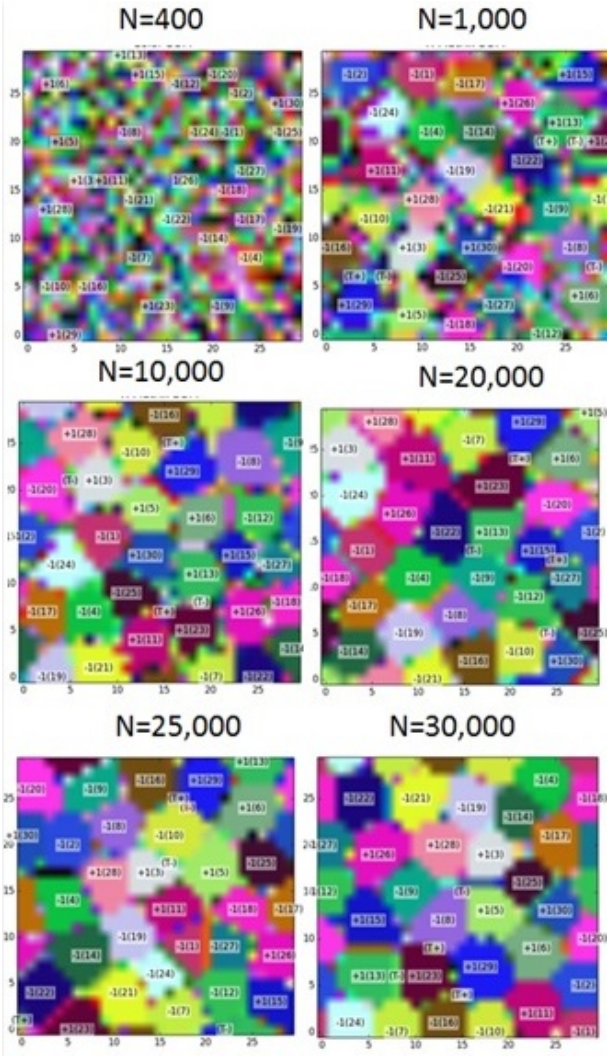

Figure 4.10: The Kohonen Map for feature X with PCA3

Each region represents the winners for the Feature vector **X** in the training data (which is labeled $+1$ or -1 depending on the case). After the SOM is created from the training vectors, the test vectors (containing the tags $+1$ or -1 for successful/ unsuccessful attempt) are mapped and compared to the output SOM in order to verify if they are being recognized correctly by SOM[81].

The use of different PCA components for the main Feature vector **X** and other features composed by the Attention and Meditation values were tested with the a Python library. The average processing speed for each classification trial was performed for five iterations of the SOM in each trial. 10- fold cross validation tests were performed in order to provide the average recognition rate for the test vectors.

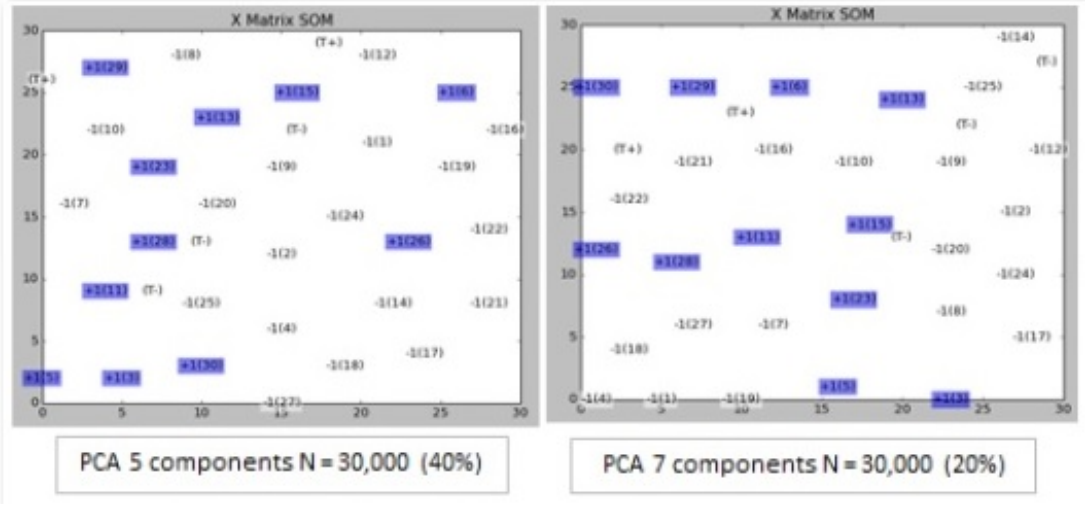

Figure 4.11: Examples of SOM for Feature vector X at 30,000 iterations with recognition rates

Table 4.5: 10-fold cross validation results for different features and recognition rates by SOM

Feature	**PCA3**			**PCA5**			**PCA7**			**PCA10**		
A	I	II	III	I	II	III	I	II	III	I	II	III
Recognition rate	0.52	0.52	0.68	0.4	0.4	0.52	0.6	0.64	0.52	0.4	0.56	0.40
Processing time (s)	455.95	572.90	686.31	528.89	645.54	786.77	568.69	737.39	855.77	670.03	828.05	982.87
Average time(s)	91.19	114.58	137.262	105.77	129.10	157.35	113.73	147.47	171.15	134.00	165.60	196.57
Performance ratio	0.57	0.45	0.49	0.37	0.30	0.33	0.52	0.43	0.301	0.30	0.34	0.20
B	I	II	III	I	II	III	I	II	III	I	II	III
Recognition rate	0.20	0.24	0.32	0.20	0.20	0.60	0.24	0.52	0.40	0.48	0.40	0.76
Processing time (s)	474.03	595.53	709.85	567.26	688.07	837.12	594.92	745.91	882.59	662.13	833.47	966.87
Average time(s)	94.81	119.11	141.97	113.452	137.614	167.42	118.98	149.18	176.52	132.43	166.69	193.374
Performance ratio	0.21	0.20	0.23	0.18	0.15	0.36	0.20	0.35	0.23	0.36	0.24	0.39
X	I	II	III	I	II	III	I	II	III	I	II	III
Recognition rate	0.26	0.32	0.4	0.4	0.44	0.56	0.40	0.60	0.64	**0.60**	0.60	0.60
Processing time (s)	444.21	557.27	665.48	518.43	698.27	839.45	608.77	744.97	865.29	**658.90**	797.70	1027.56
Average time(s)	88.84	111.45	133.10	103.69	139.65	167.89	121.75	148.99	173.06	**131.78**	159.54	205.512
Performance ratio	0.29	0.29	0.30	0.39	0.32	0.33	0.33	0.40	0.37	**0.46**	0.37	0.29
A+B	I	II	III	I	II	III	I	II	III	I	II	III
Recognition rate	0.44	0.56	0.60	0.40	0.36	0.64	0.20	0.24	0.44	0.40	0.12	0.32
Processing time (s)	484.21	1367.57	777.77	518.97	643.56	777.77	575.89	711.74	2006	640.12	811.07	1023.04
Average time(s)	96.84	273.51	155.55	103.79	128.71	155.55	115.18	142.34	401.2	128.02	162.21	204.61
Performance ratio	0.45	0.20	0.39	0.39	0.28	0.41	0.17	0.17	0.11	0.31	0.74 E-2	0.16
A-B	I	II	III	I	II	III	I	II	III	I	II	III
Recognition rate	0.44	0.4	0.56	0.48	0.52	0.4	0.48	0.44	0.44	0.4	0.56	0.60
Processing time (s)	483.21	643.56	777.77	518.97	643.56	777.77	518.97	643.56	777.77	658.90	797.70	1027.56
Average time(s)	96.64	128.71	155.55	103.79	128.71	155.55	103.79	128.71	155.55	131.78	159.54	205.51
Performance ratio	0.45	0.31	0.36	0.46	0.40	0.26	0.46	0.34	0.28	0.30	0.35	0.29
A/B	I	II	III	I	II	III	I	II	III	I	II	III
Recognition rate	0.28	0.54	0.54	0.20	0.20	0.32	0.12	0.36	0.36	0.40	0.64	0.72
Processing time (s)	448.45	570.56	679.58	518.97	643.56	777.77	518.97	643.56	777.77	662.13	833.47	966.87
Average time(s)	89.69	114.11	135.916	103.79	128.71	155.554	103.79	128.71	155.55	132.42	166.69	193.37
Performance ratio	0.31	0.47	0.40	0.19	0.15	0.21	0.12	0.28	0.23	0.30	0.38	0.37

Some of the output map examples are shown in Figure 4.11 (since the use of multiple dimensions does not output a colored map, the use of blue, $+1$, and white, -1, tags are shown in the figure)[81]. Table 4.5 presents these results for different number of iterations (I = 20,000, II=25,000 and III=30,000). The performance rate is the ratio between the recognition rate and the average SOM processing time[81].

4.2 Hardware Implementation

4.2.1 Embedded System Serial and Parallel Feature Extraction

Once the hardware (Jetson TK1 board) has been configured and all the necessary libraries installed (Appendix A). A small parallel feature obtention for feature vector **X** from section 4.1 was implemented on the embedded device's with the use of CUDA. In order to analize time vs. performance issues, the use of artificial data was implemented; the real dataset consists of 30 samples, but the simulated data is taking into account a bigger database, which was using the same data doubled and some artificial data (70 samples).

In order to compare the execution time of a serial and parallel implementation on the embedded board. In the case of the serial implementation(Appendix B1, the program uses simple memory allocation for the matrix operations that are used on by the main function.

Different memory arrangements were used to test the performance time of the GPU based code with the aid of the blocks and the main grid for a very large vector (a little change from the Serial Implementation which uses a multidimensional array). The memory is copied into de device and the feature extraction function is implemented by the CUDA environment (Appendix B2).

Table 4.6: Average execution time in msec

Type	Average Processing Time
CPU (Serial)	226.0232
CM/N=1/T=70	278.0130
CMH/N=1/T=70	273.4332
CM/N=2/T=35	266.0382
CMH/N=2/T=35	241.8568
CM/N=7/T=10	242.4340
CMH/N=7/T=10	244.9028
CM/N=10/T=7	237.4420
CMH/N=10/T=7	245.4042
CM/N=70/T=1	231.6398
CMH/N=70/T=1	229.8505

As referred by the literature[55], the command **cudaMalloc** was replaced for the instruction **cudaMallocHost** in order to use Direct Memory Access (DMA) from the Hardware and reduce by half the memory copying time; a little decrease in execution time was observed.

For a basic simulated dataset of 70 samples, five runs were executed for different memory arrays in order to test the processing time of the feature vector programs; the results are presented in Table 4.6, where N = Number of blocks, T = Number of threads, CM = **cudaMalloc** instruction and CMH = **cudaMallocHost** instruction).

4.2.2 Parallel Batch SOM (b-SOM) Implementations

The implementation of a parallel SOM could allow the use of the entire dimensional set without the need of a reduction, due to the power of the processing speed provided by GPUs and the OpenMP as described on the previous chapter. Also, as noted by the literature[63], the execution time of batch SOM (b-SOM) is linearly dependent of data size. A series of maps were implemented and observed for different features as performed for the previous classifiers [75] [76], varying from rectangular grids with different cell implementations. Some of the mapped nets and fired Best Matching Units (BMU) are presented on Figure 4.12.

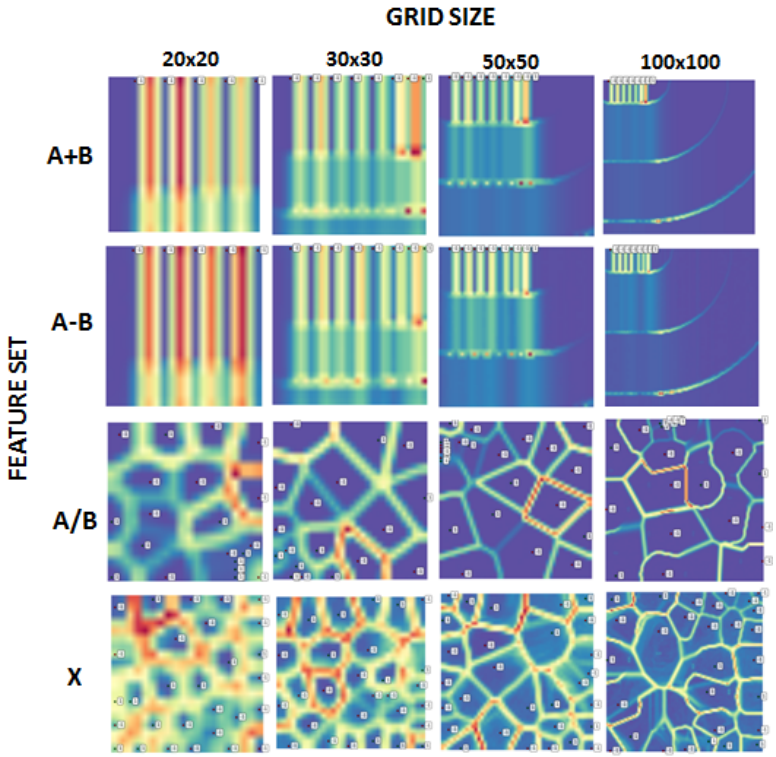

Figure 4.12: Different Feature Square Grid b-SOM Organizations

From this comparison, it can be determined that the one that provides a better clustering is feature matrix X, the data is better spread and the vectors associated with successful trials tend to be placed in the grid's center zone. Rectangular grid implementations did not provide good cluster separations.

4.2.3 Generation and Tuning of the SVM Classifier

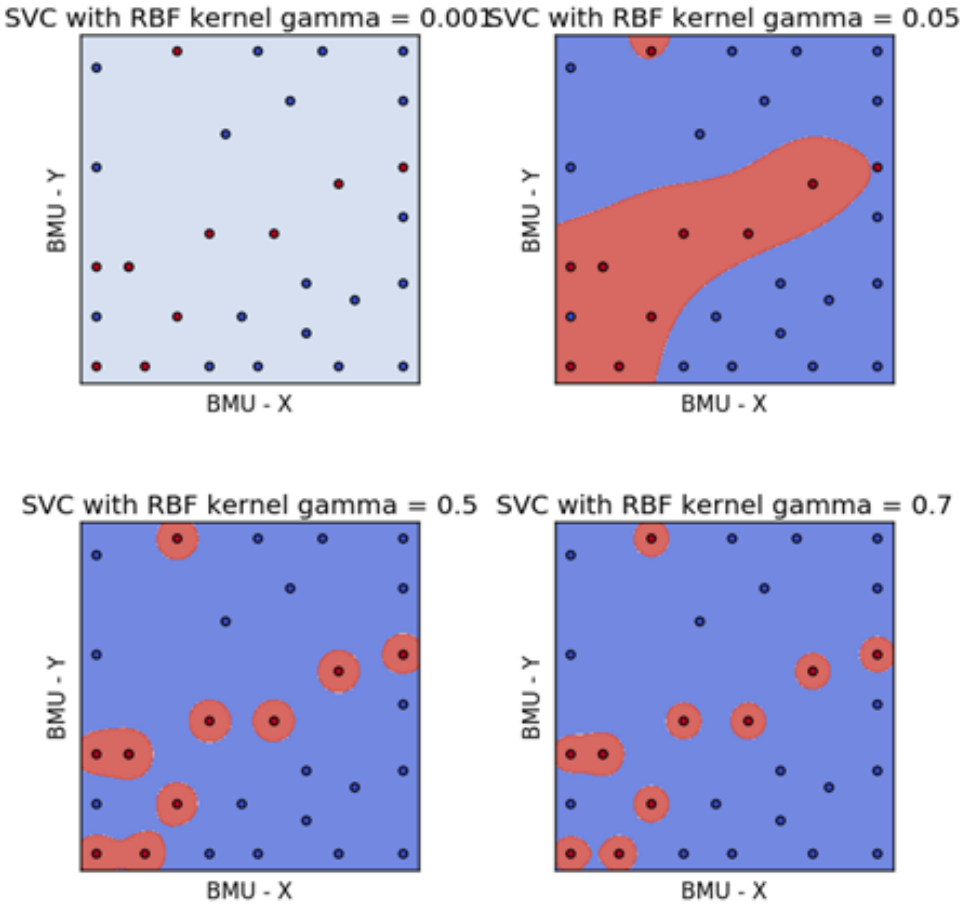

Figure 4.13: The Selected b-SOM BMUs at Different γ values for RBF-SVM

From the different generated maps, one of the best clusters was defined and selected for the training of an SVM classifier is a 20x20 b-SOM grid of feature matrix X and the BMU x and y coordinates (BMU-X and BMU-Y, respectively). The use of an RBF SVM Classifier is advised in order to obtain good classification results as stated on previous sections. After generating the classifier, it can be observed that the modification of parameter γ greatly affects the classifier (Figure 4.13). It has been found that for $0.10 \leq \gamma \leq 0.22$ the classifier gained robustness due to the division between the +1 and -1 attempt features (Figure 4.14).

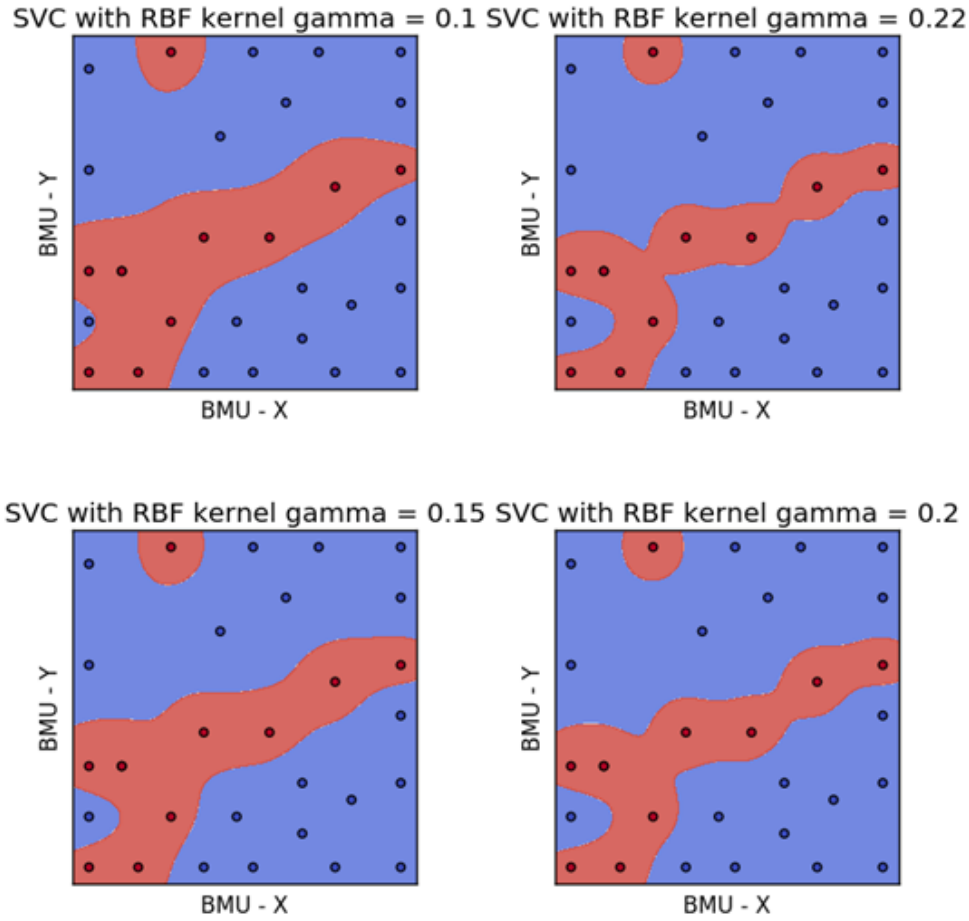

Figure 4.14: Classifier Outputs for $0.10 \leq \gamma \leq 0.22$

4.2.4 Classification Results

The "random" Cross-Validation of the previous set of classifiers was done with ten artificial signals from the random combination of +1 and -1 training vectors that were different from the originals. (e.g. a +1 **X** vector consisting from random +1 **A** and **B** trial selection) The generated test sample was entered as an update of the last b-SOM BMU vector update and the resulting BMU coordinates were then adjusted and tested with the RBF-SVM generated classifier. By additional tuning of the SVM C parameter, the margins could be modified to improve classification. The resulting test vector recognition rates for different C values are presented in Table 4.7. The best RBF-SVM classifier is presented on Figure 4.15. Note that there are two outliers, that even after removal do not alter the result.

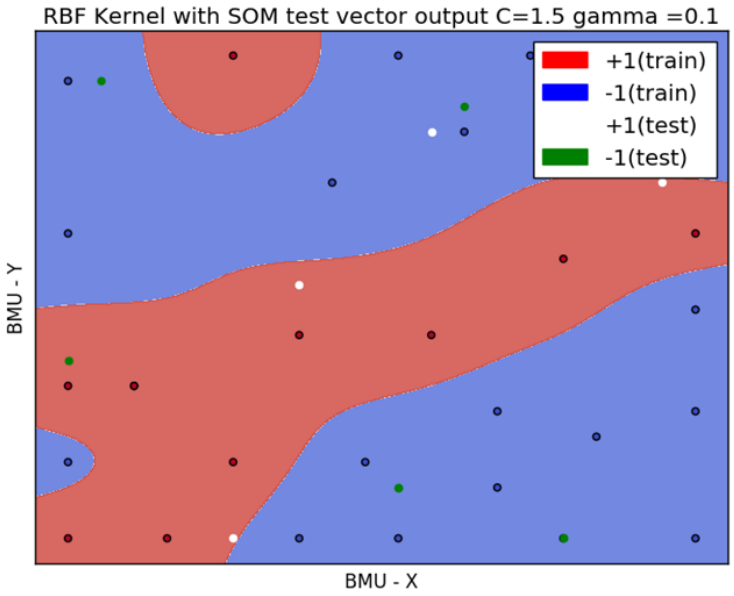

Figure 4.15: The Resulting b-SOM/RBF-SVM Classifier

Table 4.7: b-SOM/RBF-SVM Classification Results after random Cross-Validation

C	$\gamma = 0.1$	$\gamma = 0.15$	$\gamma = 0.2$	$\gamma = 0.22$
1.5	**70%**	70%	60%	50%
1.0	60%	50%	40%	40%
0.8	50%	50%	60%	60%

4.2.5 Execution Time Performance with OpenMP

In order to reduce execution time, the parallel b-SOM algorithm was run with OpenMP support (Appendix F). Five execution times for each grid were measured and averaged for the full classifier generation was measured. For implementing this classification method on an embedded system, the time performance should also be low. Table 4.8 and Figure 4.16 show the average time performance (in seconds) results for the tests with an Intel Core i7-2.30GHz (12MB RAM) computer and the proposed hardware. The plotting time (in case it is required) was also taken into account.

Table 4.8: Average time performance comparison (in seconds)

GRID SIZE	t_CPU	t_CPU /Plot	t_JetsonTK1	t_JetsonTK1 /Plot
6x6	0.04	0.04	0.78	0.96
15x15	0.53	0.18	4.68	5.04
20x20	0.95	1.15	7.73	8.52
30x30	2.08	2.46	17.97	19.13
50x50	5.60	6.78	48.25	52.65
100x100	36.54	40.25	-	-

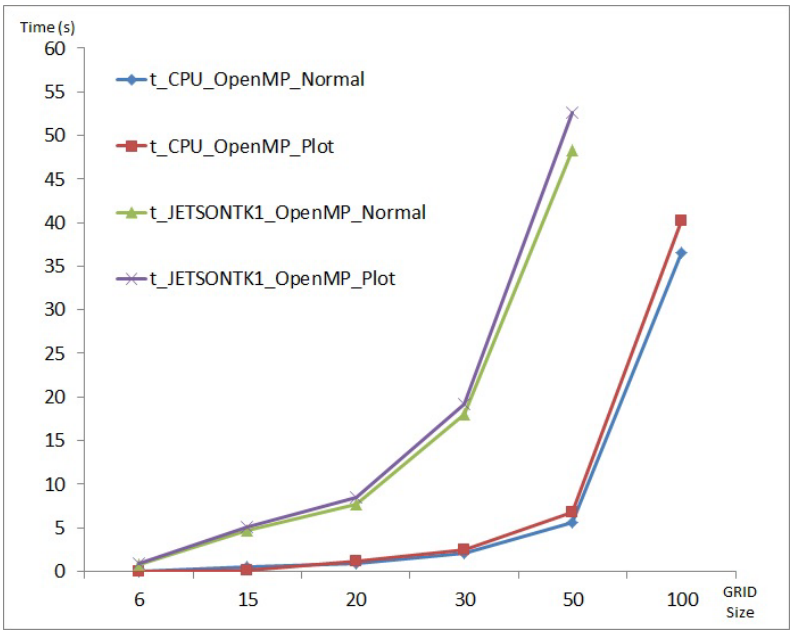

Figure 4.16: Time Performance vs. b-SOM Grid Size Plot (OpenMP)

4.2.6 CUDA Execution Performance

The CUDA implementation on the embedded board did not provide much overall benefits, due mainly to the small dataset size for the SOM training(Table 4.9 and Figure 4.17). However it is noted that the parallel optimization only affects the clustering region (which increases linearly). A parallel optimization of the Support Vector Machine implementation could change the overall result.

Table 4.9: Average time performance comparison (in seconds)

GRID SIZE	t_Jetson_OpenMP	t_Jetson_TEGRA
6x6	0.78	1.275
15x15	4.68	4.98
20x20	7.73	8.78
30x30	17.97	19.52
50x50	48.25	51.76

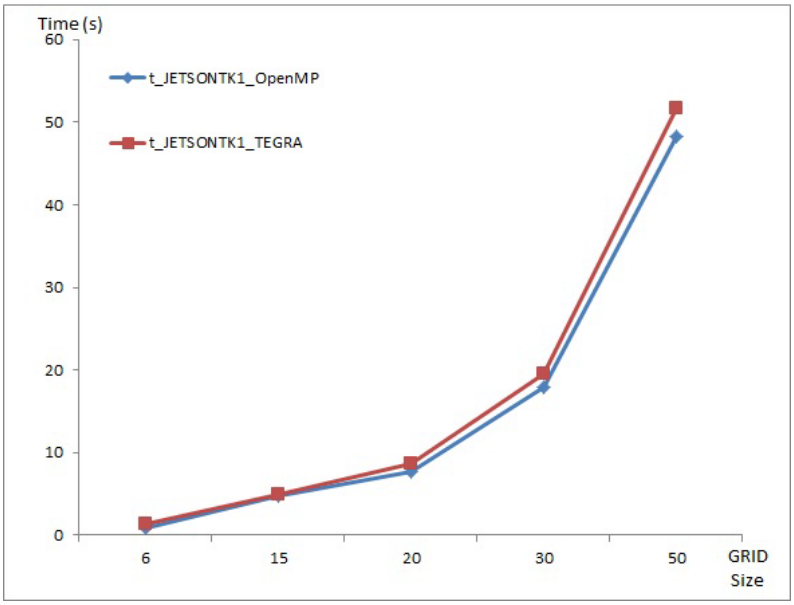

Figure 4.17: CUDA/OpenMPI bSOM Comparison

4.2.7 The System from an Overall View

The implementation of the algorithm on the embedded system (without training environment use) is presented in Figure 4.18. As end point for this research, the system is capable tu run on the proposed embedded platform with relatively short execution times.

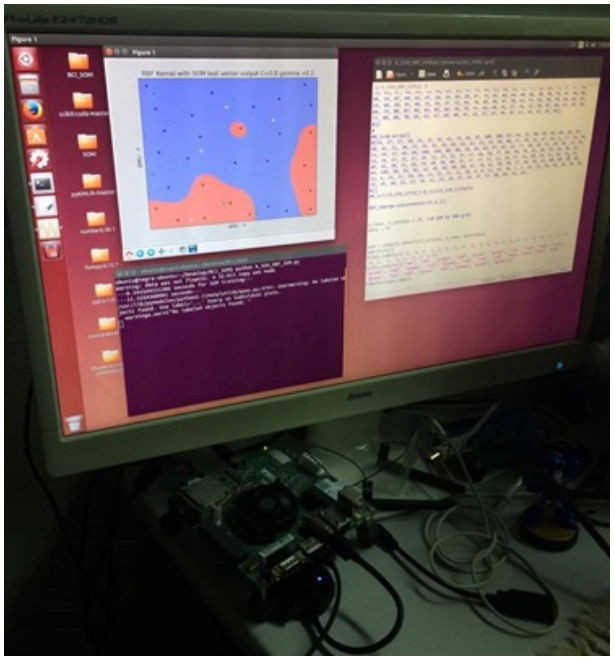

Figure 4.18: Overall view of method implementation on Jetson TK1 Board

Discussion

In this research, different results were obtained from the Supervised Learning and Unsupervised learning implementations with the aid of the b-SOM. From the literature[51], it is suggested that RBF kernel classifications are more likely to provide best results in Brain Computer Interface features, which in this case proved true if we see that the classification results from the last trained classifier provided good results. Although previous research results provided good recognition rates, it is noted that the clusters were not correctly defined, and the initial SOM implementations required a long amount of resources to provide good feature clustering.

However, it must be taken into account that the number of test samples is a little bit low. Nonetheless, the use of linear classifiers can be a good implementation for simple tools.

From previous classification attempts on previous research [74] [75] [76], the clustering by the b-SOM/RBF-SVM implementation provided the best classifier generation, which also certifies that the classification is robust. Also, it is noted that in some way, the empirical feature vector $\mathbf{X}$ contains properties that were suitable for non-linear clustering. In the case of an expected signal pattern, a synchronous signal can be characterized and recognized by static classifiers such as SVM.

For this research case, polynomial kernel training could also be considered, but when attempted to generate the classification, the execution times were greater than for the RBF-SVM kernels. Although it can be considered, the a better RBF-SVM classifier could be found without sacrificing time performance.

In terms of execution times, it is possible to have almost real time (human perception) recognition of the generated patterns after single trials. The response times are short for the small grid cases, considering the exponential time increase of the proposed method with the OpenMPI implementation of both without CUDA support.

On the other hand, the use of CUDA can not reduce the time performance due to the small ammount of data for the unsupervised clustering problem. However, the use of a parallel optimization of the Supervised Learning classification, which is increasing exponentially, could be reduced if a parallel implementation with CUDA would be tried in the near future.

Conclusions

In this research, the possibility of enhancing a simple electrode commercial grade tool through differente Machine Learning algorithms could be achieved, opening the door to a more efficient uses of low cost tools with embedded hardware.

With the development of different virtual environments and software tools, it was possible to test that specific patterns can be performed and their recognition can be achieved with the use of classifiers. The use of multiple electrode clinical grade tools was also tested and successfully achieved with the developed graphic tools.

From the implemented algorithms on this research, linear discriminant analysis based methodologies provided good classification results for empirically obtained feature vectors, especially on the DBSVM linear kernel implementations. However, their succeptibility to outliers and implementation on online classification reduces the chance to have a final implementation. Also, clusters were not well defined by only defining the principal components as features

The SOM implementation resulted initially in a very slow processing task for multidimensional data, making almost impossible the use of full dimensional vectors (180 dimensions) first due to the lack of parallel processing capabilities.

The use of the unsupervised clustering by batch SOM provides a good mapping in comparison to previous research. Moreover, the parallel-implementation allowed the use of full data in a very short time. The use of the current number of samples provided a good mapping and classification via the RBF-SVM. The parameter characterization allowed a 70% classification rate for the suggested features with an average execution time of around seven seconds on embedded hardware (OpenMP) and eight seconds with CUDA enabled execution. However, other mappings could also provide different classification results, but in terms of processing/efficiency, the 20x20 regular grid allows a good recognition at the cost of low computing needs with a conventional computer. Although time performance is good for the computer case, the OpenMPI option is still a little bit high for the selected Embedded system. The combination of unsupervised clustering and RBF-SVM Classification as suggested by literature provided the best grouping in comparison to previous research.

The overall system prototype presents the use of a single electrode BCI tool and the possibility to perform complex processing tasks with GPU capable hardware. Although the implementation of learning circuits with the aid of ASICs, it is still complex to bring these devices to a commercial environment due to their cost of development.

As further work, the use and enhancement of multiple electrode tools can be tested with this hardware by means of performing special feature extraction and its implementation on similar methods according to the algorithms used on this research, as stated on previous literature as well. Moreover, the overall execution time optimization with a parallel implementation of the SVM portion of the proposed method could bring more benefits for the current state of the implementation.

At the present time, the gap for multiple BCI research and development is still open, especially for a commercial link and consumer market development. The barrier of innovation in terms of hardware allows a broad range of applications to be developed and a handful of techniques to be brought from the scientific to the commercial field.

Acknowledgements

February 2017
Bruno Senzio-Savino Barzellato

First, I would like to thank my mother, for helping and supporting me throughout the time I spent during my research and Master's Course at Okinawa, Japan.

Second, I would like to thank the Ministry of Education, Culture, Sports, Science and Technology (MEXT) of Japan, for allowing me the obtention of the prestigious scholarship and the opportunity to come and visit Japan to perform graduate studies.

Third, I would like to express my sincere gratitude to my academic supervisor,

Prof. Dr. M. Reza Asharif, for his patience, advice and for allowing me to continue and complete my Master's Course in Okinawa, Japan.

Also, I would like to thank the members of the Thesis Committee for spending the time on reading this document and providing additional feedback to it.

Aditionally, I would like to thank the University of the Ryukyus and the staff members for allowing me to complete my degree at their university and providing me with funding for additional research activities. Also, to the group of researchers inside and outside of Japan I had the time to exchange ideas with and connect my research for useful development. In addition, I express my gratitude to Mr. Joshua Bloom from Trans Cranial Technologies (TCT) for granting me permission to use his materials for this document, Mr. Carlos E. Gutierrez for his technical support and Mr. Aditya Sharma for his daily life support.

Finally, I want to thank the Okinawan community in general, for allowing me to have an understanding of their culture and also for involving me in different social activities throughout my stay.

Summary

The use of Brain Computer Interface has become a trend in the last fifteen years, due to the use of commercial grade signal acquisition tools but also due to the evolution of modern embedded systems. However, there are still some limitations due to the precision and way of using these devices.

Currently, research is focusing on two interesting fields, such as artistic brain machine interfaces in order to provide an artistic but also a new perspective of the brain reactions on a controlled environment. Also, the use of brain waves as an important biometric to be used in smart home systems or automotive designs is being explored around the world, including Japanese companies and universities.

Some of the commercial grade BCI tools utilize non-invasive methods such as EEG, by means of performing mathematical transformations such as the Fast Fourier Transform and Power Spectral Densities to obtain specific features. In particular, commercial devices such as the Neurosky Mindwave can provide interesting values such as eSense Attention or Meditation that can provide a better way to utilize this tool.

Due to the innovation and commercial trends that this technology represents, currently there are different perspectives on studying how to use it, one of this methods includes the use of audiovisual feedback to end users by using virtual environments. Although there are some current software tools that allow to be used with trendy commercial grade devices, their portability to small embedded devices is still a little bit hard due to the system's hardware architectures.

In order to create a tool for EEG signal analysis, previous research utilices signal feature extraction and different classification methods with Machine Learning algorithms in both supervised and unsupervised learning approaches. For this research, different feature extraction methodologies were implemented, such as autoregressive models or dimension reduction by means of Principal Component Analysis. Also, the use of Scree plots of the signals obtained could allow to have a better understanding of the important number of dimensions to use for an unsupervised learning approach.

For this research, in order to collect data samples and use them for test classification and implementation, a virtual environment was built with the purpose of tracing a specific set of brain wave patterns under certain real world conditions. Before performing virtual environment tests, the test of specific brain wave pattern generation was analyzed.

From the virtual training environment experiment, 30 samples were obtained consisting of 180 second signal traces of eSense Attention and Meditation values provided by a Neurosky

Mindwave headset. The use of the obtained data used SVM and SOM classification algorithms, with the first one providing an 80% recognition rate and a 60% for the second one. The first results are expected according with previous literature on this matter, however, the use of the second method also allows the possibility to be implemented on a parallel approach allowing time reduction and self optimization due to the update features that current unsupervised learning software tools are providing.

By utilizing a special software library, the previous data was used to train a paralallel processing SOM implementation, providing interesting results as in reduction in processing time against the serial code implementations and also a considerable processing time reduction, which made it suitable for embedded system implementation.

The use of the unsupervised clustering by batch SOM provides a good mapping in comparison to previous research. Moreover, the parallel-implementation allowed the use of full data in a very short time. The use of the current number of samples provided a good mapping and classification via the RBF-SVM. The parameter characterization allowed a 70% classification rate for the suggested features. However, other mappings could also provide different classification results, but in terms of processing/efficiency, the 20x20 regular grid allows a good recognition at the cost of low computing needs with a conventional computer. Although time performance is good for the computer case, the OpenMPI option is still a little bit high for the selected Embedded system. The combination of unsupervised clustering and RBF-SVM Classification as suggested by literature provided the best grouping in comparison to previous research.

As an overall, the final system consisted of a GPU capable embedded processing system that also allowed the use of high resolution graphics, which makes it a good first step towards the design of the overall hardware device an software tools in order to customize this tool for smart home or automotive devices.

As further work, the hardware implementation and code optimization is suggested, as the use of particular features might be required for the single electrode perspective. Additionally, the use of multielectrode tools will be analyzed and other training algorithms might also be suitable in terms of an online classification perspective.

Bibliography

[1] Marvin Andujar, Chris S. Crawford, Anton Nijholt, France Jackson & Juan E. Gilbert (2015) Artistic brain-computer interfaces: the expression and stimulation of the user's affective state, *Brain-Computer Interfaces*, 2:2-3, 60-69, DOI: 10.1080/2326263X.2015.1104613

[2] K. Fladby. Brain Wave Based Authentication. *Gjovik University College*, 2008, pp.1,5-9.

[3] A. T. Campbell, T. Choudhury, S. Hu, H. Lu, M. K. Mukerjee, M. Rabbi, and R.D.S.Raizada. NeuroPhone: Brain-Mobile Phone Interface using a Wireless EEG Headset, *MobiHeld*, New Delhi, India p. 1-6, 2010.

[4] K. Akino, T. Mahajan, R. Marks, T.K. Tuzel, C.O. Wang, Y. Watanabe, S. Orlik. High-Accuracy User Identification Using EEG Biometrics, *Mitsubishi Electric Research Laboratories*, Massachusetts, 2016.

[5] I. Jayarathne, M. Cohen. BrainID: Development of an EEG-Based Biometric Authentication System, *Massachusetts*, 2016.

[6] J. Wolpaw and E. Winter Wolpaw, 2012. Brain-Computer Interfaces: Principles and Practice. *Oxford University Press*, p.1-10

[7] Brain Chip Technology: History. Brown University *http://biomed.brown.edu/Courses/BI108/BI108_2005_Groups/03/hist.htm* , May 2005.

[8] The Electric Brain. Massachusetts General Hospital// Dispatches from the frontiers of Medicine *http://protomag.com/articles/hans-berger-eeg*, May 2014.

[9] What Is BCI and How Did It Evolve? *http://neurosky.com/2015/06/what-is-bci-and-how-did-it-evolve/*, June 2015.

[10] Building Brain Machine Interfaces Neuroprosthetic Control with Electrocorticographic Signals *http://lifesciences.ieee.org/publications/newsletter/april-2012/96-building-brain-machine-interfaces-neuroprosthetic-control-with-electrocorticographic-signals*, IEEE LifeSciences, April 2012.

[11] What are brain waves? *http://www.brainworksneurotherapy.com/what-are-brainwaves*

[12] D. Veerendra, EEG Acquisition System on Mobile Platform. *http://scholarworks.wmich.edu/cgi/viewcontent.cgi?article=1128&context=masters_theses*, Western Michigan University, 2013, p.4-10

[13] 10/20 System Positioning. Trans Cranial Technologies *https://www.trans-cranial.com/local/manuals/10_20_pos_man_v1_0_pdf.pdf*, 2012

[14] N. Jatupaiboon, S. Pan-ngum and P. Israsena. Real-Time EEG-Based Happiness Detection System *The Scientific World Journal*, The Scientific World Journal Volume 2013, Figure 2.

[15] B.A. Olshausen *Aliasing*, PSC129-Sensory Processes, October 2000.

[16] H. Abut *DFT/FFT Transforms and Applications* San Diego State University, 2012.

[17] Towards a Real Time Human Brain-Computer Interface with Neurofeedback: Improving Differentiability by Blind Source Separation *http://www.brain.riken.jp/bsi-news/bsinews34/no34/research1e.html* December 2006.

[18] M. Morimoto, M. Imono, S. Tshuchiya and H. Watabe. Construction of the EEG Emotion Judgment System Using Concept Base of EEG Features *Int'l Conf. Artificial Intelligence*, 2015. p.2

[19] Gartner's 2016 Hype Cycle for Emerging Technologies Identifies Three Key Trends That Organizations Must Track to Gain Competitive Advantage. Press Release. *http://www.gartner.com/newsroom/id/3412017*, August 16th, 2016.

[20] T.E. Fink, 2012. The enhancement of Neurofeedback with a low cost and easy-to-use Neurosky EEG biofeedback training device: The MindReflectorProtocols. *International Society for Neurofeedback & Research*, pp. 3.

[21] R. Leeb, D. Friedman, G.R. Muller-Putz, R. Scherer, M. Slater and G. Pfurtscheller, 2007. Self-Paced (Asynchronous) BCI Control of a Wheelchair in Virtual Enviornments: A Case Study with a Tetraplegic. *Computational Intelligence and Neuroscience*, pp.1-6.

[22] D. C. Jangraw, A. Johri, M. Gribetz and P. Sajda, 2013. NEDE: An open-source scripting suite for developing experiments in 3D virtual environments. *Journal of Neuroscience Methods*, ELSEVIER, pp.2-5.F

[23] T.E. Fink, 2012. The enhancement of Neurofeedback with a low cost and easy-to-use Neurosky EEG biofeedback training device: The MindReflectorProtocols. *International Society for Neurofeedback & Research*, pp. 3.

[24] SharpBrains.Study: EEG-augmented brain training shows promise as first-line treatment for ADHD. *http://sharpbrains.com/blog/2016/03/09/study-eeg-augmented-brain-training-shows-promise-as-first-line-treatment-for-adhd/*, March 9th, 2016.

[25] Neurosky Inc, 2010. MindSet Communications Protocol. *Step-by-tep Guide to Parsing a Packet*, pp. 6-8.

[26] O. Cho and W. Lee, " BCI Sensor Based Environment Changing System for Immersion of 3D Game ", International Journal of Distributed Sensor Networks, Hindawi Publishing Corporation, pp. 3-4, May 2014.

[27] OpenBCI. *http://openbci.com*, 2016.

[28] Emotiv Systems. *https://www.emotiv.com*, 2016.

[29] Neurosky. *http://neurosky.com*, 2016.

[30] 3D-printed brain-sensing headset is open source. *http://www.smart2zero.com/news/3d-printed-brain-sensing-headset-open-source*, European Business Press SA, 2016.

[31] Neurosky Development. *https://sites.google.com/site/neuroskydevelopment/*, 2012.

[32] P Brunner, L Bianchi, C Guger, F Cincotti and G Schalk, 2011. Current trends in hardware and software for brain-computer interfaces (BCIs). *Journal of Neural Engineering*, 8(025001), pp.3-4.

[33] TGAM1 Chip *https://www.xenoinc.org/xi.wiki/index.php?title=TGAM1_Chip1)*, 2001.

[34] ThinkGear user Guide *http://developer.neurosky.com/docs/lib /exe/fetch.php?media=thinkgear_connector_user_guide.pdf*, 2012.

[35] G. Bauernfeind. The Future of Brain/Neural Computer Interaction: Horizon 2020. *https://www.researchgate.net/ publication/277844897_The_Future_of_BrainNeural_Computer_Interaction_Horizon_2020*, 2015

[36] EEGLAB *https://sccn.ucsd.edu/eeglab/*, 2016

[37] Inria Rennes. OpenViBE *http://openvibe.inria.fr/*, 2015

[38] neuromore Studio *http://www.neuromore.com/neuromore-studio/*, 2016

[39] L.R. Rabiner, R.W. Schafer "Digital Processing of Speech Signals",*Prentice-Hall Signal Processing Series*, p.116-120, 1978.

[40] R. Nau. Forecasting with moving averages,*Fuqua School of Business, Duke University*, p.1-3, 2014.

[41] C. Collomb. Linear Prediction and Levinson-Durbin Algorithm ,*http://ccolomb.free.fr*, p.1-7, 2009.

[42] Signal Processing Blockset-Levinson Durbin,*University of Montreal*, p.1-5, 2006.

[43] M.J.L. de Hoon, T.H.J.J. van der Hagen, H. Schoonewelle and H. van Dam. Why Yule-Walker should not be used for Autoregressive Modelling,*Elsevier*, p.2-3, 1996.

[44] Priestley,*Spectral analysis and time series*, 1994

[45] K. Baker. Singular Value Decomposition Tutorial,*Ohio State University*, p.15, 2005-2013.

[46] What is a scree plot?,*http://support.minitab.com/en-us/minitab/17/topic-library/modeling-statistics/multivariate /principal-components-and-factor-analysis/what-is-a-scree-plot/*, Minitab, 2016.

[47] Scree plot analysis,*http://www.janda.org/workshop/factor%20analysis /SPSS%20run/SPSS08.htm*, Figure.

[48] K. Mordani. Scree Plots: Interpretation and Shortcoming,*http://ba-finance-2013.blogspot.jp/2012/09/scree-plots-interpretation-and.html*, September 2012.

[49] L.I. Smith. A tutorial on Principal Component Analysis*International Institute of Information Technology*, February 2002.

[50] J. Shlens. A tutorial on Principal Component Analysis: Derivation, Discussion and Singular Value Decomposition*University of California San Diego*, March 2003.

[51] F. Lotte, M. Congedo, A. Lecuyer, F. Lamarche and B. Arnaldi. A review of classification algorithms for EEG-based brain-computer interfaces *Journal of Neural Engineering*, IOP Publishing, 2004, 4, pp.24 ¡inria-00134950¿

[52] K. Fukunaga. Statistical Pattern Recognition, second edition *Academic Press, INC*, 1990.

[53] D. Garret, D.A. Peterson, C.W. Anderson and H.M. Thaut. Comparison of linear, nonlinear and feature selection methods for eeg signal classification *IEEE Transactions on Neural System and Rehabilitation Engineering*, 1:141-144, 2003

[54] B. Blankerz, G.Curio and K.R. Muller. Classifying single eeg: Towards brain computer interfacing. *Advances in Neural Informaction Processing Systems*, 14:157-164, 2002.

[55] N. Matloff. Programming on Parallel Machines *University of California, Davis*, 2015.

[56] T. Matson, L. Meadows. A "Hands-on" Introduction to OpenMP *OpenMP.org*

[57] Danny Oude Bos, 2007. EEG-based Emotion Recognition: The Influence of Visual and Auditory Stimuli. *University of Twente*, pp.1-11.

[58] J. Light, Kalaiselvi T., X. Li and A.R. Malali, 2013. Fall Pattern Classification from Brain Signals using Machine Learning Models. *Journal of Selected Areas in Telecommunications*, pp.1-3, 5.

[59] Introduction to Population Density. National Geographic *http://nationalgeographic.org/activity/ introduction-population-density/*, 1196-2016.

[60] Z. Nazari, D. Kang, and H. Endo, 2014. Density Based Support Vector Machines. *Proceedings of the 29th International Technical Conference on Circuits/Systems*, ITC-CSCC 2014.

[61] T. Kohonen, 1997. Self-Organizing Maps. *Springer*, pp.106-115.

[62] I. Jahan, M. Prilepok, V. Snasel and M. Penhaker. Similarity Analysis of EEG Data Based on Self Organizing Map Neural Network *Advances in Electrical and Electronic Engineering*, Volume 12-5:550-553, 2014

[63] P. Wittek, S.C. Gao, I.S. Lim and L. Zhao. Somoclu: An Efficient Parallel Library for Self-Organizing Maps *arXiv:1305.1422.*, 2015

[64] J. Sanders and E. Kandrot. CUDA by Example: An Introduction to General Purpos GPU-Programming *Addison-Wesley*, 2011

[65] Introduction to GPU and CUDA Programming . Cornell Virtual Workshop *https://cvw.cac.cornell.edu/gpu/memory_arch?AspxAutoDetectCookieSupport=1*, 2016

[66] A. Naruse. OPENACC による GPU コンピューティング. *NVIDIA Developer Technologies*, 2013

[67] The Parallela Board, *https://www.parallella.org/*, 2014

[68] Arduino Galileo, *https://www.arduino.cc/en/ArduinoCertified/IntelGalileo*, 2016

[69] Raspberry Pi 3, *https://www.raspberrypi.org*, 2016

[70] Jetson TK1, *http://elinux.org/Jetson_TK1*, 2016

[71] NVIDIA Computing Capability, *docs.nvidia.com/*, 2016

[72] Blender (Software) *https://en.wikipedia.org/wiki/Blender_(software)*, 2016

[73] Unity (game engine) *https://en.wikipedia.org/wiki/Unity_(game_engine)*, 2016

[74] B. Senzio-Savino, K. Yamada, 2014. Test and Development of a Mind Wave Signal Pattern Password Application. *Proceedings of the 15th System Integration Conference*,SICE 2014, pp.1182-1184.

[75] B. Senzio-Savino, M.R. Alsharif, C.E. Gutierrez, K. Yamashita, 2015.Synchronous Emotion Pattern Recognition with a Virtual Training Environment. *International Conference on Artificial Intelligence*, pp. 650-654.

[76] B. Senzio-Savino, M.R. Alsharif, C.E. Gutierrez, K. Yamashita and J. Noble. Path Detection in Virtual Environment for Synchronous EEG by Density Based Support Vector Machine. *Journal of Information and Communications Engineering*, 1(1), 2015

[77] H. Takahashi, M.R. Alsharif, B. Senzio-Savnio and S.K. Setarehdan. Brain Wave Attetion/Meditation Signal Feature Extraction and Classification with MA and AR based on PCA and SVM. *IEEJ Proceecings*, December, 2016

[78] Principal Component Analysis in Python. *http://scikit-learn.org/stable/modules/generated/ sklearn.decomposition.PCA.html*, 2010-2014.

[79] Singular Value Decomposition with Python. *http://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.svd.html*, 2008-2014.

[80] LinearSVC kernel. *http://scikitlearn.org/stable/modules/generated/sklearn.svm.LinearSVC.html*, 2010-2014.

[81] B. Senzio-Savino, M.R. Alsharif, C.E. Gutierrez, C. Penaloza, K. Yamashita, F. Alsharif, M. Khosravy and M. Shamsi, 2016. Brain Wave Pattern Classification from Virtual Training Environment by Self-Organizing Maps. *Proceedings of the 31st International Technical Conference on Circuits/Systems, Computers and Communications* , pp. 779-782.

[82] Self-Organizing Maps in Python. *http://www.aijunkie. com/ann/som/som1.html*, 2006-2015.

[83] Jetson Hacks - WiFi Kernel and CUDA libraries *http://jetsonhacks.com/2014/10/12/installing-wireless-intel-7260-adapter-nvidia-jetson-tk1/*, 2015

5 Appendix

Apendix A: Configuring the NVIDIA Jetson TK1 Board

One of the main problems encountered at the initial setup of the NVIDIA Jetson TK1 is the cross-compiling feature that is required to flash the OS and install the necessary CUDA repositories in the embedded board from a host machine; the host requires to have an AMD64 architecture in order to perform the ARMv7 libraries compilation.

If the cross-compiling feature is to be skipped, it is then necessary to install the entire CUDA environment and perform native device driver installation[83], which will require a proper connection to the Internet. However, the support for WiFi devices for the unmodified version of the board is not documented, therefore the need of a new kernel version and implementation was required.

Once the libraries and hardware are optimized, we can run some example configuration files and simulations to test some of the SoC's capabilities (Figure A.1).

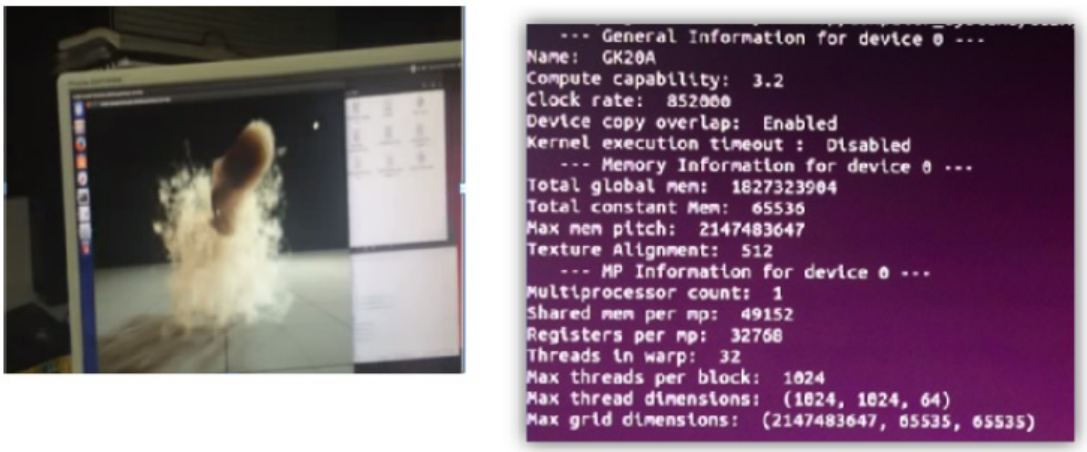

Figure A.1 Execution of a smoke simulation with CUDA and GPU information program

Apendix B: Single Parallel Feature Extraction with CUDA and the Jetson TK1

Serial Feature Extraction

The following code was used to obtain the feature vector with the use of CUDA program only for the Cortex-A15 processor. The program uses simple memory allocation for the matrix operations that are used on by the main function.

nvcc signal_power_cpu.cu -o signal_power_cpu

```
/*
 * University of the Ryukyus
 * Computer Systems 01/30/2016
 *
 * Signal vector Feature Extraction Comparison V0.1
 *
 * Author: Bruno Senzio-Savino Barzellato (k158579)
 *
 */

/* This is the serial code implementation
 * we have two vectors (Attention and Meditation)
 * with 180 elements obtained from BCI Training
 *
 * We have 30 training samples and 5 test samples
 * The purpose is to obtain a feature vector X
 * for further classification with the use of
 * parallel computing.
 *
 * In order to increase data samples for simulation purposes
 * the previous vectors are repeated several times
 *
 * From previous classification results, the effective
 * feature vector for SVM classification was
 *
 * X=(((A*A)*(A-B)/(A+B+alpha))*alpha ; alpha=0.001 ;
 *
*/
```

```
// Libraries
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Use of predefined databases
#include "attention70.h"
#include "meditation70.h"

//Setup
#define N   70
#define training_set_size 60
#define test_set_size 10
#define alpha 0.001


double **featue_matrix(int rows, int cols);
void print_matrix(int rows, int cols, double **mat);
void free_matrix(int rows, double **mat);

//Feature matrix declaration
double **feature_matrix(int rows, int cols, int a[][N], int b[][N]){

    double **mat = (double **) malloc(sizeof(double *)*rows);
    int i=0,j=0;
    for(i=0; i<rows; i++)
    /* Allocate array, store pointer */
        mat[i] = (double *) malloc(sizeof(double)*cols);

       for(i=0; i<rows; i++){
           for(j=0; j<cols; j++){
               mat[i][j]=((a[i][j]*a[i][j])*(a[i][j]-b[i][j]))
                                   /(a[i][j]+b[i][j]+alpha);
           }
       }
     return mat;
}

//Print Matrix function
```

```
void print_matrix(int rows, int cols, double **mat){
    int i=0,j=0;
  for(i=0; i<rows; i++){     /* Iterate of each row */
        printf( "Feature [%d] \n",i+1); /* Print each row element */
        for(j=0; j<cols; j++){  /* In each row,
                                   go over each col element  */
            printf( "x [%d][%d] %lf\n",i+1,j+1,  mat[i][j]);
            /* Print each row element */
        }

    }
}

//Free memory
void free_matrix(int rows, double **mat){
    int i=0;
    for(i=0;i<rows;i++)
        free(mat[i]);
    free(mat);
}

int main( void ) {
    clock_t begin, end;  //Stopwatch
    double time_spent;
    double **matrix;
    int rows=training_set_size+test_set_size;
    int cols=N;

    begin = clock();
    matrix=feature_matrix(rows,cols, a, b);
    print_matrix(rows,cols,matrix);
    free_matrix(rows,matrix);

    end = clock();
    time_spent=(double)(end-begin)/CLOCKS_PER_SEC;
    printf("Time spent= %f sec\n",time_spent);
    return 0;
}
```

Parallel Feature Extraction

```
/*
 * University of the Ryukyus
 * Computer Systems 01/30/2016
 *
 *  Parallel Signal vector Feature Extraction Comparison V0.1
 *
 * Author: Bruno Senzio-Savino Barzellato (k158579)
 *
 */

/* This is the CUDA code implementation
 * we have two vectors (Attention and Meditation)
 * with 180 elements obtained from BCI Training
 *
 * We have 30 training samples and 5 test samples
 * The purpose is to obtain a feature vector X
 * for further classification with the use of
 * parallel computing.
 *
 * In order to increase data samples for simulation purposes
 * the previous vectors are repeated several times
 *
 * From previous classification results, the effective
 * feature vector for SVM classification was
 *
 * X=(((A*A)*(A-B)/(A+B+alpha))*alpha ; alpha=0.001 ;
 *
*/

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

//This library handles Errors used below
// by printing out the error
#include <error_handle.h>
```

```
//Simplified datasets
#include "attention_70_single.h"
#include "meditation_70_single.h"

#define size   70
#define training_set_size 60
#define test_set_size 10
#define N (training_set_size+test_set_size)*size
#define alpha 0.001

//Global Feature Extraction Function

__global__ void feature( int *a, int *b, double *f ) {
    int tid = blockIdx.x;    // this thread handles the
                  data at its thread id
    //int tid = threadIdx+ blockIdx.x*blockDim.x; //use
                       // if threads are implemented
    if (tid < N)
        f[tid] = ((a[tid]*a[tid])*
                  (a[tid]-b[tid]))/
                  (a[tid]+b[tid]+alpha);
    tid+=blockDim.x; //*gridDim.x; is used if threading
}

int main( void ) {
    double f[N];
    int *dev_a, *dev_b;
    double *dev_f;
    clock_t begin, end;
    double time_spent;

    int rows=training_set_size+test_set_size;
    int cols=size;

    begin=clock();
    // allocate the memory on the GPU
    // For speed up, cudaMalloc is replaced to cudaMallocHost
    HANDLE_ERROR( cudaMallocHost( (void**)&dev_a, N * sizeof(int)));
    HANDLE_ERROR( cudaMallocHost( (void**)&dev_b, N * sizeof(int)));
```

```
    HANDLE_ERROR( cudaMallocHost( (void**)&dev_f,
    N * sizeof(double)));

    // copy the arrays 'a' and 'b' to the GPU
    HANDLE_ERROR( cudaMemcpy( dev_a, a, N * sizeof(int),
                              cudaMemcpyHostToDevice ) );
    HANDLE_ERROR( cudaMemcpy( dev_b, b, N * sizeof(int),
                              cudaMemcpyHostToDevice ) );

    feature<<<N,1>>>( dev_a, dev_b, dev_f );

    // copy the array 'c' back from the GPU to the CPU
    HANDLE_ERROR( cudaMemcpy( f, dev_f, N * sizeof(double),
                              cudaMemcpyDeviceToHost ) );


    // display the results
    /*for(int k=0; k<rows; k++){
            for(int i=0; i<rows; i++){
                printf( "Feature [%d] \n",i+1);
                for(int j=0; j<cols; j++){
                    printf( "x [%d][%d] %lf\n",i+1,j+1,  f[k*N]);
                }

            }
    }*/

    for(int j=0; j<N; j++)
        printf( "x [%d] %lf\n",j,f[j]);

    // free the memory allocated on the GPU
    HANDLE_ERROR( cudaFree( dev_a ) );
    HANDLE_ERROR( cudaFree( dev_b ) );
    HANDLE_ERROR( cudaFree( dev_f ) );
    end=clock();
    time_spent=(double)(end-begin)/CLOCKS_PER_SEC;
    printf("Time spent= %f sec\n",time_spent);
    return 0;
}
```

Apendix C: Mind Plotter Implementation

```
# BRAIN MODULE TESTER V 0.7
# UNIVERSITY OF THE RYUKYUS 2014
#    FACULTY OF ENGINEERING - IT DIVISION
#    YAMADA-KEN
#    AUTHOR: BRUNO SENZIO-SAVINO BARZELLATO
#    DATE: 25/08/2014

#Comments:
# Still missing performance-wise programming and custom
peak generation and comparison

#Import libraries

import numpy as np
import matplotlib.pyplot as plt
import serial
import time
import winsound

#Predefined Pattern declaration
selected_pattern_1=[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
selected_expected=[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
selected_peaks=0
selected_ramps=0
selected_ztimes=[0,0,0,0,0,0]
selected_threshold=80
expected_threshold=[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
graph_threshold=[1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,
1,1,1,1,1,1,1,1,1,1,1,1,1,1,1]

R=[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]

#Predefined thresholds
```

```
top=[100,100,100,100,100,100,100,100,100,
100,100,100,100,100,100,100,100,100,100,100,
100,100,100,100,100,100,100,100,100,100,100,100,100,100];

#Pattern A

pattern_a=[10,10,10,10,10,10,10,10,10,10,50,
50,50,50,50,90,90,90,90,90,90,90,50,50,50,
50,50,10,10,10,10,10,10];
expected_a=[0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0];
peaks_a=1
ramps_a=0
zeroes_a=[16,22,0,0,0,0]

#Pattern B

pattern_b=[10,50,50,50,50,50,90,90,90,90,90,90,90,
90,50,50,50,50,50,50,50,90,90,90,90,90,90,90,
50,50,50,50,10];
expected_b=[0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0];
peaks_b=2
ramps_b=0
zeroes_b=[7,14,22,28,0,0]

#Pattern C

pattern_c=[10,50,50,50,90,90,90,90,90,50,50,50,50,
90,90,90,90,90,90,50,50,50,90,90,90,90,90,
50,50,50,50,50,10];
expected_c=[0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0,
0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0];
peaks_c=3
ramps_c=0
zeroes_c=[5,9,15,19,23,27]
```

```
#Pattern D

pattern_d=[10,10,10,10,10,10,10,10,10,10,50,50,50,50,
90,90,90,90,90,90,90,90,50,50,50,50,50,
10,10,10,10,10,10];
expected_d=[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0];
peaks_d=1
ramps_d=0
zeroes_d=[15,19,0,0,0,0]

#Pattern E

pattern_e=[10,50,50,50,50,50,90,90,90,90,90,90,90,
90,60,60,50,50,50,60,60,90,90,90,90,90,90,
90,50,50,50,50,10];
expected_e=[0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0];
peaks_e=2
ramps_e=0
zeroes_e=[8,14,22,28,0,0]

#Pattern F

pattern_f=[10,10,20,20,30,30,40,40,50,50,60,60,70,70,
80,80,90,90,90,90,90,90,90,90,90,90,90,
90,90,90,90,90,90];
expected_f=[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1];
peaks_f=0
ramps_f=1
zeroes_f=[17,0,0,0,0,0]

#Combined pattern examples
```

```
#Pattern G (A for meditation//B for attention)

pattern_g_a=[10,50,50,50,50,50,90,90,90,90,90,90,50,
50,50,50,50,10,10,10,10,10,10,10,10,10,10,
10,10,10,10,10];
expected_g_a=[0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0];
pattern_g_b=[10,10,20,20,30,30,40,40,50,50,60,60,70,70,
79,79,90,90,90,90,90,90,90,90,90,90,90,
90,90,90,90,90,90];
expected_g_b=[0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1];

#Pattern H

pattern_h_a=[10,50,50,50,50,50,90,90,90,90,90,90,90,
90,50,50,50,50,50,50,50,90,90,90,90,90,
90,90,50,50,50,50,10];
expected_h_a=[0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1
, 1, 1, 0, 0, 0, 0, 0];
pattern_h_b=[10,50,50,50,50,50,90,90,90,90,90,90,
90,90,50,50,50,50,50,50,50,90,90,90,90,
90,90,90,50,50,50,50,10];
expected_h_b=[0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 1, 1, 1, 0, 0, 0, 0, 0];

#Pattern I

pattern_i_a=[10,50,50,50,50,50,90,90,90,90,90,90,
90,90,50,50,50,50,50,50,50,90,90,90,90,
90,90,90,50,50,50,50,10];
expected_i_a=[0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0];
pattern_i_b=[10,50,50,50,50,50,90,90,90,90,90,90,90,
```

```
90,50,50,50,50,50,50,50,90,90,90,90,90,90,
90,50,50,50,50,10];
expected_i_a=[0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
1, 1, 1, 1, 0, 0, 0, 0, 0];


#Creating the pattern reading matrix
trace=range(33)
timers=range(33)

#Initializing the trace pattern
for i in range(33):
        trace[i]=0

#Option selection

option = raw_input(
"Please choose an option: \n(1)Attention pattern
\n(2)Meditation pattern \n(3)Mixed pattern \n")
clear=0

if option=='1':
    mode = raw_input("What kind of pattern?(a~c)\n")
    if mode=='a':
        selected_pattern = pattern_a
        selected_peaks=peaks_a
        selected_ramps=ramps_a
        selected_ztimes=zeroes_a
        selected_expected=expected_a
    elif mode=='b':
        selected_pattern = pattern_b
        selected_peaks=peaks_b
        selected_ramps=ramps_b
        selected_ztimes=zeroes_b
        selected_expected=expected_b
    elif mode=='c':
        selected_pattern = pattern_c
        selected_peaks=peaks_c
```

```
            selected_ramps=ramps_c
            selected_ztimes=zeroes_c
            selected_expected=expected_c
        else:
            clear=1
            print("That is not a valid option, please
            enter a valid option\n");
    elif option=='2':
        mode = raw_input("What kind of pattern?(d~f)\n")
        if mode=='d':
            selected_pattern = pattern_d
            selected_peaks=peaks_d
            selected_ramps=ramps_d
            selected_ztimes=zeroes_d
            selected_expected=expected_d
        elif mode=='e':
            selected_pattern = pattern_e
            selected_peaks=peaks_e
            selected_ramps=ramps_e
            selected_ztimes=zeroes_e
            selected_expected=expected_e
        elif mode=='f':
            selected_pattern = pattern_f
            selected_peaks=peaks_f
            selected_ramps=ramps_f
            selected_ztimes=zeroes_f
            selected_expected=expected_f
        else:
            clear=1
            print("That is not a valid option,
            please enter a valid option\n");
    elif option=='3':
        mode = raw_input("What kind of pattern?(g~i)\n")
        if mode=='g':
            selected_pattern = pattern_g_a
            selected_pattern_1 = pattern_g_b
        elif mode=='h':
            selected_pattern = pattern_h_a
            selected_pattern_1 = pattern_h_b
```

```
    elif mode=='i':
        selected_pattern = pattern_i_a
        selected_pattern_1 = pattern_h_b
    else:
        clear=1
        print("That is not a valid option,
        please enter a valid option\n");
else:
        clear=1
        print("That is not a valid option,
        please enter a valid option\n");

if clear==0:
        selected_threshold=int(raw_input
        ("Please enter a threshold value(1~99)"))
        for x in range(33):
                graph_threshold[x]=selected_threshold*
                graph_threshold[x]

        print("Please take a look at the
        plot and close when ready...\n");
        plt.plot(timers,selected_pattern,'b',
        selected_pattern_1,'r',graph_threshold,'b--',top,'b--')
        plt.show()
        correct = raw_input("Is this correct?(y/n)")
        if correct=='y':

                #trace=[0, 0, 0, 0, 0, 0, 0, 1, 1,
                1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
                0, 0, 0, 1,
                0, 1, 0, 0, 0, 0, 0, 0];
                trace=      [12,20,20,45,55,34,26,70,87,54,20,18,25,
                30,40,50,72,86,15,79,45,35,70,81,80,76,15,23,
                35,57,60,70,56];
                #Wait for key to be pressed in order to continue
                raw_input("PUT HELMET AND
                PRESS ENTER WHEN READY...")

                #Define serial port
```

```
ser = serial.Serial('COM4',9600)

for i in range(33):
        trace[i]=int(ser.readline())
        print trace[i]

plt.plot(timers,selected_pattern,'b',
trace,'g',graph_threshold,
'b--',top,'b--')
plt.show()

for x in range(33):
    expected_threshold[x]=
    selected_threshold*
    selected_expected[x]

for x in range(33):
    R[x]=trace[x]-expected_threshold[x]
    if R[x]==trace[x] and R[x]>=80:
        R[x]=1
    if R[x]==trace[x] or R[x]<0:
        R[x]=0
    if R[x]>0:
        R[x]=1

trace=R
print(trace)
print(R)

number_of_zeroes=2*selected_peaks
wrong=0
looking_for_peaks=-1
for i in range(number_of_zeroes+1):
    correct=1
    peak=0
    if i==0:

    for a in range(selected_ztimes[0]):
```

```
                if trace[a]!=0:
                    correct=0
                        wrong=1
                        break
                if i>0 and i<number_of_zeroes:
                     a=selected_ztimes[i-1]
                     looking_for_peaks=(-1)**i
                     while a<selected_ztimes[i]:
                          if looking_for_peaks==1:
                               if trace[a]!=0:
                                    correct=0
                          elif looking_for_peaks==-1:
                               if trace[a]==1:
                                    peak=1
                          a+=1
                     if peak!=1 and looking_for_peaks==-1:
                          correct=0
                          wrong=1
                          break
                     if correct==0:
                          wrong=1
                          break
                if i==number_of_zeroes:
                     a=selected_ztimes[i-1]
                     while a<33:
                          if trace[a]!=0:
                               correct=0
                               wrong=1
                               break
                          a+=1
            print("Wrong:")
            print(wrong)
            print(trace)
    elif correct=='n':
            print("Please
            restart program and make sure
            to select the right option\n");
```

Apendix D: 3D Virtual Environment Arduino Sketches and File Classification Script

Arduino Micro Code (.ino)

```
const int buttonPin = 4;          // input pin for pushbutton
int previousButtonState = HIGH;   // for checking the
state of a pushButton
int counter = 0;                  // button push counter

void setup() {
  // make the pushButton pin an input:
  pinMode(buttonPin, INPUT);
  // initialize control over the keyboard:
  Keyboard.begin();
}

void loop() {
  // read the pushbutton:
  int buttonState = digitalRead(buttonPin);
  // if the button state has changed,
  if ((buttonState != previousButtonState)
    // and it's currently pressed:
  && (buttonState == HIGH)) {
    // increment the button counter
    counter++;
    // type out a message
    //Keyboard.print("You pressed the button ");
    Keyboard.print(counter);
    Keyboard.println("x");
  }
  // save the current button state for comparison next time:
  previousButtonState = buttonState;
}
```

Arduino UNO Code (.ino)

```
#include <Brain.h>

// Set up the brain parser, pass it the
hardware serial object you want to
listen on.
```

```
Brain brain(Serial);

void setup() {
    // Start the hardware serial.
    pinMode(13,OUTPUT);
    Serial.begin(9600);
}

void loop() {
    // Expect packets about once per second.
    // The .readCSV() function returns a string (well, char*)
    listing the most recent brain data,
    in the following format:
    // "signal strength, attention, meditation,
    delta, theta, low alpha, high alpha, low beta,
    high beta, low gamma, high gamma"
    if (brain.update()) {
        Serial.println(brain.readErrors());
        Serial.println(brain.readAttention());
        if(brain.readAttention()>=80) digitalWrite(13, HIGH);
        if(brain.readAttention()<80)
        digitalWrite(13, LOW);
    }
}
```

File Classifier Script (Python)

```
# BRAIN  DATA CLASSIFIER V0.7
# UNIVERSITY OF THE RYUKYUS 2015
#   FACULTY OF ENGINEERING – IT DIVISION
#   ASHARIF–KEN
#   AUTHOR: BRUNO SENZIO–SAVINO BARZELLATO
#   DATE: 14/03/2015

#Import libraries
import numpy as np
import matplotlib.pyplot as plt
import serial.tools.list_ports
import serial
import csv
```

```
import time
import io

#Defining data file descriptors for data sorting
fd = ['attention.csv','meditation.csv',
'delta.csv','theta.csv',
'low_alpha.csv','high_alpha.csv',
'low_beta.csv','high_beta.csv','low_gamma.csv',
'high_gamma.csv']
stream_file = 'stream.csv'
stream_vector_length = 180

#Defining file descriptors for learning data vectors
f_l = ['learning_attention.csv',
'learning_meditation.csv']

#Defining plot color vector
color_vector = ['b','g','r','c','m','y','k','w']

welcome_message ="""BRAIN  DATA CLASSIFIER V0.2 \n
   UNIVERSITY OF THE RYUKYUS 2015 \n
   FACULTY OF ENGINEERING - IT DIVISION \n
   ASHARIF-KEN \n
   AUTHOR: BRUNO SENZIO-SAVINO BARZELLATO \n
   DATE: 14/03/2015\n\n
   """

print welcome_message

#Displaying available COM PORTS and selecting the right device
#com_ports = list(serial.tools.list_ports.comports())

#while len(com_ports) == 0:
#        com_ports = list(serial.tools.list_ports.comports())
#        print
"NO COM PORTS AVAILABLE"
#        raw_input()

#print "Please select from the list the
```

```
device you want to utilize"

#for i in range(len(com_ports)):
#        print str(i+1) + ". " + str(com_ports[i][1])

port_number = 4  #raw_input()

#Define serial port as Arduino virtual
SERCOM from FTDI
ser_com = serial.Serial(3)
print "Found connection at PORT " + ser_com.name
command = ser_com.readline()


#Obtaining stream and dividing
print "Press the red button when ready"
while command[0] != 'p':
        command = ser_com.readline()

while command[0] == 'p':
        command = ser_com.readline()


#The dataset is divided in 11 parts:
poorSignalLevel, attention, meditation,
delta, theta,lowAlpha, highAlpha,
lowBeta, highBeta,lowGamma, highGamma
#If raw EEG is in the package, the
general values are transmitted at fs = 512 Hz
#The stream comes from FFT to a preconditioned signal

#Writing the stream into a file
with io.open(stream_file,'w') as stream:
        #while command[0] != 'p':
        #Streaming total game data
        for x in range(stream_vector_length):
                print str(x)+ ":" + command
                command = ser_com.readline()
                if command[0] != 'p':
```

```
                stream.writelines(unicode(command))
stream.close()
ser_com.close()

#writing the stream data into the
corresponding csv files
files = [open(fd[i],'w') for i in range(len(fd))]
with open(stream_file,'r') as input:
        for line in input:
                data_package = line.split(',')
                for j in range(1,10):
                        files[j-1].write
                        (unicode(data_package[j])+'\n')
#closing the files
for f in files:
        f.close()

#Declaring individual plot vectors
for attention and meditation
attention = []
meditation = []

#Defining learning structure for file manipulation
learning_vectors = [attention, meditation]

#Creating plotting vectors
learning_vector_files = [open(fd[i],'rb') for i in range(2)]

for x in range(len(learning_vector_files)):
        c = csv.reader(learning_vector_files[x])
        for row in c:
                learning_vectors[x].append(row)

for f in learning_vector_files:
        f.close()

#Creating resulting vectors from stream
X = []
```

```
Y = []

R_vectors=[X,Y]

for a in range(len(R_vectors)):
        for b in range(len(learning_vectors[a])):
                R_vectors[a].append(
                int(learning_vectors[a][b][0]))

#Writing resulting vectors on the respective files
l_files = [open(f_l[i], 'w') for i in range(len(f_l))]

for y in range(len(l_files)):
          l_files[y].write(unicode(R_vectors[y]))

for f in l_files:
        f.close()

print learning_vectors[0]
print learning_vectors[1]

num = len(attention)

#Plotting learning vectors
plt.title('TEST RESULTS')
plt.text(110, 10, 'Blue: Attention')
plt.text(110, 5, 'Green: Meditation')
plt.grid(True)
plt.xlabel('time[s]')
plt.ylabel('Power %')
plt.xlim([0,stream_vector_length])
plt.ylim([0,100])
for p in range(len(learning_vector_files)):
        plt.plot(range(num),learning_vectors[p],
        color_vector[p])

plt.show()
```

Apendix E: Scree plot and SOM Implementation scripts

Scree Plots (Python)

```
## Scree plot by SVD of RAW and feature data V 1.0
## University of the Ryukyus – DSP Lab
## January 20th, 2016
##
## Bruno Senzio-Savino, M.R. Asharif

# This Python script obtains the eigenvalues of
# three matrices, A, B, and X which are referred to
# Attention, Meditation and Feature data from BCI
# training, respectively.
#
# Each matrix is 30x180 from origin, therefore,
# it is needed to obtain a square matrix from SVD
# that is 180x180 with the use of Python libraries.
# Once each matrix, A_s, B_s and X_s is constructed,
# their eigenvalues are obtained and plotted accordingly.

# These plots helped us decide the maximum number of components
# to be used for PCA reduction for whitening and training for
# a Self-Organized-Map classified

# Importing the necessary libraries for array creation and SVD
# and plot

import numpy as np
import matplotlib.pyplot as plt

# Declaring the raw data as arrays for component multiplication

#Attention data matrix [30x180]
#Local arrange for appendix viewers to
understand signal origins
(Check A and B on original script)

A, B

#Obtention of Feature matrix X
```

```
#scaling factor
alpha=0.001
X=(((A*A)*(A-B))/(A+B))*alpha

#Note: Division by zero is assumed as zero, as defined previously

#We use SVD for matrix a, where square matrix S_a = P_a*D_a*Q_a'
P_a, D_a, Q_a = np.linalg.svd(A, full_matrices=False)

#Obtaining the Square matrix of A
S_a = np.dot(np.dot(P_a, np.diag(D_a)), Q_a)
S_a = np.matrix(S_a)
L_a=S_a.T*S_a

#We use SVD for matrix B, where square matrix S_b = P_b*D_b*Q_b'
P_b, D_b, Q_b = np.linalg.svd(B, full_matrices=False)

#Obtaining the Square matrix of B
S_b = np.dot(np.dot(P_b, np.diag(D_b)), Q_b)
S_b = np.matrix(S_b)
L_b=S_b.T*S_b


#We use SVD for matrix X, where square matrix S_x = P_x*D_x*Q_x'
P_x, D_x, Q_x = np.linalg.svd(X, full_matrices=False)

#Obtaining the Square matrix of X
S_x = np.dot(np.dot(P_x, np.diag(D_x)), Q_x)
S_x = np.matrix(S_x)
L_x=S_x.T*S_x


#Obtaining the eigenvalue vectors for each square matrix
eigenvalue_vector_a=np.linalg.eigvals(L_a)
eigenvalue_vector_b=np.linalg.eigvals(L_b)
eigenvalue_vector_x=np.linalg.eigvals(L_x)

plt.figure(1, figsize = (10, 20))
```

```
plt.grid(True)
plt.subplot(311, axisbg = 'w')

#Plotting the eigenvalue curve for A

plt.title('Eigenvalue plot for A')
plt.ylabel('Value')
plt.xlim([0, np.size(eigenvalue_vector_a)])

for k in range(1,np.size(eigenvalue_vector_a)):
        plt.scatter(k,eigenvalue_vector_a[k])

plt.subplot(312, axisbg = 'w')

#Plotting the eigenvalue curve for B

plt.title('Eigenvalue plot for B')
plt.ylabel('Value')
plt.xlim([0, np.size(eigenvalue_vector_b)])

for k in range(1,np.size(eigenvalue_vector_b)):
        plt.scatter(k,eigenvalue_vector_b[k])

plt.subplot(313, axisbg = 'w')

#Plotting the eigenvalue curve for X

plt.title('Eigenvalue plot for X')
plt.xlabel('Eigenvalue')
plt.ylabel('Value')
plt.xlim([0, np.size(eigenvalue_vector_x)])

for k in range(1,np.size(eigenvalue_vector_x)):
        plt.scatter(k,eigenvalue_vector_x[k-1])

plt.show()
```

SOM Implementations (Python)

```
# BRAIN  DATA CLASSIFIER V0.7
# UNIVERSITY OF THE RYUKYUS 2015
```

```
#    FACULTY OF ENGINEERING - IT DIVISION
#    ASHARIF-KEN
#    AUTHOR: BRUNO SENZIO-SAVINO BARZELLATO
#    DATE: 14/03/2015

#Import libraries
import numpy as np
import matplotlib.pyplot as plt
import serial.tools.list_ports
import serial
import csv
import time
import io

#Defining data file descriptors for data sorting
fd = ['attention.csv','meditation.csv','delta.csv',
'theta.csv','low_alpha.csv','high_alpha.csv',
'low_beta.csv','high_beta.csv','low_gamma.csv',

'high_gamma.csv']
stream_file = 'stream.csv'
stream_vector_length = 180

#Defining file descriptors for learning data vectors
f_l = ['learning_attention.csv', 'learning_meditation.csv']

#Defining plot color vector
color_vector = ['b','g','r','c','m','y','k','w']

welcome_message ="""BRAIN  DATA CLASSIFIER V0.2 \n
   UNIVERSITY OF THE RYUKYUS 2015 \n
   FACULTY OF ENGINEERING - IT DIVISION \n
   ASHARIF-KEN \n
   AUTHOR: BRUNO SENZIO-SAVINO BARZELLATO \n
   DATE: 14/03/2015\n\n
   """

print welcome_message
```

```
#Displaying available COM PORTS and selecting the right device
#com_ports = list(serial.tools.list_ports.comports())

#while len(com_ports) == 0:
#        com_ports = list(serial.tools.list_ports.comports())
#        print "NO COM PORTS AVAILABLE,
PLEASE CONNECT DEVICE AND PRESS ENTER"
#        raw_input()

#print "Please select from the list the device you want to utilize"

#for i in range(len(com_ports)):
#        print str(i+1) + ". " + str(com_ports[i][1])

port_number = 4  #raw_input()

#Define serial port as Arduino virtual SERCOM from FTDI
ser_com = serial.Serial(3)
print "Found connection at PORT " + ser_com.name
command = ser_com.readline()


#Obtaining stream and dividing
print "Press the red button when ready"
while command[0] != 'p':
        command = ser_com.readline()

while command[0] == 'p':
        command = ser_com.readline()


#The dataset is divided in 11 parts: poorSignalLevel, attention,
meditation, delta, theta, lowAlpha,
highAlpha, lowBeta, highBeta, lowGamma, highGamma
#If raw EEG is in the package,
the general values are transmitted at fs = 512 Hz
#The stream comes from FFT to a preconditioned signal

#Writing the stream into a file
```

```
with io.open(stream_file,'w') as stream:
        #while command[0] != 'p':
        #Streaming total game data
        for x in range(stream_vector_length):
                print str(x)+ ":" + command
                command = ser_com.readline()
                if command[0] != 'p':
                        stream.writelines(unicode(command))
stream.close()
ser_com.close()

#writing the stream data into the corresponding csv files
files = [open(fd[i],'w') for i in range(len(fd))]
with open(stream_file,'r') as input:
        for line in input:
                data_package = line.split(',')
                for j in range(1,10):
                files[j-1].write(unicode(data_package[j])+'\n')

#closing the files
for f in files:
        f.close()

#Declaring individual plot vectors for attention and meditation
attention = []
meditation = []

#Defining learning structure for file manipulation
learning_vectors = [attention, meditation]

#Creating plotting vectors
learning_vector_files = [open(fd[i],'rb') for i in range(2)]

for x in range(len(learning_vector_files)):
        c = csv.reader(learning_vector_files[x])
        for row in c:
                learning_vectors[x].append(row)

for f in learning_vector_files:
```

```
        f.close()

#Creating resulting vectors from stream
X = []
Y = []

R_vectors=[X,Y]

for a in range(len(R_vectors)):
        for b in range(len(learning_vectors[a])):
                R_vectors[a].append(int(learning_vectors[a][b][0]))

#Writing resulting vectors on the respective files
l_files = [open(f_l[i], 'w') for i in range(len(f_l))]

for y in range(len(l_files)):
          l_files[y].write(unicode(R_vectors[y]))

for f in l_files:
        f.close()

print learning_vectors[0]
print learning_vectors[1]

num = len(attention)

#Plotting learning vectors
plt.title('TEST RESULTS')
plt.text(110, 10, 'Blue: Attention')
plt.text(110, 5, 'Green: Meditation')
plt.grid(True)
plt.xlabel('time[s]')
plt.ylabel('Power %')
plt.xlim([0,stream_vector_length])
plt.ylim([0,100])
for p in range(len(learning_vector_files)):
        plt.plot(range(num),learning_vectors[p],color_vector[p])

plt.show()
```

Apendix F: Parallel b-SOM/RBF-SVM with OpenMPI

```
## Parallel b-SOM/RBF-SVM Implementation for Python
## V. 0.8
## Author: Bruno Senzio-Savino
## Asharif Laboratory
## December 2016

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import somoclu
import time
from sklearn import svm

## LOAD DATA FOR A and B, check previous appendix to check values
on paper

start_time=time.time()

alfa = 0.001
Xt=(((A*A)*(A-B))/(A+B))*alfa


# Test vector generation for random cross-validation
# Random combination of features

#A_1=np.array([
#[70, 90, 100, 97, 90, 88, 78, 81, 84, 51, 51, 24, 1, 14, 14, 35,
#57, 64, 60, 69, 90, 87, 100, 91, 90, 97, 91, 100, 100, 83, 67,51,
#50, 56, 66, 67, 50, 66, 54, 64, 69, 53, 75, 70, 63, 70, 63, 67,
#90, 93, 100, 97, 97, 100, 100, 100,  87, 63, 54, 64, 69, 70, 74,
#66, 57, 78, 77, 77, 70, 43, 47, 44, 63, 70, 66, 63, 56, 53, 44,
#44, 53, 69, 69, 70, 77, 69, 80, 70, 67, 67, 66, 66, 54, 67,  64,
#61, 67, 56, 54, 64, 69, 67, 75, 81, 66, 64, 60, 66, 78, 70, 74,
#64, 50, 60, 53, 48, 50, 41, 40, 35, 34, 37, 53, 64, 78, 69, 56,
#47, 50, 63, 75, 83, 78, 78, 66, 63, 61, 66, 74, 80, 87, 84, 83,
#83, 66, 70, 70, 61, 66, 48, 41, 27, 26, 27, 34, 47, 44, 51, 50,
#54, 77, 80, 80, 70, 61, 63, 67, 69, 60, 47,  26, 27, 37, 44, 67,
#67, 54, 57, 54, 41],
```

```
#])
#
#B_1=np.array([
#[34, 37, 17, 10, 21, 35, 43, 61, 74, 66, 93, 100, 100, 87, 64,
#35, 38, 53, 61, 69, 60, 56, 60, 61, 51, 38, 27, 34, 41, 54, 66,
#63, 63, 63, 56, 60, 40, 37, 30, 11, 27, 14, 3, 17, 14, 35, 53,
#34, 57, 69, 69, 100, 100, 88, 80, 70, 69, 83, 97, 87, 53, 27,
#10, 16, 21, 29, 26, 23, 29, 24, 26, 27, 21, 17, 20, 13, 17, 21,
#37, 57, 66, 78, 77, 84, 90, 78, 61, 38, 29, 23, 30, 35, 40, 43,
#38, 35, 20, 24, 30, 30, 40, 35, 17, 17, 11, 13, 34, 37, 41, 30,
#23, 11, 16, 34, 53, 84, 100, 96, 93, 83, 56, 48, 43, 40, 60, 81,
#78, 61, 40, 20, 27, 56, 56, 61, 60, 60, 57, 40, 21, 4, 14, 27,
#34, 26, 21, 21, 21, 11, 10, 4, 8, 21, 14, 10, 7, 1, 14, 23, 26,
#35, 34, 38, 34, 29, 29, 30, 21, 17, 10, 11, 23, 34, 37, 34, 47,
#57, 60, 60, 43, 24],
#])

#Test feature X obtention
#X_1=(((A_1*A_1)*(A_1-B_1))/(A_1+B_1))*alfa

#Xt_new=np.concatenate((Xt,X_1))

# b-SOM Parameters

n_rows, n_columns = 20, 20
data = Xt

#Generation and training with Somoclu

som = somoclu.Somoclu(n_columns, n_rows, data=data)
som.train()

#Label identification for trained parameters

train_labels=([-1, -1, 1, -1, 1, 1, -1, -1, -1, -1, 1, -1, 1,
-1, 1, -1, -1,-1, -1, -1, -1, -1, 1, -1, -1, 1, -1, 1, 1,1])
train_colors=(["red", "red", "green", "red", "green", "green",
"red", "red", "red", "red", "green", "red", "green", "red",
"green", "red", "red", "red", "red", "red", "red", "red",
```

```
"green", "red","red", "green", "red", "green", "green", "green"])

labels=train_labels

#Map Visualization

#som.view_umatrix(bestmatches=True, bestmatchcolors=train_colors,
labels=labels)

# Obtain new BMU for Classification
#Xt[29]=(((A_1*A_1)*(A_1-B_1))/(A_1+B_1))*alfa
#
#som.update_data(Xt)
#som.train()
#som.view_umatrix(bestmatches=True, bestmatchcolors=train_colors,
labels=labels)
#

#Obtain main BMUs
bmu=som.bmus

# RBF-SVM Parameters
h = .02  # step size in the mesh
C = 1.0  # SVM regularization parameter

# In order to visualize better, array is declared (can be
initialized with cycle on interpreter)
#cycle on interpreter version)
X=np.array([[np.asscalar(bmu[0][0]),-np.asscalar(bmu[0][1])],
            [np.asscalar(bmu[1][0]),-np.asscalar(bmu[1][1])],
            [np.asscalar(bmu[2][0]),-np.asscalar(bmu[2][1])],
            [np.asscalar(bmu[3][0]),-np.asscalar(bmu[3][1])],
            [np.asscalar(bmu[4][0]),-np.asscalar(bmu[4][1])],
            [np.asscalar(bmu[5][0]),-np.asscalar(bmu[5][1])],
            [np.asscalar(bmu[6][0]),-np.asscalar(bmu[6][1])],
            [np.asscalar(bmu[7][0]),-np.asscalar(bmu[7][1])],
            [np.asscalar(bmu[8][0]),-np.asscalar(bmu[8][1])],
            [np.asscalar(bmu[9][0]),-np.asscalar(bmu[9][1])],
            [np.asscalar(bmu[10][0]),-np.asscalar(bmu[10][1])],
```

```
            [np.asscalar(bmu[11][0]),-np.asscalar(bmu[11][1])],
            [np.asscalar(bmu[12][0]),-np.asscalar(bmu[12][1])],
            [np.asscalar(bmu[13][0]),-np.asscalar(bmu[13][1])],
            [np.asscalar(bmu[14][0]),-np.asscalar(bmu[14][1])],
            [np.asscalar(bmu[15][0]),-np.asscalar(bmu[15][1])],
            [np.asscalar(bmu[16][0]),-np.asscalar(bmu[16][1])],
            [np.asscalar(bmu[17][0]),-np.asscalar(bmu[17][1])],
            [np.asscalar(bmu[18][0]),-np.asscalar(bmu[18][1])],
            [np.asscalar(bmu[19][0]),-np.asscalar(bmu[19][1])],
            [np.asscalar(bmu[20][0]),-np.asscalar(bmu[20][1])],
            [np.asscalar(bmu[21][0]),-np.asscalar(bmu[21][1])],
            [np.asscalar(bmu[22][0]),-np.asscalar(bmu[22][1])],
            [np.asscalar(bmu[23][0]),-np.asscalar(bmu[23][1])],
            [np.asscalar(bmu[24][0]),-np.asscalar(bmu[24][1])],
            [np.asscalar(bmu[25][0]),-np.asscalar(bmu[25][1])],
            [np.asscalar(bmu[26][0]),-np.asscalar(bmu[26][1])],
            [np.asscalar(bmu[27][0]),-np.asscalar(bmu[27][1])],
            [np.asscalar(bmu[28][0]),-np.asscalar(bmu[28][1])],
            [np.asscalar(bmu[29][0]),-np.asscalar(bmu[29][1])],
            ])

#SVM labels
y=np.array([0, 0, 1, 0, 1, 1,0,0,0,0, 1,0, 1, 0, 1, 0, 0, 0, 0,0,
0, 0, 1, 0, 0, 1, 0, 1, 1, 1])

rbf_svc = svm.SVC(kernel='rbf', gamma=0.1, C=0.8).fit(X, y)


# create a mesh to plot in
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

# Predict classes
Z = rbf_svc.predict(np.c_[xx.ravel(), yy.ravel()])

# Put the result into a color plot
Z = Z.reshape(xx.shape)
```

```
plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)

# Plot also the training points
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm)

#TEST DATA (Obtained previously with method mentioned above)
plt.scatter(5, -19,color='white')
plt.scatter(7, -9,color='white')
plt.scatter(10, -17,color='green')
plt.scatter(10, -17,color='green')
plt.scatter(11, -3,color='white')
plt.scatter(18, -5,color='white')
plt.scatter(1, -1,color='green')
plt.scatter(12, -2,color='green')
plt.scatter(15, -19,color='green')
plt.scatter(0, -12,color='green')

# Plot the label coordinates
plt.xlabel('BMU - X')
plt.ylabel('BMU - Y')
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.xticks(())
plt.yticks(())
plt.title("RBF Kernel with SOM test vector output C=0.8 gamma =0.1")

#Class colors
red_patch = mpatches.Patch(color='red', label='+1(train)')
blue_patch = mpatches.Patch(color='blue', label='-1(train)')
white_patch = mpatches.Patch(color='white', label='+1(test)')
black_patch = mpatches.Patch(color='green', label='-1(test)')

plt.legend(handles=[red_patch,blue_patch,white_patch,black_patch])
plt.show()

#Show program execution time
print("---%s seconds---" % (time.time()-start_time))
```